Does College Pay Off?

AI-Powered ROI Analytics on Databricks

Project handover & step-by-step rebuild guide

Owner	Shaqran
Built with	Databricks (warehouse, ai_query, Jobs), dbt Core, GitHub, Streamlit, Python/SQL
AI used	Claude Code (AI pair-engineer) + Claude (guide/reviewer) + Llama 3.3 70B via Databricks ai_query
Repository	github.com/shaqran39/college-roi-analytics
Live dashboard	college-roi-analytics-wdsk3fzehd7jx4ncmyr3sr.streamlit.app
Build period	30 Sep to 16 Oct 2026
Status	Complete (Power BI explored, left out on purpose)

Contents

Contents 2

1. Executive summary 3

2. Architecture: how the pieces connect 4

3. Tools, accounts and costs 5

4. Key concepts in plain English 6

5. Decision log: what we chose and what it impacted 7

6. Step-by-step rebuild guide 8

6.0 Phase 0: Laptop setup and Claude Code . . . 8

6.1 Project scaffold and CLAUDE.md . . . 9

6.2 Choosing the dataset . . . 11

6.3 Phase 1: Databricks setup and loading the raw data . . . 11

6.4 GitHub: repository, CLI login and pull requests . . . 12

6.5 Phase 2: Staging model (stg_graduates) . . . 13

6.6 Phase 3: Intermediate model (int_graduates_enriched) . . . 14

6.7 Phase 4: Marts and the star schema . . . 16

6.8 Phase 5: AI report cards and key findings . . . 22

6.9 Phase 6: Public Streamlit dashboard . . . 31

6.10 Phase 7: Automation with a Databricks Job . . . 45

6.11 Power BI (explored, left out on purpose) . . . 66

7. Impact map: if you change X, what happens to Y 70

8. Repository map and key files 71

Models at a glance . . . 71

Final load SQL (ingestion/databricks_load_raw.sql) . . . 72

Job definition (orchestration/college_roi_refresh_job.yml) . . . 72

Dashboard data access (dashboards/db.py) . . . 74

CLAUDE.md (final) . . . 75

9. Security and credentials 77

10. Troubleshooting and lessons learned 78

11. Operations runbook 79

12. How to work with Claude Code (the workflow that worked) 80

13. Findings (for the write-up and interviews) 81

14. Wrap-up checklist and future work 82

Appendix: list of figures 83

1. Executive summary

This project answers one question with data: **which college majors and college types actually pay off over 10 years, and how much do internships, finishing on time and debt change the answer?** It uses a dataset of 30,000 (synthetic) graduates.

It was built as an **AI workflow** in two senses:

- **AI as the builder.** Claude Code wrote, ran, tested and fixed almost all of the SQL and Python. Shaqran acted as the lead: setting goals, reviewing plans, approving changes, testing results and catching mistakes.
- **AI inside the product.** Databricks `ai_query` (Llama 3.3 70B) writes a plain-English report card for each of the 19 majors and a summary paragraph of the key findings, every time the pipeline runs.

What exists at the end:

Piece	What it is	Where
Cloud warehouse	Databricks Free Edition. Raw, staging, intermediate and marts schemas in Unity Catalog	Databricks: workspace catalog
Transformations	dbt Core project: 1 staging, 1 intermediate, star schema (1 fact + 3 dims) + 8 marts, 40+ tests	GitHub: dbt/
AI layer	mart_major_report_cards + mart_key_findings, written by ai_query with guardrails	GitHub: dbt/models/marts/
Dashboard	4-page public Streamlit app reading live from Databricks (Overview, Majors, Colleges & choices, Deep dive with What-if tool)	Streamlit Community Cloud
Automation	Databricks Job: new CSV lands, then load, then dbt build (models + AI + tests), failure email	Databricks Jobs + orchestration/ YAML
Version control	11 pull requests, every change reviewed and merged through a PR	GitHub

Headline findings (synthetic data, group comparisons, not proof of cause):

- Electrical engineering: typical graduate came out **US$525.8K ahead** after 10 years, paying back their costs in **1.3 years**.
- Fine arts: typical graduate was **US$35.5K worse off**; **88.9%** were worse off than not going to college. Humanities also negative.
- Ivy/elite colleges had the highest typical return (US$304K) but more graduates worse off (20.3%) and the most debt (US$80K). For-profit colleges were worst on every measure (36.8% worse off).
- Internships went with +US$116.5K overall, but only +US$2.1K (arts) to +US$82.5K (STEM) within the same field: choice of major inflates the headline number.
- GPA and region barely mattered. More debt went with lower returns (about US$240K for small debts down to about US$125K for US$150K+).

2. Architecture: how the pieces connect

Data flows top to bottom. Each layer only reads from the layer above it, so a fix in one place flows down automatically.

```
college_major_roi_YYYY-MM.csv   (you upload a CSV with a NEW name)
        |
        v
Databricks Volume  /Volumes/workspace/raw/landing/        <- file-arrival trigger watches this f
  older
        |   Job task 1: load_raw  (ingestion/databricks_load_raw.sql, newest file only)
        v
RAW          workspace.raw.college_major_roi              (30,000 rows + _source_file, _loaded_a
  t)
        |   Job task 2: dbt build  (only if task 1 succeeded)
        v
STAGING      workspace.staging.stg_graduates              clean, rename, cap selectivity at 100
        v
INTERMEDIATE workspace.intermediate.int_graduates_enriched  payback_years, debt ratio, roi_band
        v
MARTS        workspace.marts.*
               star schema : fct_graduate_outcomes + dim_major, dim_institution_tier, dim_region
               summaries   : mart_roi_by_major, mart_roi_by_tier, mart_choices_effect,
                             mart_roi_by_band, mart_outcome_spread_by_major, mart_what_if
               AI (ai_query): mart_major_report_cards, mart_key_findings
        |
        +--> 40+ dbt tests (any failure stops the job and emails you)
```

```
        v
Streamlit dashboard (public)  reads workspace.marts with a read-only token, caches 6 hours
```

The two "AI roles", and where each one lives:

Role	Tool	What it did	When it runs
AI as builder	Claude Code (terminal, on the laptop)	Wrote and tested all models, dashboard, job SQL; opened every PR	Only when Shaqran gives it a task
AI in the pipeline	Databricks ai_query + Llama 3.3 70B	Writes 19 report cards + 1 findings paragraph from the marts	Every dbt build (manual or automatic)
Guide / reviewer	Claude (this chat)	Planned phases, explained concepts, reviewed every plan/screenshot, caught bugs and risks	Throughout

3. Tools, accounts and costs

Tool	Why we used it	Cost	Account needed
Python 3.12 + venv	Runs dbt, Streamlit and helper scripts locally	Free	No
Git + GitHub	Version control, pull requests, public portfolio	Free	GitHub (shaqran39)
GitHub CLI (gh)	Lets Claude Code create repos and PRs	Free	Uses your GitHub login
Claude Code	AI pair-engineer that writes/runs code in the project folder	Claude plan	Claude account
Databricks Free Edition	Cloud warehouse, SQL warehouse, ai_query, Jobs	Free (non-commercial, usage limits)	Databricks
dbt Core + dbt-databricks	Turns SQL files into tested, documented tables	Free	No
Streamlit Community Cloud	Hosts the public dashboard from GitHub	Free	Sign in with GitHub
Power BI Desktop	Explored for a second dashboard, not finished	Free (Desktop)	Optional
DuckDB	Local practice database in the first scaffold. Not used after we moved to Databricks	Free	No

Watch out: Databricks Free Edition: one small (2X-Small) serverless SQL warehouse, max 5 concurrent job tasks, some AI models unavailable, non-commercial use only, and inactive accounts can be deleted. The warehouse "sleeps", so the first query after a while takes up to ~30 seconds.

4. Key concepts in plain English

Term	Plain-English meaning	Where it shows up here
AI workflow	A loop: you give a goal, the AI plans, writes code, runs it, reads errors, fixes, and you review.	Every phase
Claude Code	An AI agent that works inside your project folder: it can read files, run commands and edit code (with permission).	Built all models and the dashboard
CLAUDE.md	The rulebook Claude Code reads at the start of every session (project facts, naming rules, what not to do).	Root of the repo
Plan mode / auto mode	Plan mode: Claude only plans until you approve. Auto mode: it acts without asking. Shift+Tab cycles modes.	Used plan mode for each phase
Databricks	A cloud data platform: storage, compute, SQL warehouse, notebooks, jobs, AI functions.	Holds all tables
dbt	Organises SQL into layered models (staging, intermediate, marts) with tests and docs. Runs on top of Databricks.	dbt/ folder
Staging / intermediate / mart	Clean the raw data / add business logic once / build answer-ready tables for dashboards.	Phases 2 to 4
Star schema	A central fact table (one row per graduate, the numbers) linked to dimension tables (descriptions like major, tier, region).	Phase 4
Test (dbt)	A query that must return nothing; if it returns rows, the build fails. E.g. unique IDs, valid values.	40+ tests
Branch / PR / merge	A safe copy of the code / a request to add it to main with a visible diff / accepting it.	PRs #1 to #11
Personal access token	A password-like key that lets a tool connect to Databricks. Can be scoped (e.g. SQL only).	dbt, Streamlit, Power BI
ai_query	A Databricks SQL function that sends a prompt to an AI model and returns text into a column.	Phase 5
Hallucination	When an AI states something that is not in the data. Checked for at every AI step.	Phase 5
File-arrival trigger	A Databricks Job setting that starts the job when a new file appears in a folder.	Phase 7
Least privilege	Give each tool only the access it needs (e.g. the dashboard key can only run SQL).	Tokens

5. Decision log: what we chose and what it impacted

When	Decision	Why	What it impacted
30 Sep	Stack: dbt + a warehouse + a dashboard, built with Claude Code	Modern data stack, strong for DA/BA/junior DE roles	Everything after
1 Oct	dbt vs Databricks: use both (dbt runs on Databricks)	They do different jobs: Databricks = platform, dbt = transformation	dbt-databricks adapter, prod target
1 Oct	Project folder at Documents\ai-data-warehouse	Simple, local, then pushed to GitHub	All paths in this guide
1 Oct	Olist e-commerce dataset (first plan)	Rich multi-table data	Superseded on 6 Oct
1 Oct	Cloud-based + AI in both roles	Shaqran wanted it public and AI-powered	Added Streamlit + ai_query to the plan
1 Oct	Databricks Free Edition as the warehouse	Big CV name; free	Free Edition limits shaped Phases 5-7
6 Oct	Switched dataset to college_major_roi.csv	Shaqran uploaded this file; relatable story	Whole project rewritten for 1 table; business question changed
8 Oct	Work through branches + PRs	Safe changes, visible history	Every phase ends with PR + merge
13 Oct	Readable natural keys (major names) in the star schema	Only 19/5/4 values, easier to read	Interview answer: surrogate keys at scale
13 Oct	Visuals: Power BI + Streamlit	Power BI for CV, Streamlit for public link + AI	Power BI later dropped
14 Oct	AI takeaway labels decided by SQL rules, AI only writes wording	AI called risky majors "a solid choice"	Consistent, checkable verdicts
14 Oct	Build on prod only, drop local DuckDB builds	CSV lives only in Databricks	CLAUDE.md updated
15 Oct	Separate read-only (BI Tools/sql) tokens per tool	Least privilege	Safer public dashboard
15 Oct	No free-text "Ask the data" box on the public site	Prompt-injection and cost risk	README promise removed
15 Oct	Automate with a Databricks Job on file arrival, no retries, failure email	Hands-off refresh, fail loudly, no wasted AI calls	Phase 7
16 Oct	Load the newest CSV, upload new data with new file names	Same-name re-uploads do not fire the trigger	Refresh procedure
16 Oct	Power BI left out of this project	Shaqran wants to learn it properly first	Final scope and resume entry

6. Step-by-step rebuild guide

Follow the phases in order. Each phase lists: **goal**, **what you do**, **what to tell Claude Code**, **what came out**, **what went wrong and how it was fixed**, and **what it impacts later**. Screenshots are from the real build (hostnames, emails and IDs are blacked out).

6.0 Phase 0: Laptop setup and Claude Code

Goal: a working project folder, Python environment and Claude Code on Windows.

1. Open the project folder in File Explorer, type `powershell` in the address bar to open a terminal there.
2. Check `python --version` (3.10 to 3.12) and `git --version`. Install from python.org / git-scm.com if missing.
3. Create and activate a virtual environment, then install packages:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt        # duckdb, dbt-core, dbt-duckdb, dbt-databricks,
                                       # databricks-sql-connector, pandas, plotly, streamlit, jupy
    ter
```

1. Install Claude Code: `irm https://claude.ai/install.ps1 | iex`

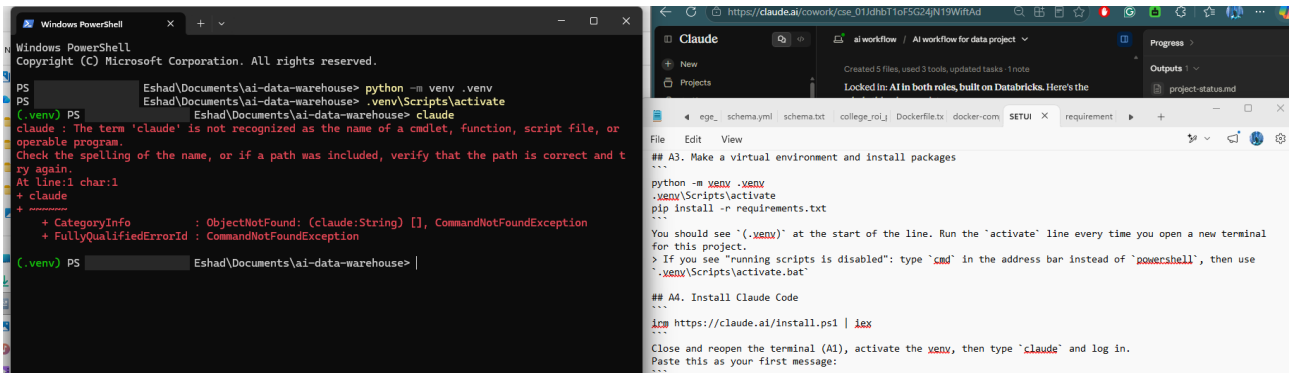


Figure 1. First attempt: typing claude failed with "not recognized" because Claude Code had not been installed yet (and pip install had been skipped).

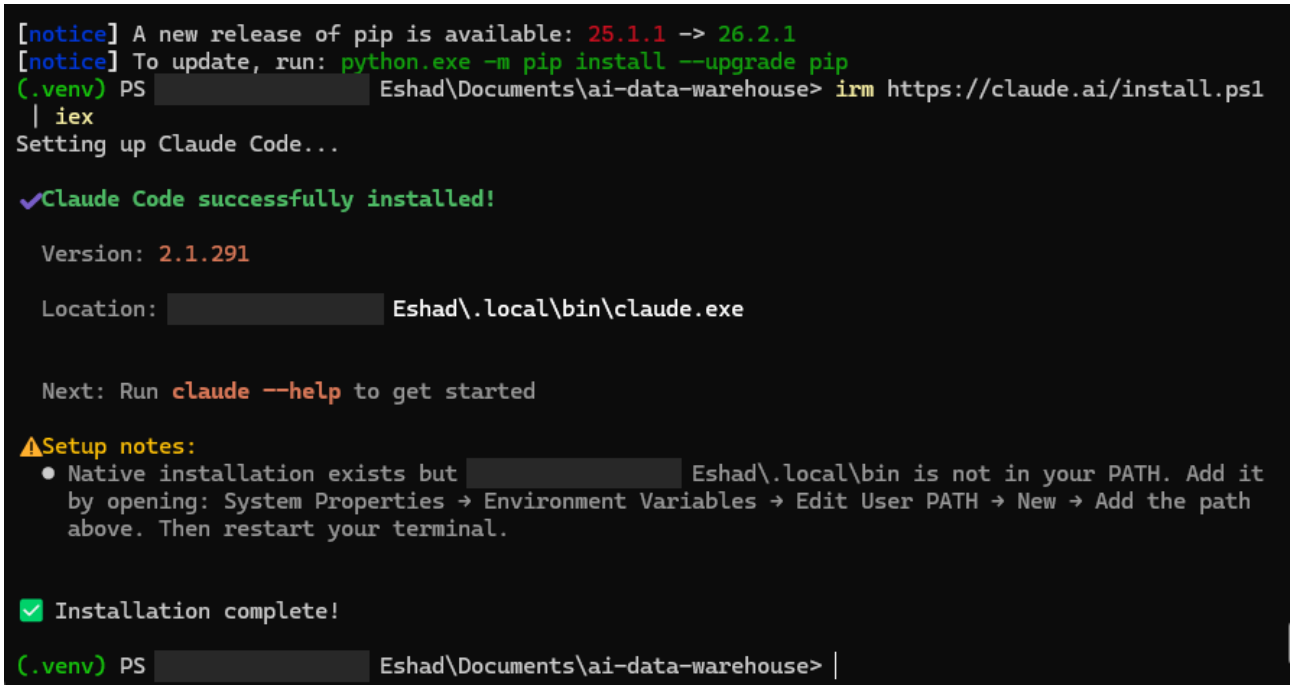


Figure 2. Claude Code installed, but the installer warned that C:\Users\...\.local\bin was not on PATH.

Fix for the PATH warning (run once, then close and reopen PowerShell):

```
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User"
    ) + ";$env:USERPROFILE\.local\bin", "User")
```

Then: `.venv\Scripts\activate` and `claude`, log in with your Claude account.

Watch out: If you activate the venv in PowerShell and see "running scripts is disabled", open Command Prompt instead and use .venv\Scripts\activate.bat.

6.1 Project scaffold and CLAUDE.md

The starter project was generated (by Claude in chat) and written into the folder. Key files:

File / folder	Purpose	Impact
CLAUDE.md	Rules Claude Code follows: stack, naming, layer rules, "explain simply", "never commit secrets"	Every Claude Code session obeys it; update it when the project changes
README.md	Public project page + roadmap checklist	The /next-step command reads the roadmap
SETUP_GUIDE.md	Beginner setup steps A to D	Used for Databricks/dbt setup
.claude/commands/explain.md, next-step.md	Custom slash commands /explain and /next-step	Learning aid; /next-step reads README
ingestion/	SQL that loads the CSV into Databricks	Task 1 of the Job
dbt/	dbt project: models, tests, macros, profiles.yml	Everything in the warehouse
dashboards/	Streamlit app	Public site
.gitignore	Keeps data, venv and secrets out of Git	Security

First message to Claude Code (moves the shortcut files into .claude, which remote tools are not allowed to write):

Prompt given to Claude Code: `Move the two files in _setup/claude-commands/ into .claude/commands/ and delete the _setup folder. Then read CLAUDE.md and README.md and explain this project back to me simply.`

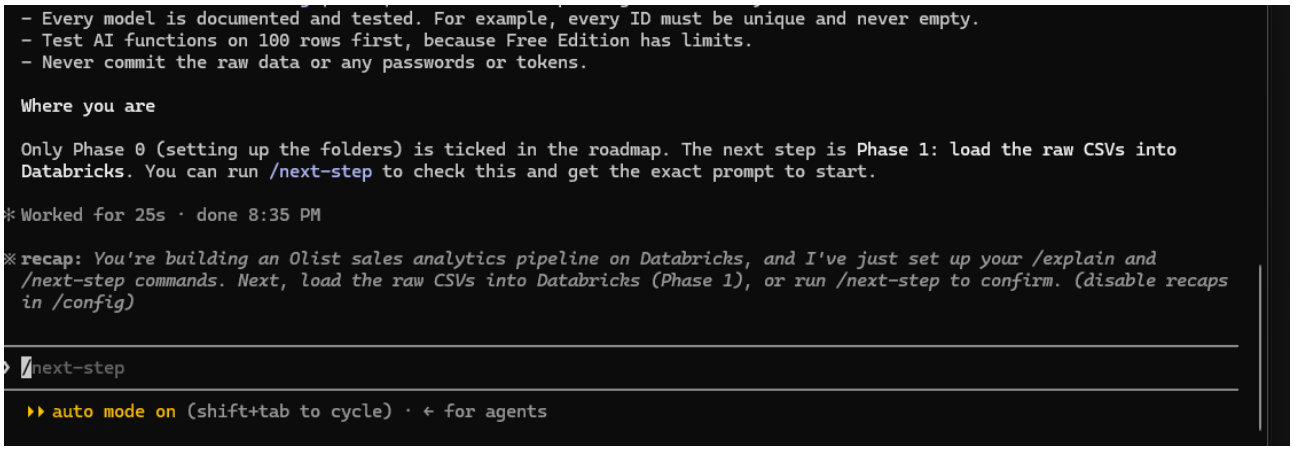


Figure 3. Claude Code explains the project back and points to /next-step. Note "auto mode on" at the bottom: switch it off with Shift+Tab while learning.

Stale context: Claude Code read the README early in its session, then the README was changed outside the session. It proposed rewriting a file that was already correct because it was working from a stale copy. Fix: tell it "I updated README.md outside this session. Re-read it from disk." Lesson: the AI only knows what it has read.

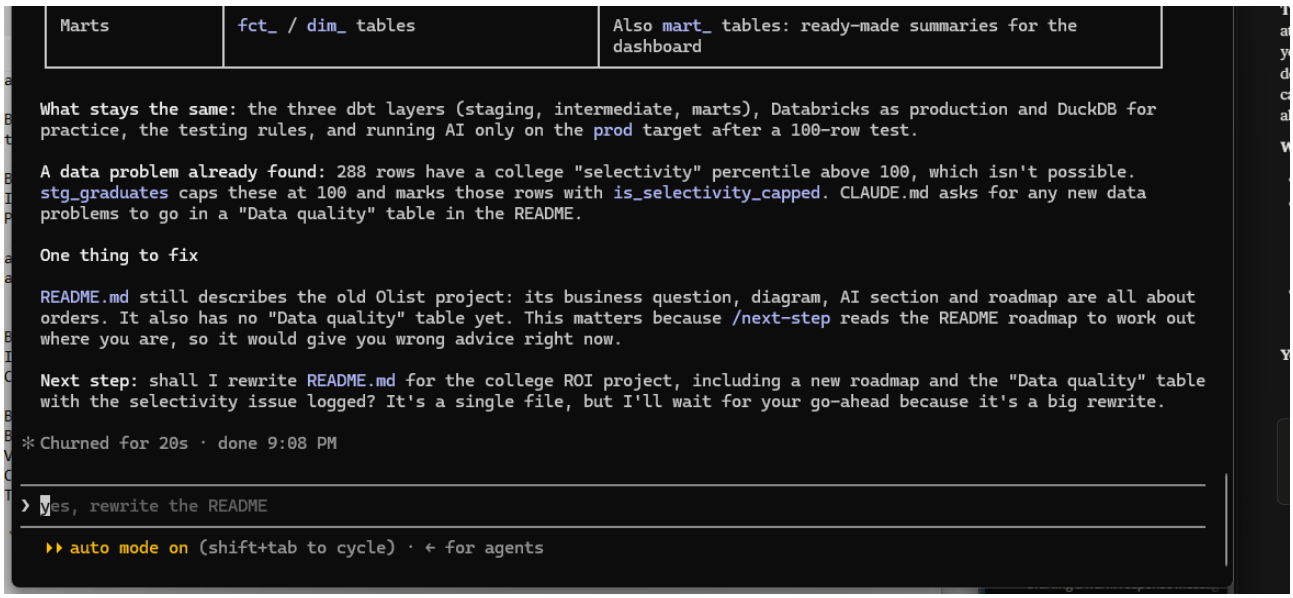


Figure 4. Claude Code, working from an old copy, wrongly says the README still describes the old project. It had already been updated on disk.

6.2 Choosing the dataset

The first plan used the Olist e-commerce dataset (9 tables). On 6 Oct Shaqran uploaded `college_major_roi.csv` instead and chose to switch.

Property	Value
Rows / columns	30,000 graduates x 18 columns, one row per graduate
Columns	grad_id, major (19), major_category (7), institution_tier (5), institution_selectivity_pctile, gpa, had_internship, completed_on_time, region (4), net_cost_usd, debt_usd, hs_baseline_10yr_usd, added_earnings_10yr_usd, earnings_10yr_usd, net_roi_usd, roi_pct, positive_roi, high_roi
Nature	Synthetic (simulated) data; say so on the dashboard and in interviews
Quality issue found on day 1	288 rows have selectivity percentile above 100 (impossible). Capped at 100 and flagged in staging
Second issue (found later)	Money columns rounded to nearest US$100, so formulas are off by exactly US$100 in ~7,500 rows. Expected; a test allows it

Impact: the business question, every model name, the README, CLAUDE.md and the load SQL were rewritten for one table. The star schema came later from splitting this one table into a fact and dimensions.

6.3 Phase 1: Databricks setup and loading the raw data

1. Sign up at databricks.com/learn/free-edition.
2. SQL Editor: `create schema if not exists workspace.raw; create volume if not exists workspace.raw.landing;`
3. Catalog, workspace, raw, Volumes, landing: **Upload to this volume** the CSV.
4. Run the load SQL (ingestion/databricks_load_raw.sql) in the SQL Editor. The first version read one fixed file; the final version (section 6.10) reads the newest file. Result: 30,000 rows.
5. Connection details: SQL Warehouses, your warehouse, Connection details: copy **Server hostname** and **HTTP path**.
6. Create a token for dbt: profile icon, Settings, Developer, Access tokens, Generate new token (name dbt-laptop, lifetime 90 days, Other APIs / all-apis).

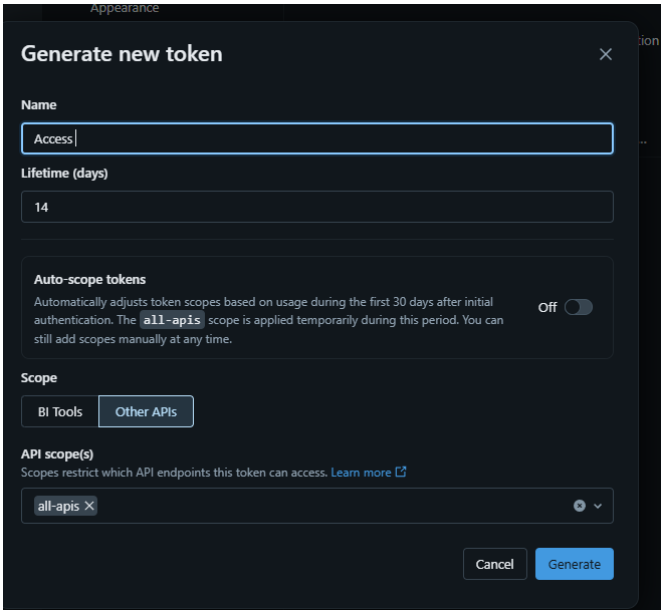


Figure 5. Token dialog for the dbt token. Rename it (e.g. dbt-laptop) and set lifetime to 90 days so it does not expire mid-project.

Save the three values as Windows environment variables in a **separate** PowerShell window (never paste a token into any chat):

```
setx DATABRICKS_HOST "dbc-xxxx.cloud.databricks.com"
setx DATABRICKS_HTTP_PATH "/sql/1.0/warehouses/xxxx"
setx DATABRICKS_TOKEN "dapi..."          # then close and reopen the terminal

cd dbt
dbt debug --target prod     # All checks passed!
dbt build --target prod     # PASS=8 on the first staging model
```

dbt/profiles.yml reads those variables, so no secret is ever in a file:

```
# Two places dbt can build the warehouse:
#   dev  = DuckDB on your laptop (free, fast, for practice):  dbt build
#   prod = Databricks Free Edition in the cloud:               dbt build --target prod
#
# Databricks secrets come from environment variables, NEVER typed into this file.
# See SETUP_GUIDE.md Part B for how to set them.
college_roi:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: ../warehouse/college.duckdb
      threads: 4

    prod:
      type: databricks
      catalog: workspace
      schema: analytics
      host: "{{ env_var('DATABRICKS_HOST') }}"
      http_path: "{{ env_var('DATABRICKS_HTTP_PATH') }}"
      token: "{{ env_var('DATABRICKS_TOKEN') }}"
      threads: 4
```

Watch out: setx changes only apply to NEW terminal windows. If dbt says an env var is missing, close and reopen PowerShell. Claude Code must also be restarted (/exit, then claude) to see new variables.

6.4 GitHub: repository, CLI login and pull requests

Prompt given to Claude Code: `Set up git for this project and push it to a new public GitHub repo called college-roi-analytics under my account shaqran39. Check .gitignore first so no data or secrets get uploaded.`

Claude Code installed the GitHub CLI and asked you to log in (this has to be done by you, in a browser):

```
& "C:\Program Files\GitHub CLI\gh.exe" auth login --web --git-protocol https
```

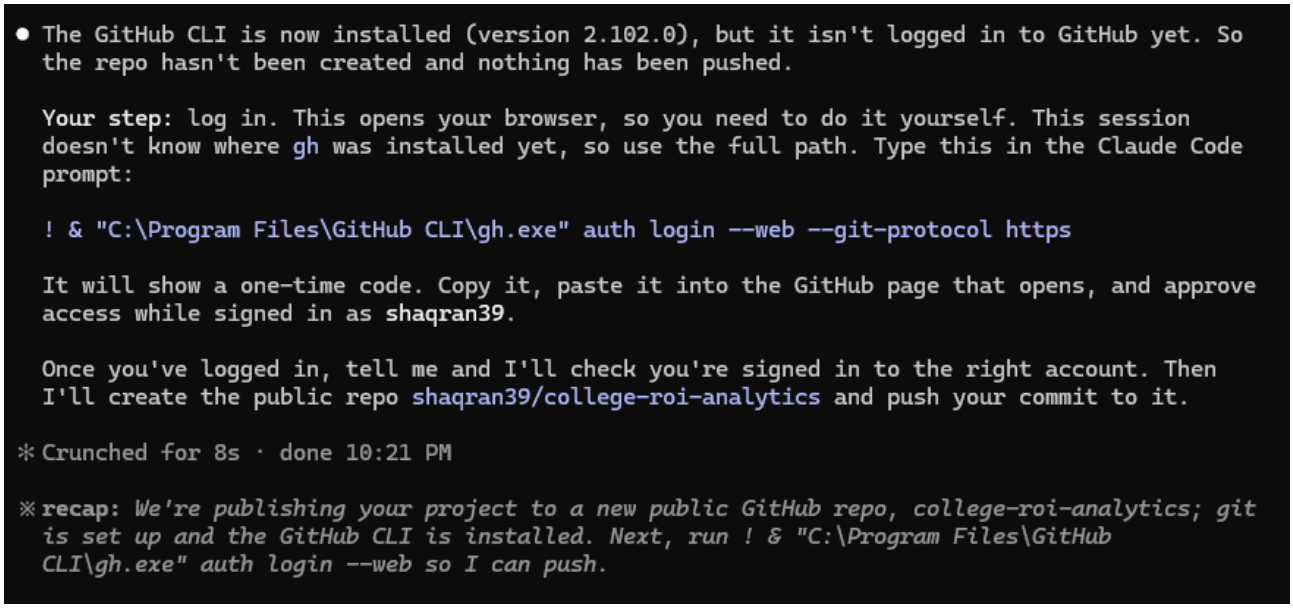


Figure 6. Claude Code installed the GitHub CLI and handed the login step back to the human.

If you shut the PC mid-way, nothing is lost: files, git commits, Databricks data and setx variables persist. Resume the last Claude Code chat with `claude --continue`.

The PR habit used for every phase: Claude works on a new branch, commits, opens a PR (gh pr create), you review, it merges, then it switches back to main and pulls. 11 PRs were merged in total.

6.5 Phase 2: Staging model (stg_graduates)

Goal: clean, renamed, typed version of the raw table. No joins.

```
-- One row per graduate. Rename columns, turn 0/1 flags into true/false,
-- and fix the known data issue: selectivity percentiles above 100.

with source as (

    select * from {{ source('college', 'college_major_roi') }}

),

cleaned as (

    select
        -- ids
        grad_id,

        -- education
        major,
        major_category,
        institution_tier,
        least(institution_selectivity_pctile, 100)   as selectivity_pctile,
        institution_selectivity_pctile > 100         as is_selectivity_capped,
        cast(gpa as decimal(3, 2))                    as gpa,
```

```
        had_internship = 1                          as has_internship,
        completed_on_time = 1                       as is_completed_on_time,
        region,

        -- money (USD)
        net_cost_usd,
        debt_usd,
        hs_baseline_10yr_usd,
        added_earnings_10yr_usd,
        earnings_10yr_usd,
        net_roi_usd,
        roi_pct,
        positive_roi = 1                            as is_positive_roi,
        high_roi = 1                                as is_high_roi

    from source

)

select * from cleaned
```

Tests: unique + not_null on grad_id, accepted_values on major_category, institution_tier and region, and a custom range test (selectivity 0-100, GPA 0-4, no negative cost or debt). All 8 passed on the real file.

Why this matters: Downstream models never see an impossible percentile. is_selectivity_capped keeps the fix visible. Every later layer reads stg_graduates, so this is the single place data is cleaned.

6.6 Phase 3: Intermediate model (int_graduates_enriched)

Prompt given to Claude Code: `Phase 3: build an intermediate model int_graduates_enriched from stg_graduates. Add useful columns for the analysis, like a debt-to-earnings ratio and an ROI band (negative / low / medium / high). Show me your plan first and explain each new column simply. Once I approve, build it with docs and tests and run dbt build --target prod.`

```
-- One row per graduate. Keeps every stg_graduates column and adds
-- easy-to-read measures: yearly earnings, debt vs salary, payback time and an ROI band.

with graduates as (

    select * from {{ ref('stg_graduates') }}

),

enriched as (

    select
        *,

        -- yearly view of the 10-year totals (USD)
        round(earnings_10yr_usd / 10.0, 0)                              as avg_annual_earnings_
    usd,
        round(added_earnings_10yr_usd / 10.0, 0)                        as avg_annual_added_ear
    nings_usd,

        -- debt
        debt_usd > 0                                                    as has_debt,
```

```
        round(debt_usd / nullif(earnings_10yr_usd / 10.0, 0), 2)          as debt_to_annual_earni
    ngs_ratio,

        -- years of the yearly earnings premium needed to earn back the net cost
        round(net_cost_usd / nullif(added_earnings_10yr_usd / 10.0, 0), 1) as payback_years,

        -- built on the dataset's own flags so the band always agrees with them
        case
            when is_high_roi          then 'high'
            when not is_positive_roi  then 'negative'
            when net_roi_usd < 200000 then 'low'
            else 'medium'
        end                                                               as roi_band

    from graduates

)

select * from enriched
```

New column	Meaning
avg_annual_earnings_usd	10-year earnings / 10
has_debt	Borrowed anything?
debt_to_annual_earnings_ratio	Debt / one year of pay (0.5 = half a year of pay)
payback_years	Years of extra pay needed to earn back net cost
roi_band	negative / low (< US$200K) / medium / high (top quartile, matches is_high_roi)

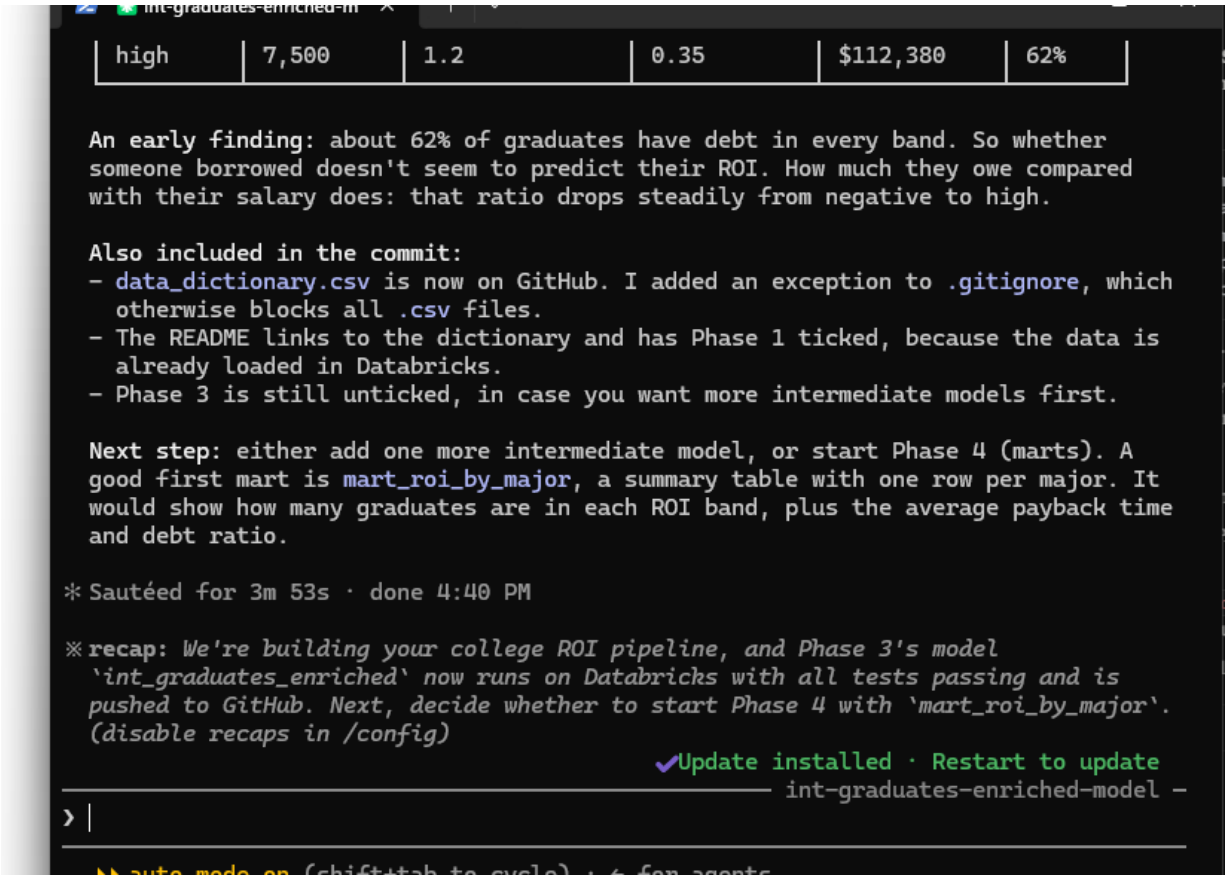


Figure 7. Phase 3 result: ROI bands summarised, early finding that about 62% of graduates have debt in every band. Claude also added data_dictionary.csv.

Why this matters: Business logic defined once here means "high ROI" means the same thing in every mart, chart and AI prompt.

Watch out: Claude worked on a branch and, with auto mode on, did extra work without asking. Turn auto mode off (Shift+Tab) and always ask for the PR to be merged so main stays current.

6.7 Phase 4: Marts and the star schema

Prompt given to Claude Code: `Re-read CLAUDE.md and README.md first. Tick Phase 3 in the README. Then Phase 4: on a new branch, build mart_roi_by_major (one row per major) with grad count, median net ROI, % of grads in each ROI band, median payback years, % not paid back within 10 years, and median debt-to-earnings ratio. Plan first and explain each column simply. After I approve: build with docs and tests, run dbt build --target prod, show me the top 5 and bottom 5 majors, then open a PR and merge it.`

add up to 100, and no graduate was lost or counted twice.

Top 5 majors (by the middle graduate's 10-year gain)

Major	Median net ROI	% negative	% high	Median payback	Debt ÷ yearly pay
Electrical engineering	$525,800	0.1%	66.3%	1.3 yrs	0.38
Business analytics	$479,500	0.0%	58.6%	1.4 yrs	0.40
Chemical engineering	$466,650	0.3%	57.2%	1.5 yrs	0.39
Computer science	$464,750	0.2%	56.3%	1.5 yrs	0.37
Computer engineering	$446,600	0.1%	53.7%	1.5 yrs	0.40

Bottom 5 majors

Figure 8. mart_roi_by_major, top 5: engineering and tech pay back in about 1.3 to 1.5 years.

Bottom 5 majors

Major	Median net ROI	% negative	% high	Median payback	Debt ÷ yearly pay
Fine arts	-$35,500	88.9%	0%	18.9 yrs	0.94
Humanities	-$11,200	62.4%	0%	11.7 yrs	0.92
Psychology	$1,800	48.3%	0%	9.8 yrs	0.84
Social work	$5,050	45.1%	0%	9.4 yrs	0.84
Education	$8,750	41.9%	0%	9.0 yrs	0.81

In plain words: engineering and tech grads typically earn back their college cost in about 1.5 years and have little debt compared with their pay. Most fine arts grads, and most humanities grads, end up worse off after 10 years than if they had skipped college. Their debt is also close to a full year's salary.

As expected, "% not paid back in 10 years" is almost the same as "% negative ROI". For 9 majors they differ, by at most 0.7 points. The gap is grads who broke Jump to bottom (ctrl+End) ↓ yback rounds to exactly 10.0 years, so they

✔Update installed · Restart to update

tick-phase-3-mart-roi –

Figure 9. Bottom 5: fine arts and humanities have negative typical returns and debt close to a full year of pay.

Then two more summary marts:

Prompt given to Claude Code: `Make sure the mart_roi_by_major PR is merged into main and switch back to main. Then on a new branch, build 2 more marts in the same style:`
`1. mart_roi_by_tier: one row per institution tier with the same metrics, plus median net cost and median debt.`
`2. mart_choices_effect: compare median net ROI, % negative ROI and median payback for grads WITH vs WITHOUT an internship, and who finished on time vs late, ideally split by major category.`
`Plan first. After I approve: build with docs and tests, run dbt build --target prod, show me the key results, tick Phase 4 in the README, then open a PR and merge it.`

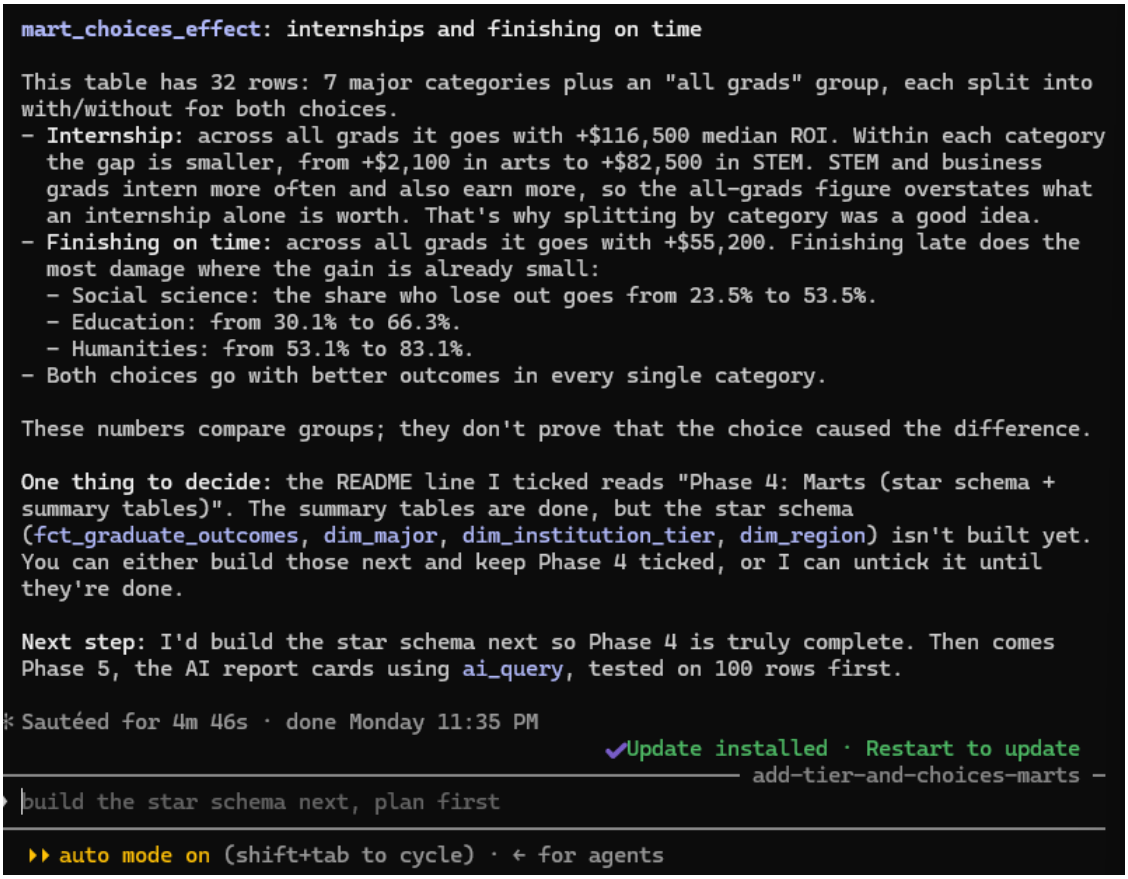


Figure 10. mart_choices_effect: the internship gap shrinks once you compare within the same field (a confounding variable). Claude notes group differences are not proof of cause.

Then the star schema:

Prompt given to Claude Code: `Build the star schema next, plan first. Before starting, make sure the tier and choices marts PR is merged into main. Explain simply what a star schema is and why we need it.`

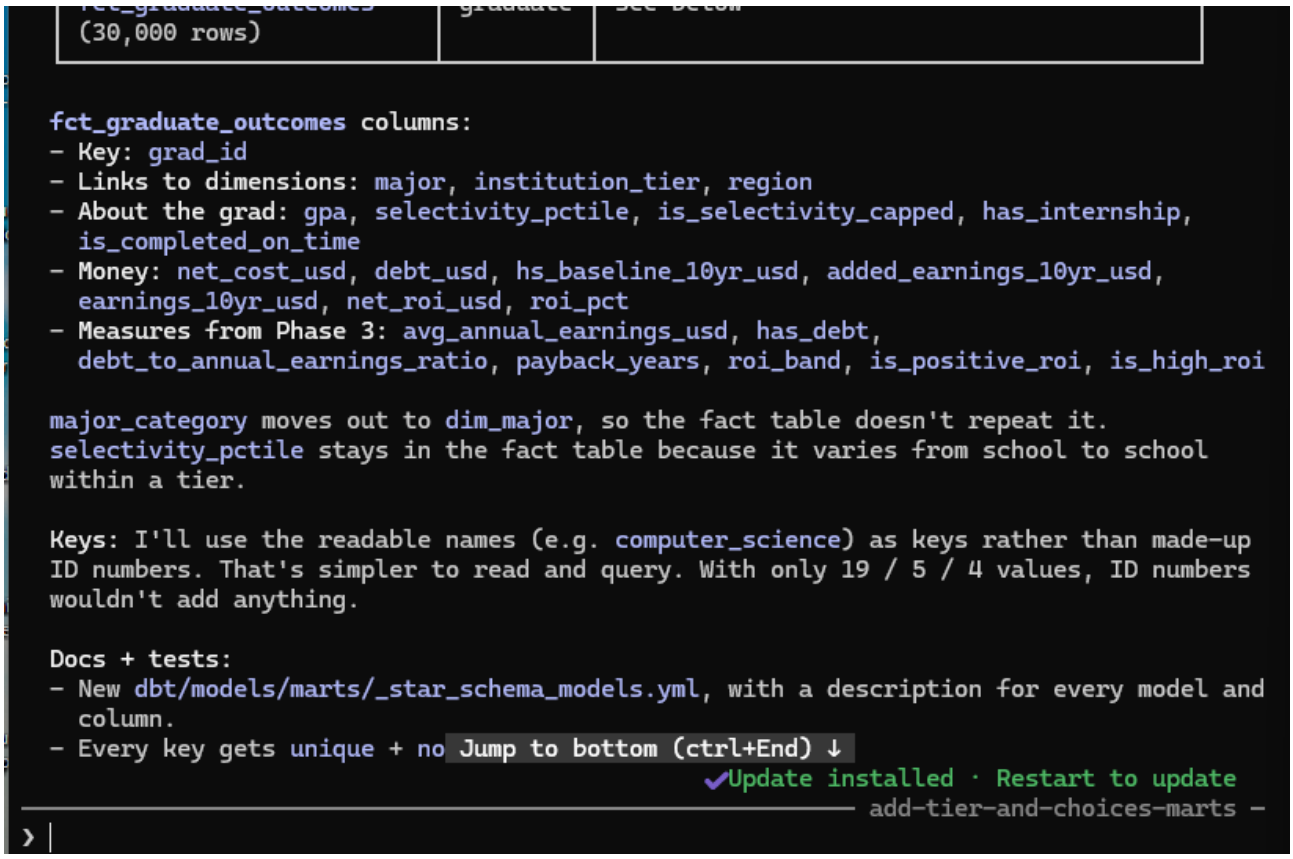


Figure 11. Star schema plan, part 1: fct_graduate_outcomes columns and the decision to use readable names as keys.

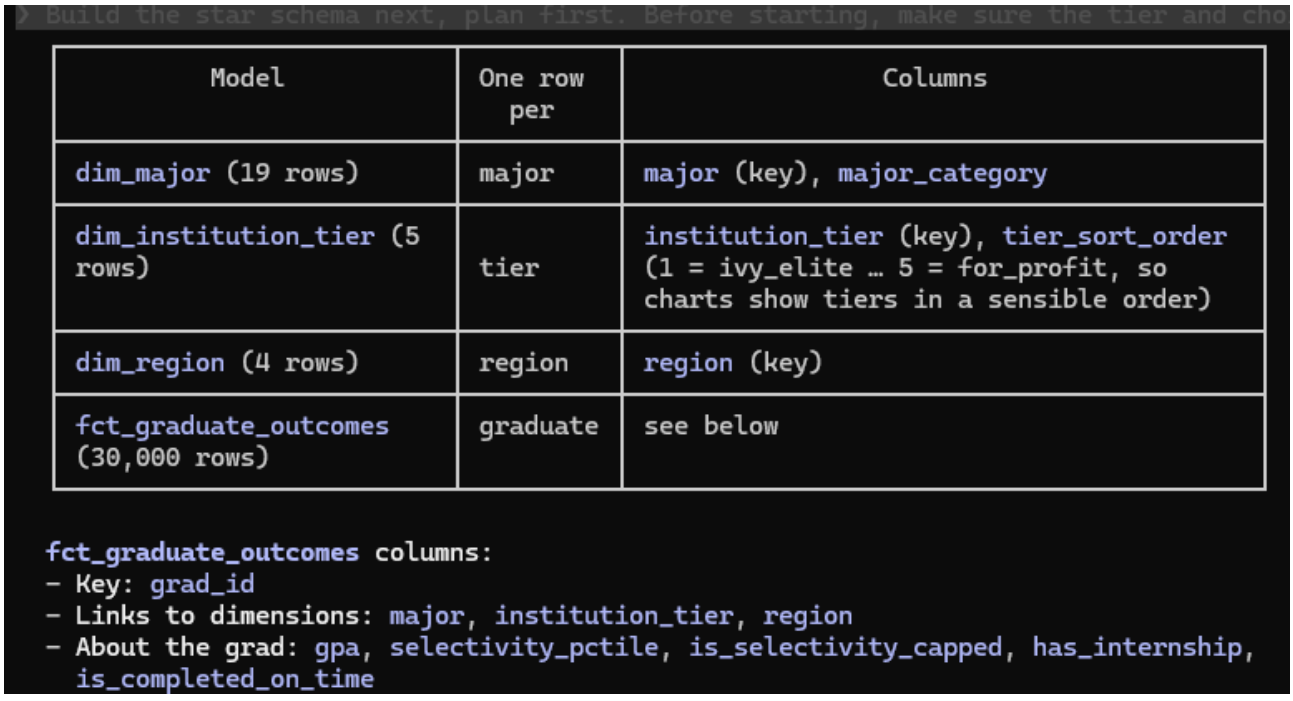


Figure 12. Star schema plan, part 2: dimension tables and their keys.

```
major_category moves out to dim_major, so the fact table doesn't repeat it.
selectivity_pctile stays in the fact table because it varies from school to school
within a tier.

Keys: I'll use the readable names (e.g. computer_science) as keys rather than made-up
ID numbers. That's simpler to read and query. With only 19 / 5 / 4 values, ID numbers
wouldn't add anything.

Docs + tests:
- New dbt/models/marts/_star_schema_models.yml, with a description for every model and
  column.
- Every key gets unique + not_null tests.
- accepted_values on tier, region and major_category.
- relationships tests from each link column in the fact table to its dimension, so
  every grad's major, tier and region must exist in the matching dimension table
  (required by CLAUDE.md).
- One custom test, assert_fct_graduate_outcomes_matches_source.sql: the fact table has
  exactly the same 30,000 grads as int_graduates_enriched, and net_roi_usd adds up to
  the same total.

Then:
1. Run dbt build --target prod --select dim_major+ dim_institution_tier+ dim_region+
   fct_graduate_outcomes+.
2. Show you row counts plus  Jump to bottom (ctrl+End) ↓ edian ROI by region, which
                                        ✔Update installed · Restart to update
──────────────────────────────────────────────── add-tier-and-choices-marts ─
❯
```

Figure 13. Star schema plan, part 3: relationships tests and a reconciliation test (same 30,000 rows and same total ROI as the source).

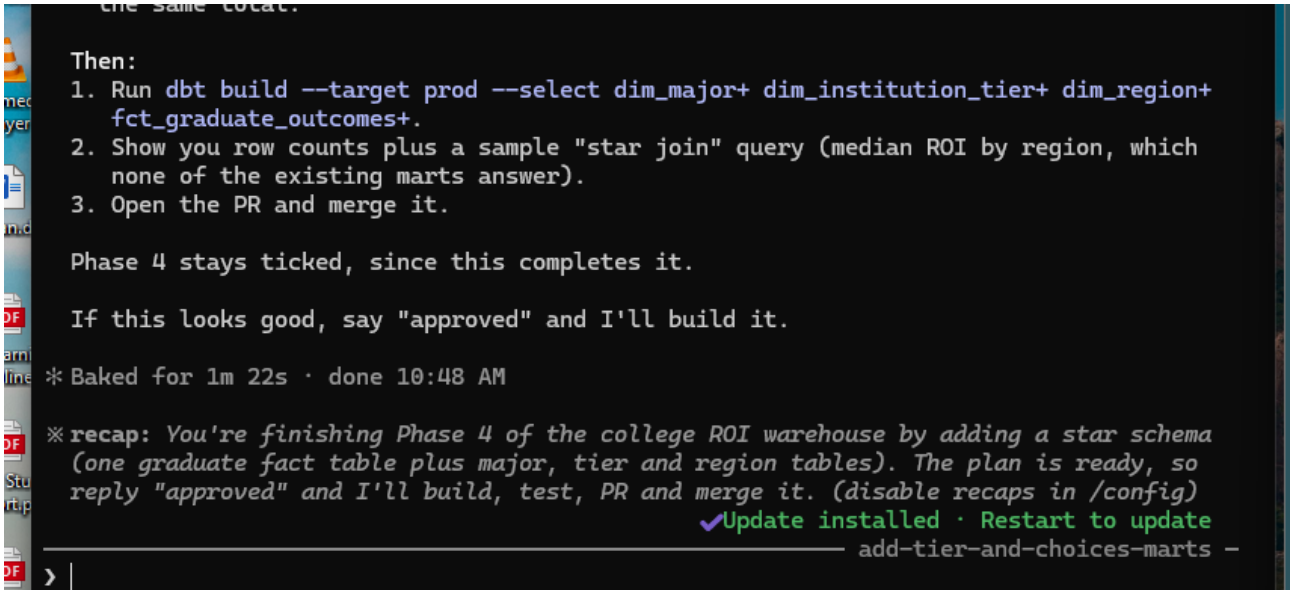


Figure 14. Star schema plan, part 4: run, sample star join, PR and merge.

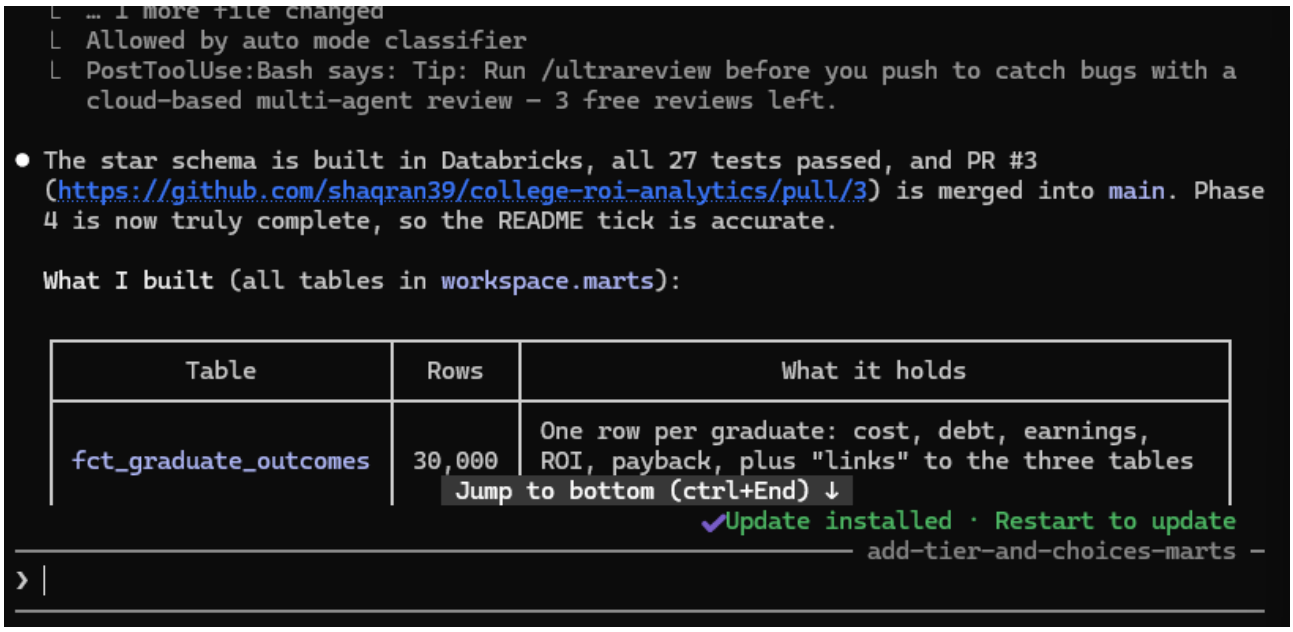


Figure 15. Star schema built: 27 tests passed, PR #3 merged.

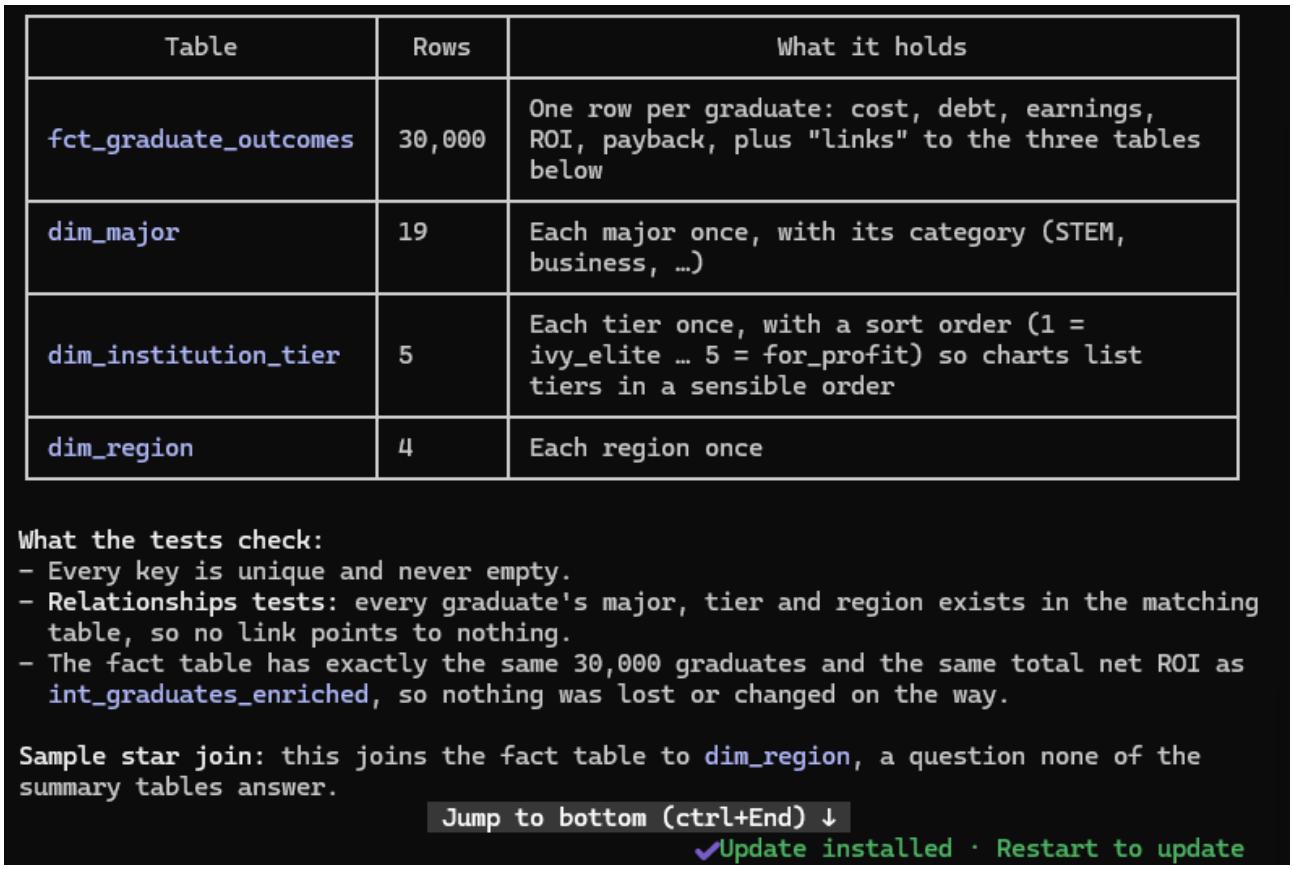


Figure 16. The four tables of the star schema and what each holds.

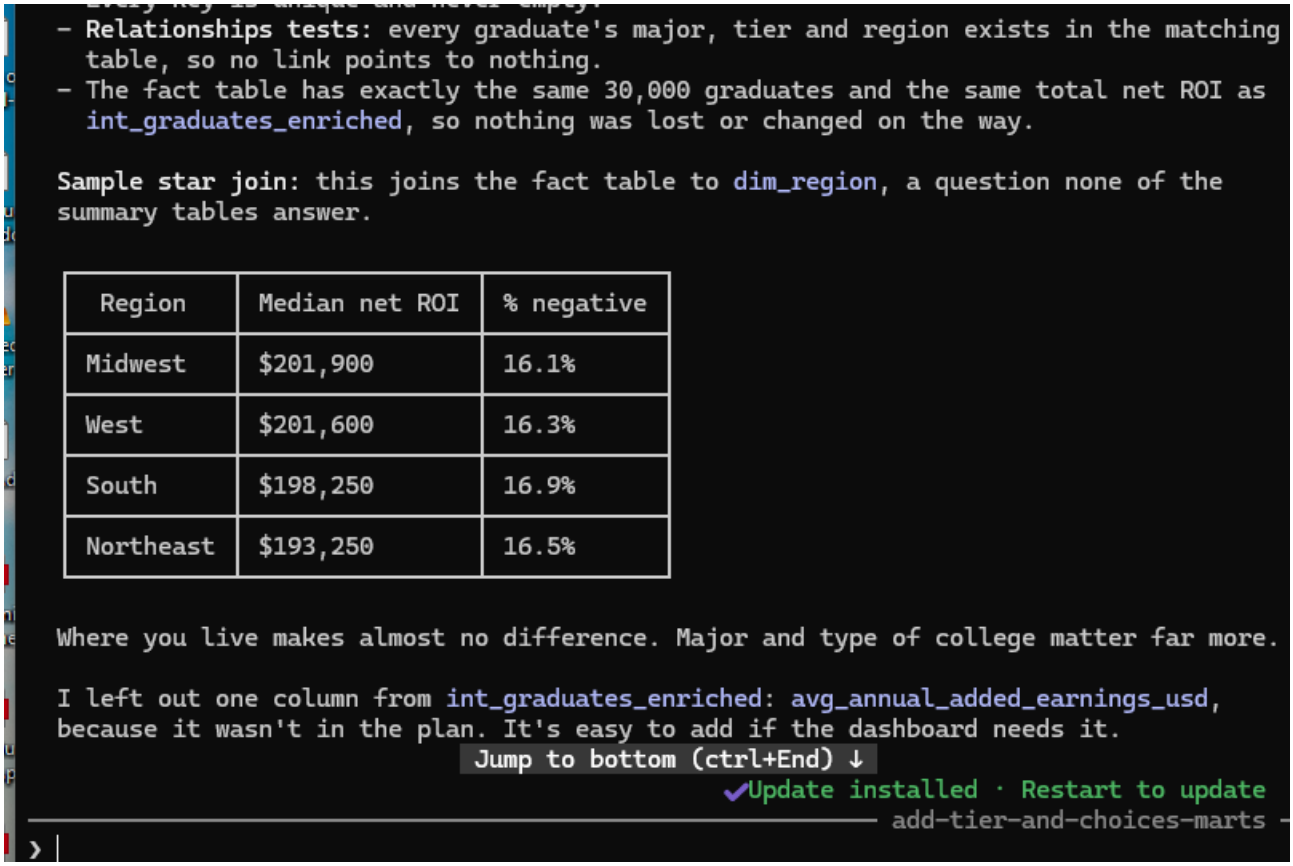


Figure 17. First "star join": median ROI by region. Region makes almost no difference.

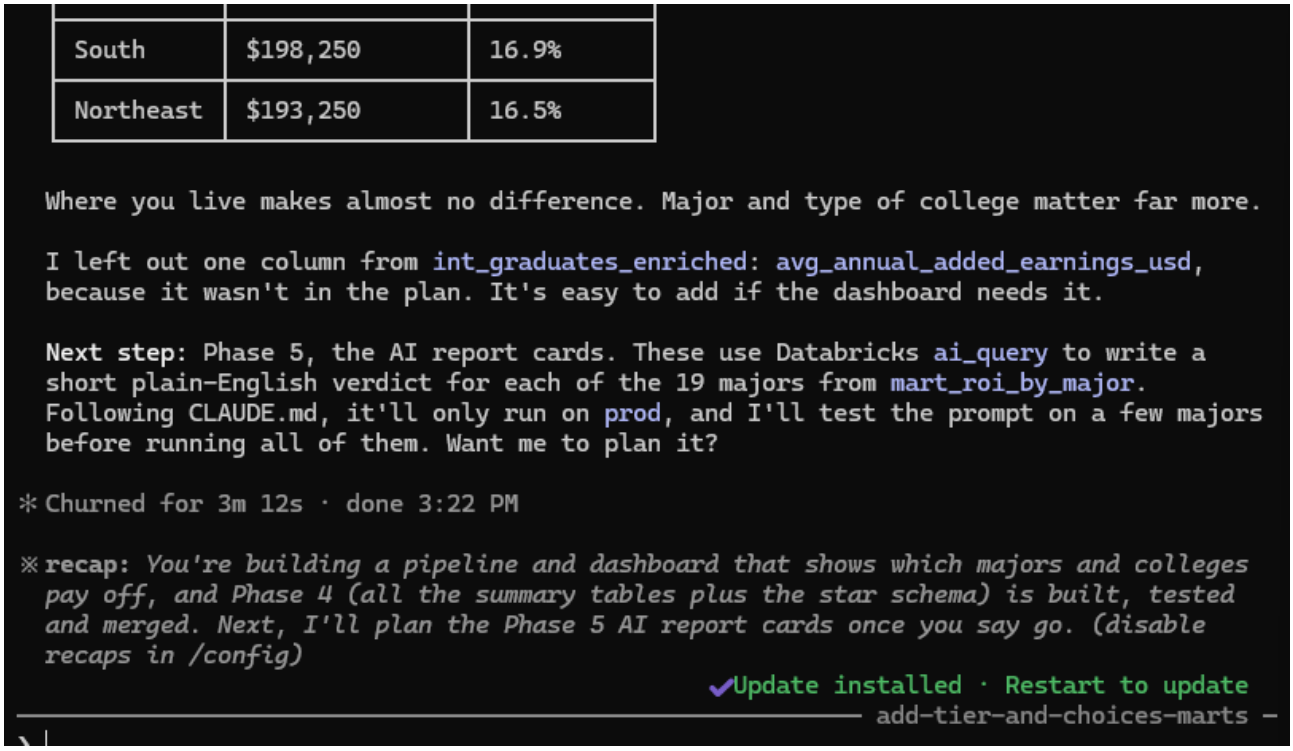
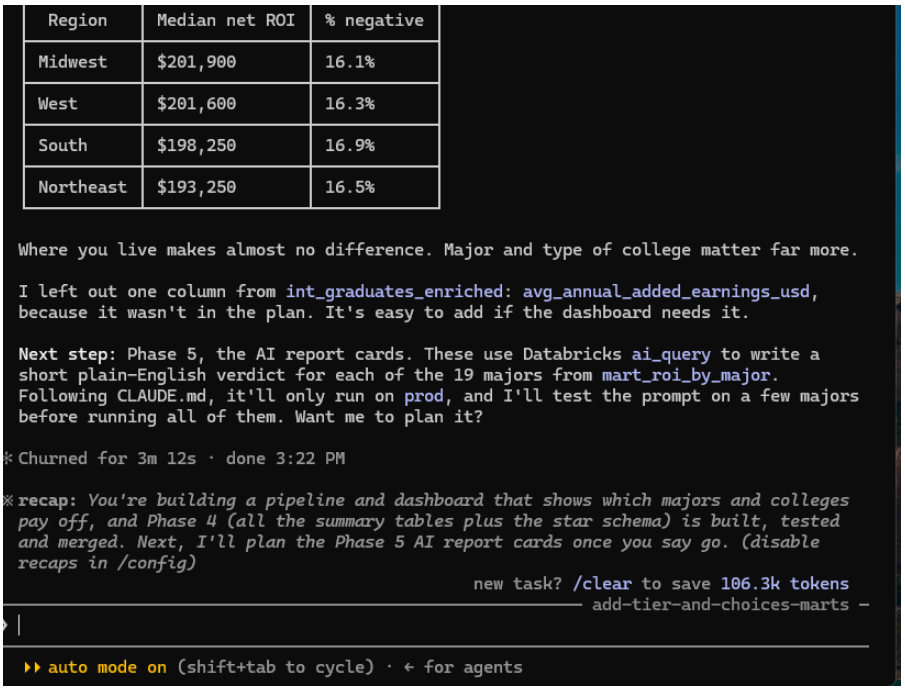


Figure 18. Phase 4 complete; Claude proposes Phase 5.

Why this matters: The reconciliation test (fact table = source row count and total ROI) is the same idea as SQL data reconciliation in a BA role. Natural keys were fine for 19/5/4 values; at scale use surrogate keys.

6.8 Phase 5: AI report cards and key findings

Start each phase with `/clear` (fresh context; CLAUDE.md and README carry the project knowledge) and plan mode.



Region	Median net ROI	% negative
Midwest	$201,900	16.1%
West	$201,600	16.3%
South	$198,250	16.9%
Northeast	$193,250	16.5%

Figure 19. Claude Code suggests /clear to free 106K tokens of context before a new phase.

Prompt given to Claude Code: `Read CLAUDE.md and README.md. Switch to main and pull the latest. Then plan Phase 5 on a new branch:`
`1. Check which AI model endpoints my Databricks Free Edition workspace has available for ai_query, and pick one.`
`2. Build mart_major_report_cards: one row per major with a 2-3 sentence plain-English verdict generated by ai_query from mart_roi_by_major.`
`3. The prompt must only use the numbers given in the row, no made-up facts, and use Australian English.`
`4. Build mart_key_findings: one AI-written summary paragraph of the whole project using the marts.`
`5. Test on 3 majors first and show me the outputs before running all 19.`
`6. Add tests (not null, one row per major). Only run on prod.`
`Plan first and explain it simply. After I approve: build, run, PR and merge.`

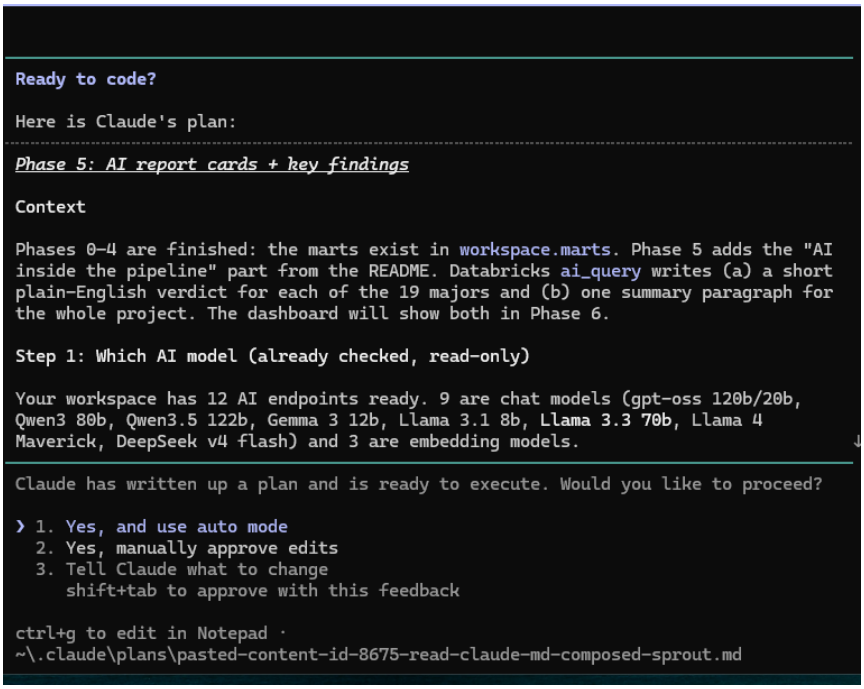


Figure 20. Phase 5 plan: 12 AI endpoints available; Llama 3.3 70B chosen.

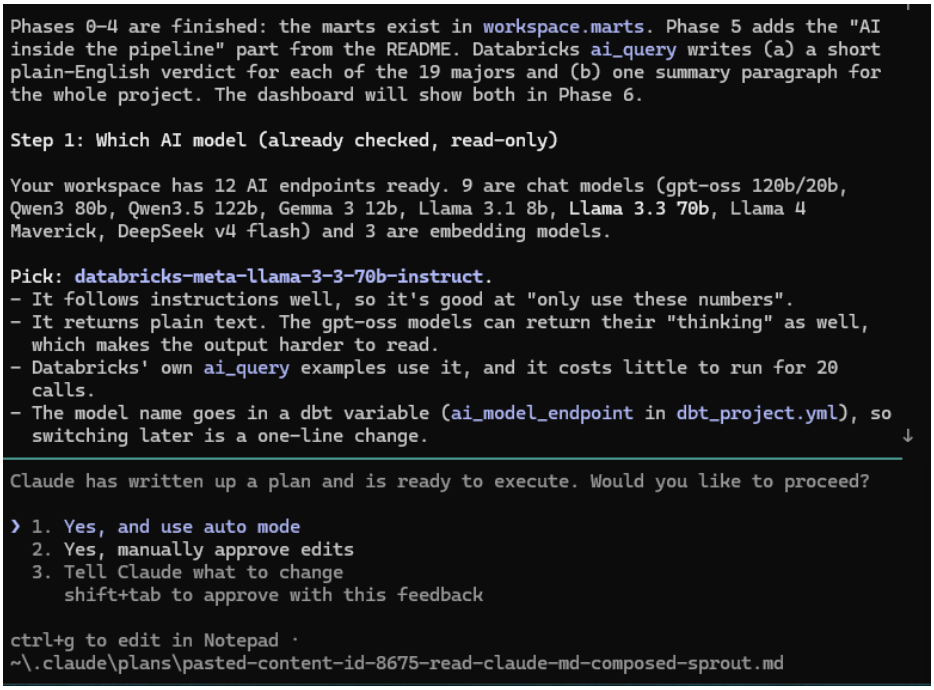


Figure 21. Why Llama 3.3 70B: follows instructions, plain text output, cheap; model name kept in a dbt variable.

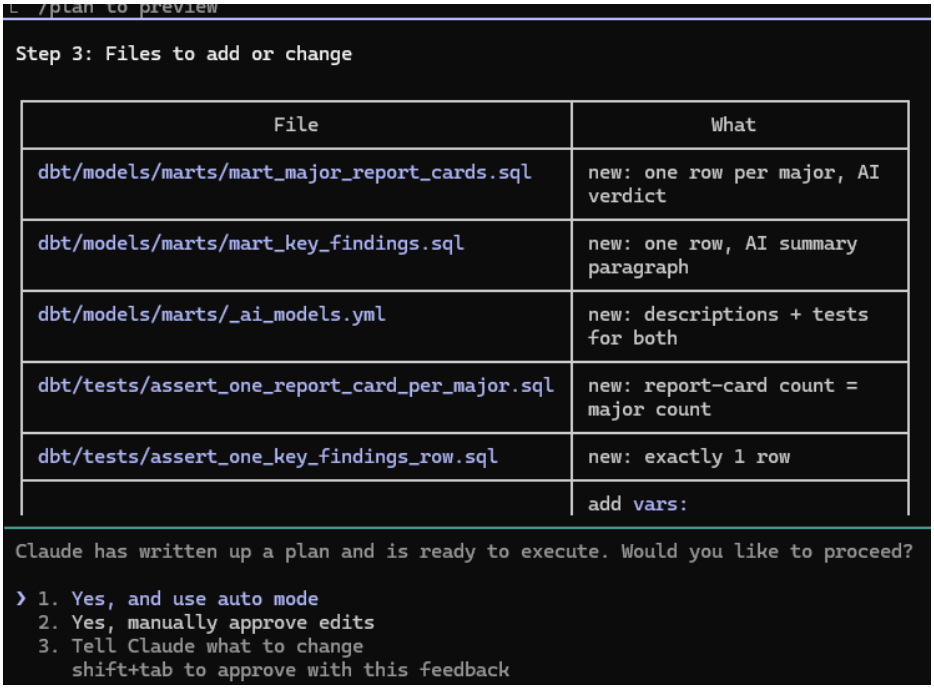


Figure 22. Files to add: two AI marts, their YAML, two row-count tests.

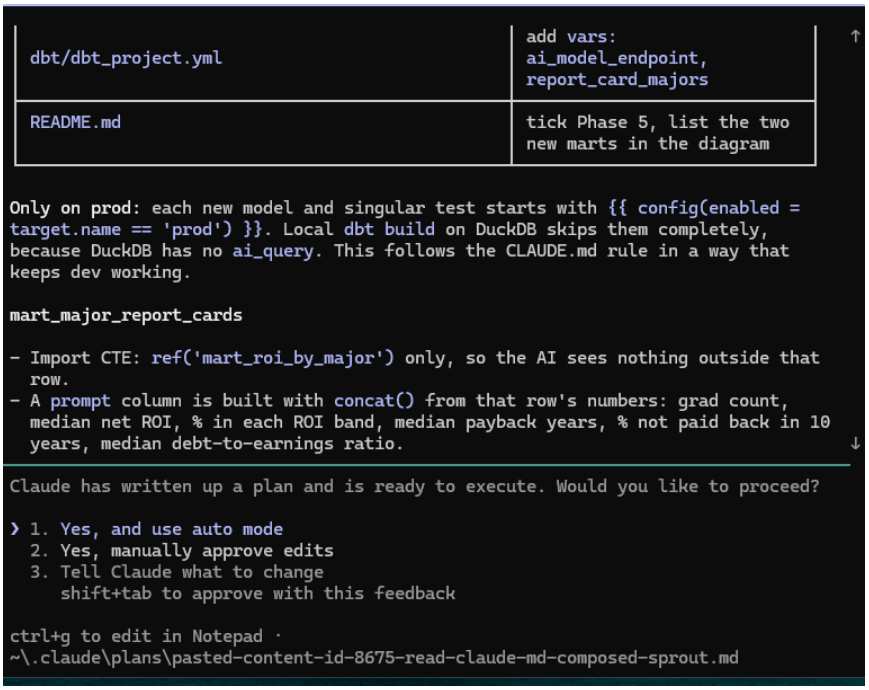


Figure 23. AI models only on prod; prompt built from one row of mart_roi_by_major.

```
mart_major_report_cards

- Import CTE: ref('mart_roi_by_major') only, so the AI sees nothing outside that
  row.
- A prompt column is built with concat() from that row's numbers: grad count,
  median net ROI, % in each ROI band, median payback years, % not paid back in 10
  years, median debt-to-earnings ratio.
- Prompt rules, written into the prompt itself:
  - Write 2-3 sentences in plain English, for a student choosing a major.
  - Use only the numbers given. Don't add other numbers, salaries, job names,
    reasons, or comparisons with other majors.
  - Use Australian English spelling (-ise / -our spellings, e.g. analyse,
    favourable). Money is US dollars, so write "US$".
  - Plain text only: no headings, bullet points or quote marks.
- Call: ai_query('{{ var("ai_model_endpoint") }}', prompt, modelParameters =>
  named_struct('temperature', 0.2, 'max_tokens', 200)). The low temperature keeps
  answers steady between runs.
- Test filter: if the report_card_majors var holds a list, only those majors are
  sent. It defaults to empty, which means all 19.

Claude has written up a plan and is ready to execute. Would you like to proceed?

❯ 1. Yes, and use auto mode
  2. Yes, manually approve edits
  3. Tell Claude what to change
     shift+tab to approve with this feedback

ctrl+g to edit in Notepad ·
~\.claude\plans\pasted-content-id-8675-read-claude-md-composed-sprout.md
```

Figure 24. Prompt rules: only given numbers, Australian English, temperature 0.2.

```
  sent. It defaults to empty, which means all 19.
- Output columns: major, the key numbers carried over for the dashboard, verdict,
  model_endpoint, generated_at.

mart_key_findings

- Import CTEs: mart_roi_by_major, mart_roi_by_tier, mart_choices_effect.
- Logic CTEs turn the marts into a short fact sheet (text):
  - the top 3 and bottom 3 majors by median net ROI
  - all 5 tiers: median net ROI, % negative ROI, median debt
  - the overall lift (major_category = 'all') for internship and for on-time
    completion
- One ai_query call. Same rules as above, but the output is one paragraph of 4-6
  sentences. It must say the data is synthetic, and that the choice effects are
  differences between groups, not proof of cause.
- Output columns: finding_id (= 1), summary_paragraph, input_facts (the exact fact
  sheet sent, so anyone can check the AI didn't invent numbers), model_endpoint,
  generated_at.

Tests

Claude has written up a plan and is ready to execute. Would you like to proceed?

❯ 1. Yes, and use auto mode
  2. Yes, manually approve edits
  3. Tell Claude what to change
     shift+tab to approve with this feedback

ctrl+g to edit in Notepad ·
~\.claude\plans\pasted-content-id-8675-read-claude-md-composed-sprout.md
```

Figure 25. mart_key_findings: one paragraph, must say data is synthetic; input_facts stores the exact fact sheet.

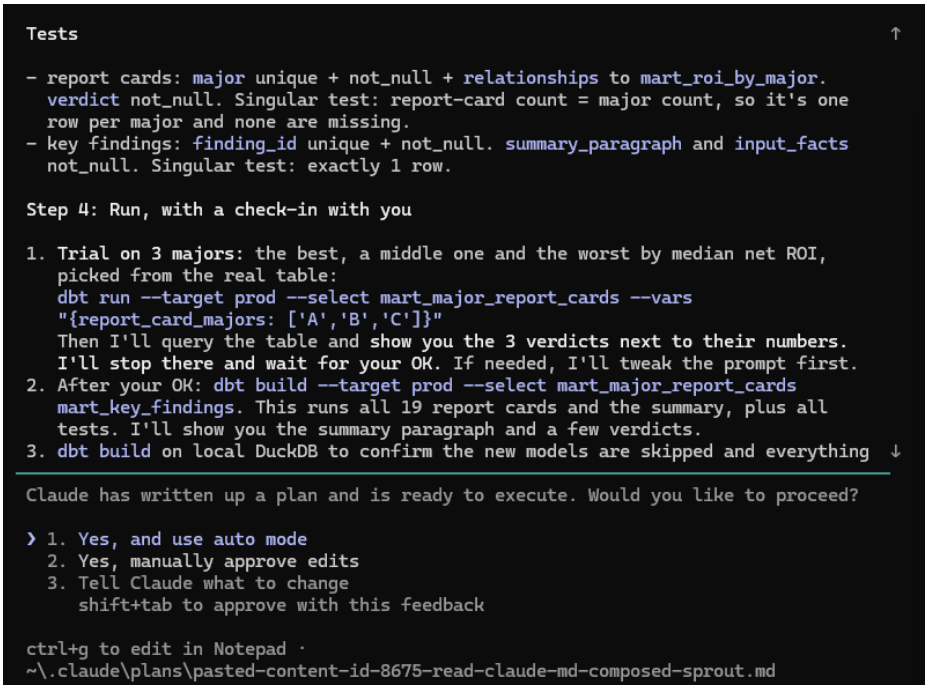


Figure 26. Tests and the 3-major trial with a check-in.

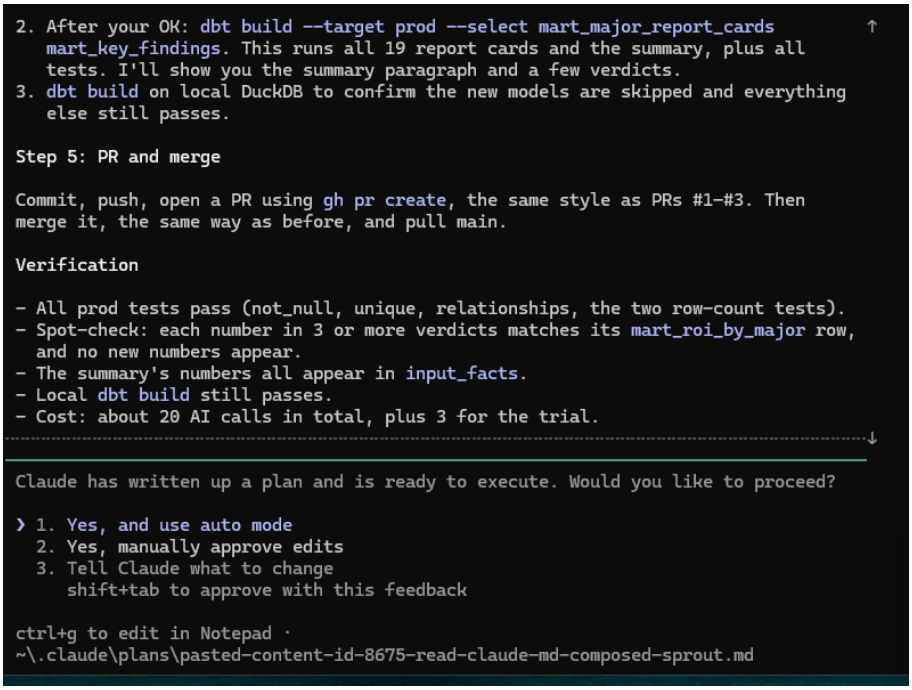


Figure 27. Verification plan and cost (~23 AI calls planned).

Choose "Yes, manually approve edits" (not auto mode) so it really stops at the trial.

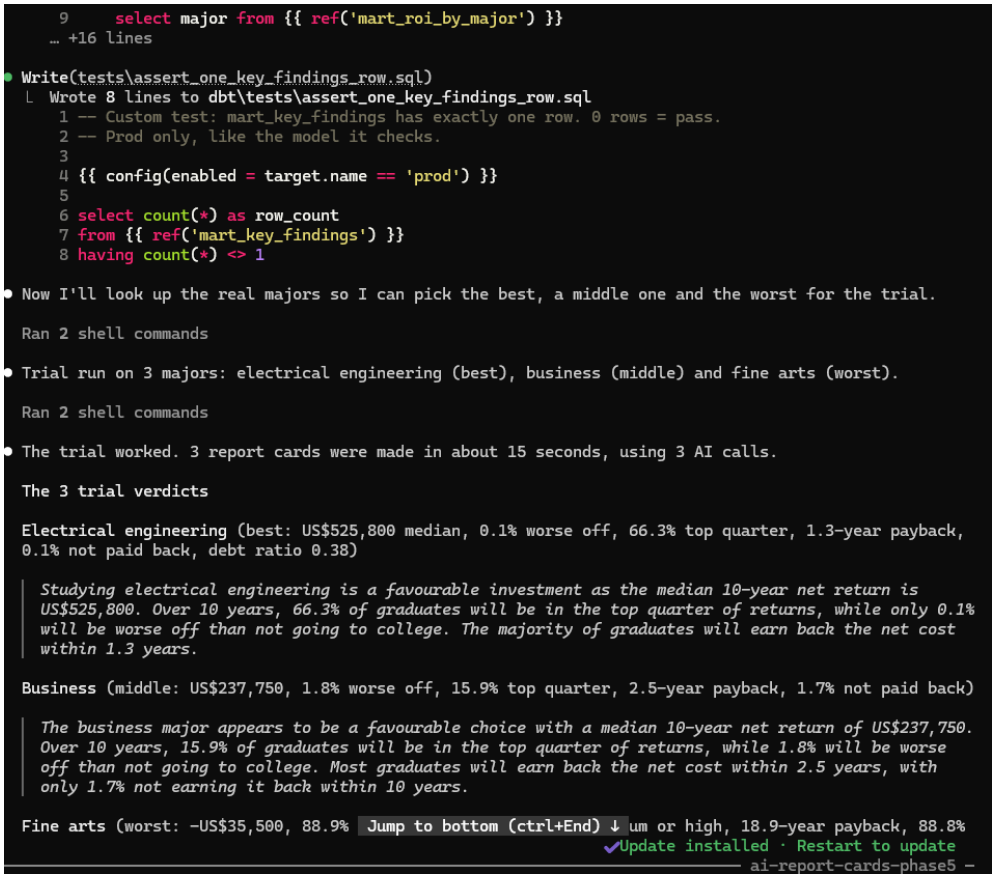


Figure 28. Trial on 3 majors: every number matched the table.



Figure 29. Claude found three wording problems, including "the majority... within 1.3 years" (1.3 is the median, so it means half). A meaning error, not a number error.

Prompt given to Claude Code: `Yes, make all 3 changes. Also: start each verdict with the plain takeaway first (e.g. "A strong financial bet." / "Financially risky."), then the numbers. Re-run the same 3 majors and show me before running all 19.`

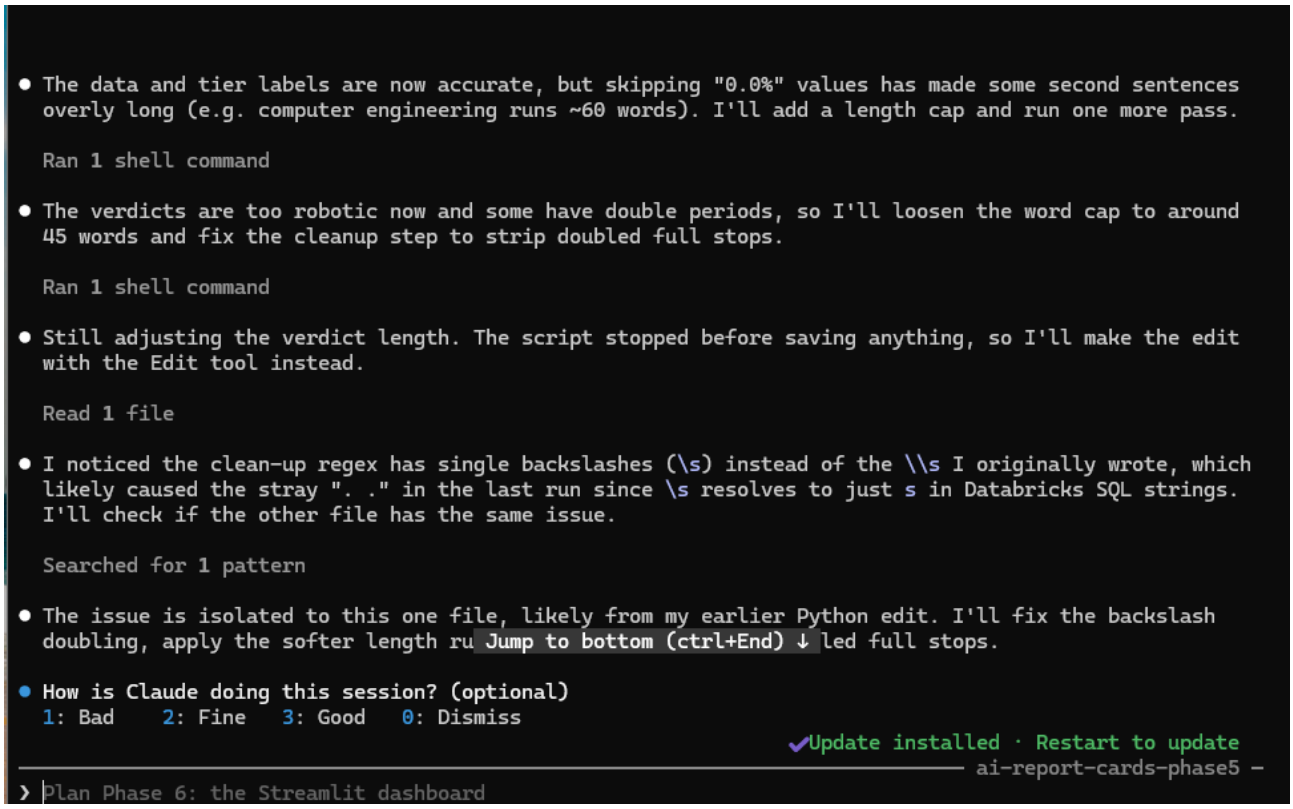


Figure 30. Prompt tuning: length cap, double full stops, a lost regex backslash found and fixed.

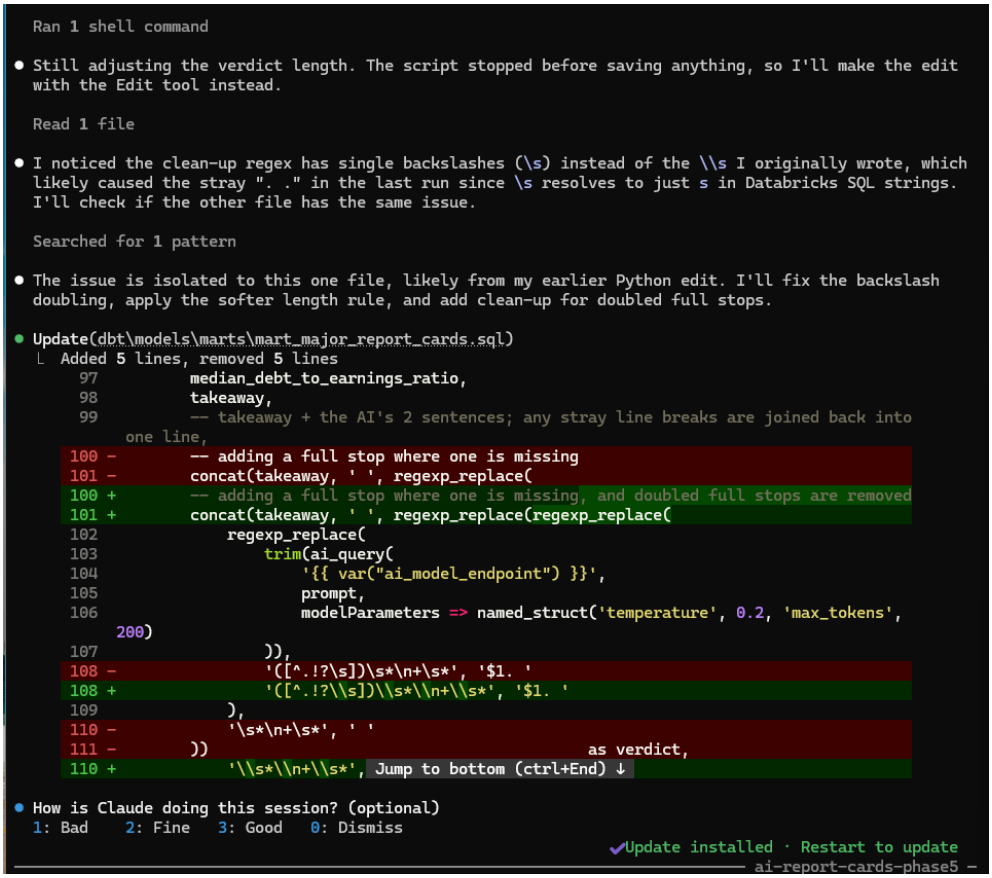


Figure 31. The regex fix in mart_major_report_cards.sql.

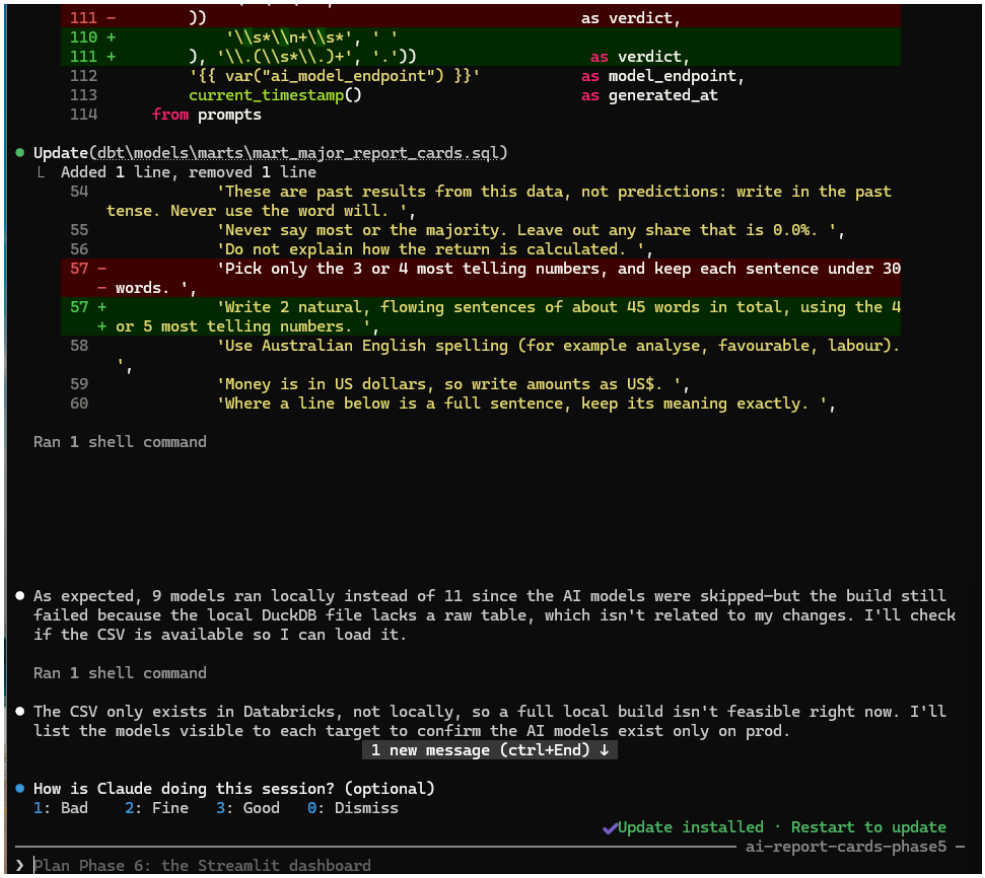


Figure 32. Prompt rule changes (past tense, never "most", about 45 words).

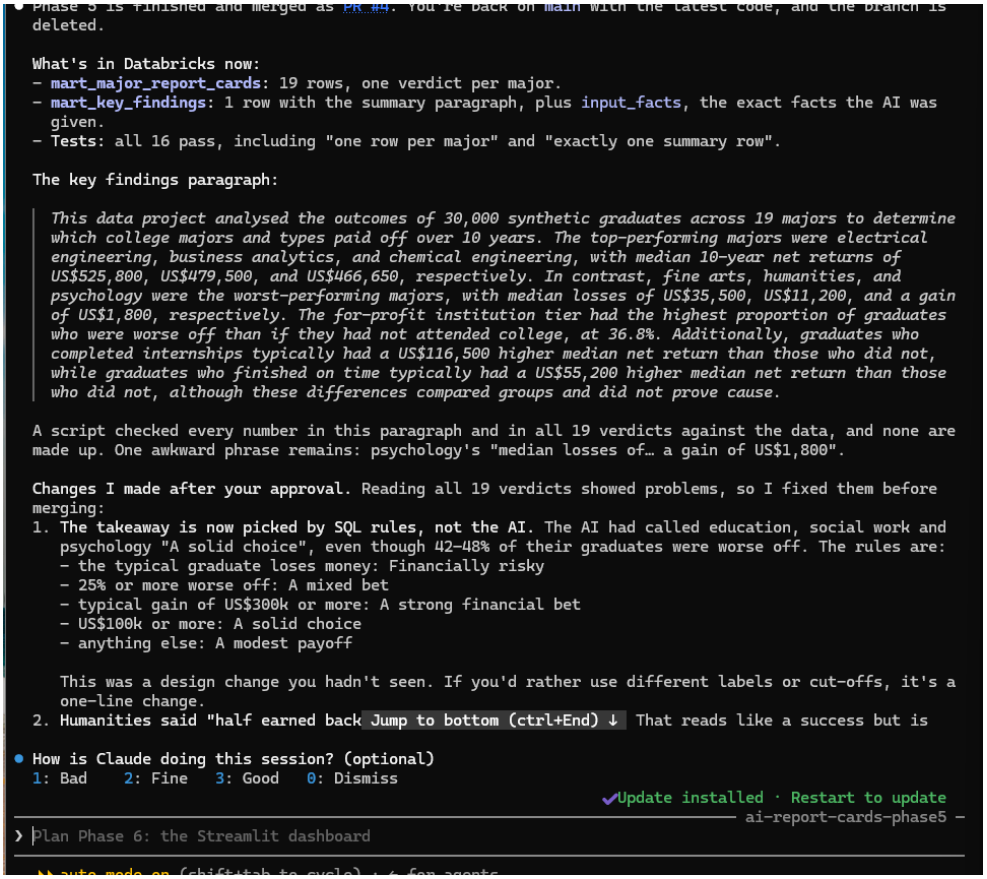


Figure 33. Phase 5 merged (PR #4): 19 report cards + key findings paragraph, 16 tests pass.

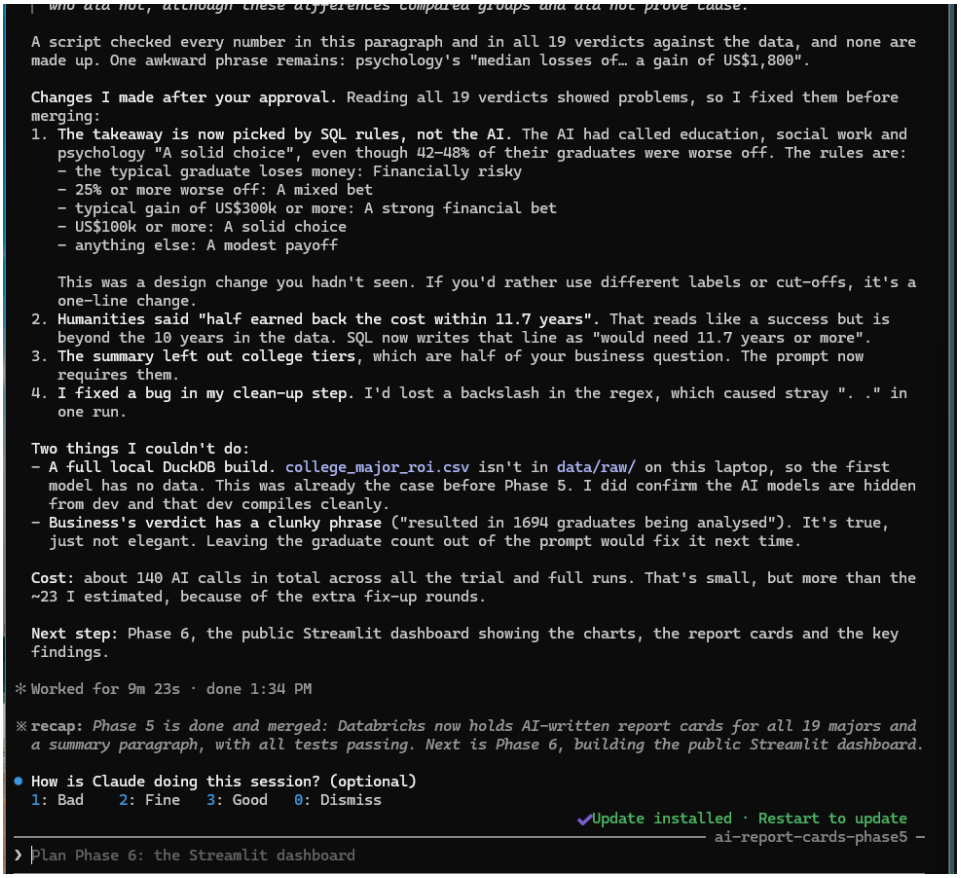


Figure 34. Changes after review: takeaway labels moved to SQL rules because the AI called 42-48%-worse-off majors "A solid choice".

Final guardrails in mart_major_report_cards.sql (rules part of the prompt):

```
with_takeaway as (

    select
        *,
        case
            when median_net_roi_usd < 0      then 'Financially risky.'
            when pct_roi_negative >= 25      then 'A mixed bet.'
            when median_net_roi_usd >= 300000 then 'A strong financial bet.'
            when median_net_roi_usd >= 100000 then 'A solid choice.'
            else                                  'A modest payoff.'
        end as takeaway
    from selected_majors

),

prompts as (

    select
        *,
        concat(
            'You are writing a short verdict for a student deciding whether to study a college maj
    or. ',
```

```
            'The verdict starts with this takeaway, which is already written: ', takeaway, ' ',
            'Write the next 2 sentences only, in plain English, backing up the takeaway with the n
    umbers. ',
            'Do not repeat the takeaway. ',
            'Rules: use ONLY the numbers given below. Do not add any other numbers, salaries, job
    titles, ',
            'reasons, causes or comparisons with other majors. ',
            'These are past results from this data, not predictions: write in the past tense. Neve
    r use the word will. ',
            'Never say most or the majority. Leave out any share that is 0.0%. ',
            'Do not explain how the return is calculated. ',
            'Describe money outcomes only as came out ahead or worse off. Never use the words loss
    , losses, lost, gain or gains. ',
            'Write 2 natural, flowing sentences of about 45 words in total, using the 4 or 5 most
    telling numbers. ',
            'Use Australian English spelling (for example analyse, favourable, labour). ',
            'Money is in US dollars, so write amounts as US$. ',
            'Where a line below is a full sentence, keep its meaning exactly. ',
            'Reply with plain text only: no heading, no bullet points, no quote marks.\n\n',
```

Interview point: Rules decide, AI only words it. The numbers can all be right and the meaning still wrong (median vs majority). Always review AI text against the data before running it on everything.

6.9 Phase 6: Public Streamlit dashboard

Prompt given to Claude Code: `Read CLAUDE.md and README.md. Two quick fixes first: (1) fix psychology's awkward "losses... a gain" phrasing and drop the grad count from the report card prompt, (2) update CLAUDE.md to say we only build on prod now, skip local DuckDB builds.`

`Then plan Phase 6a: a public Streamlit dashboard in dashboards/ that reads LIVE from the Databricks marts.`
`- Page 1 Overview: key findings paragraph + 3-4 headline numbers`
`- Page 2 Majors: ROI by major chart + each major's AI report card`
`- Page 3 Colleges & choices: tier comparison + internship / on-time effect by category`
`- Credentials via Streamlit secrets, never committed. Cache queries so the warehouse isn't hit on every click.`
`- Clean, simple design. Note that the data is synthetic.`
`Run it locally for me to see first, then help me deploy it to Streamlit Community Cloud. Plan first.`

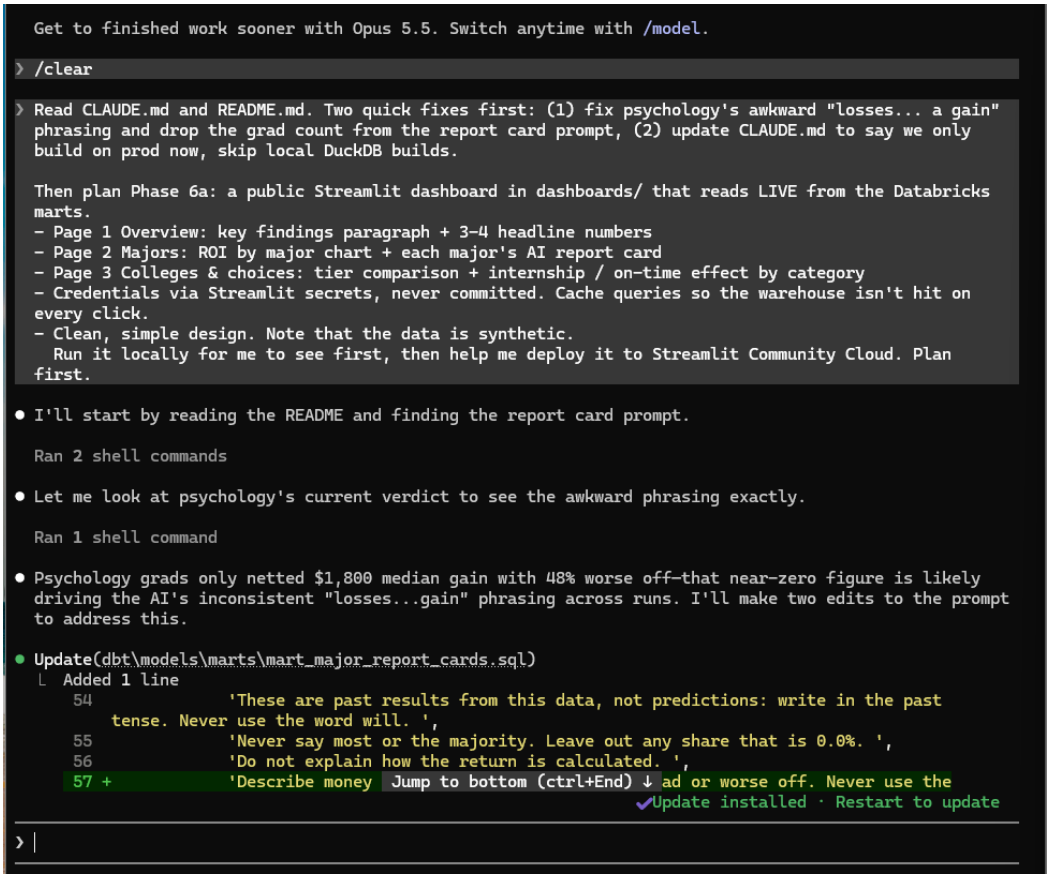


Figure 35. Quick fixes applied: "came out ahead / worse off" wording only.

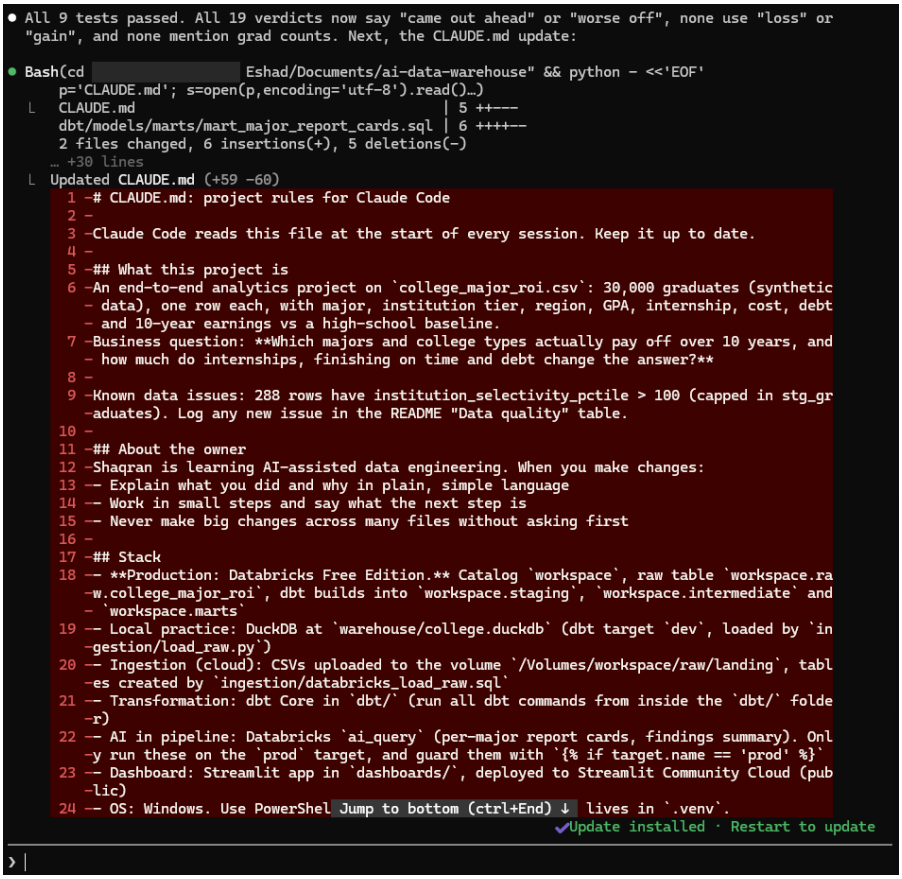


Figure 36. CLAUDE.md updated to prod-only builds.

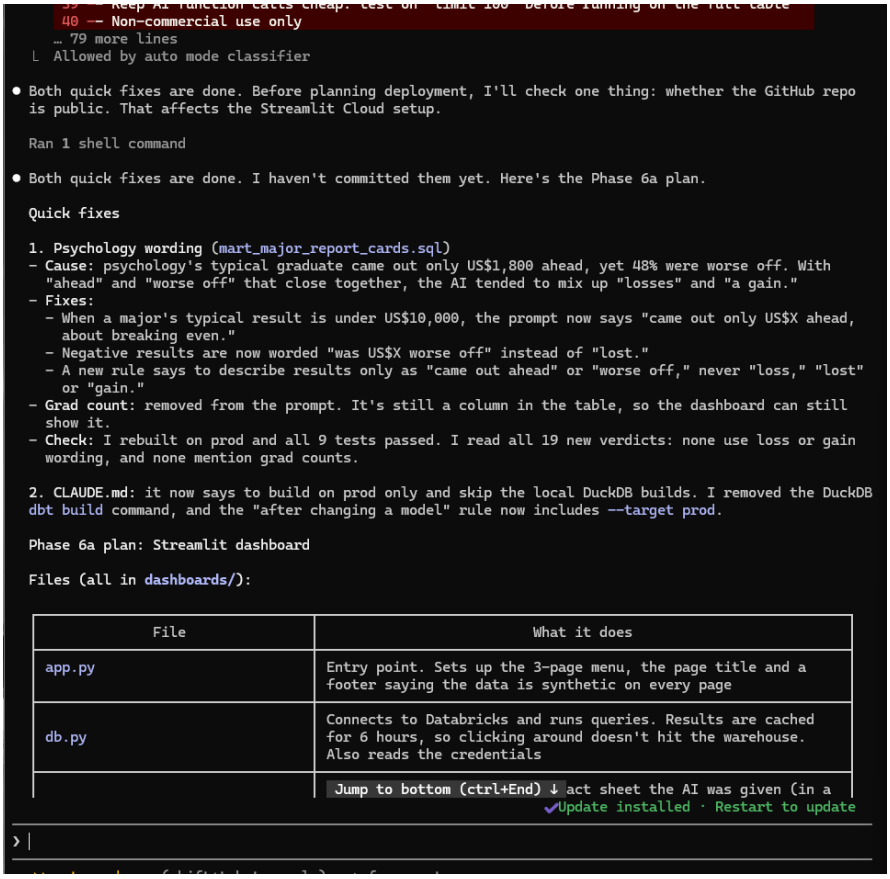


Figure 37. Dashboard plan: app.py, db.py with 6-hour cache, pages.

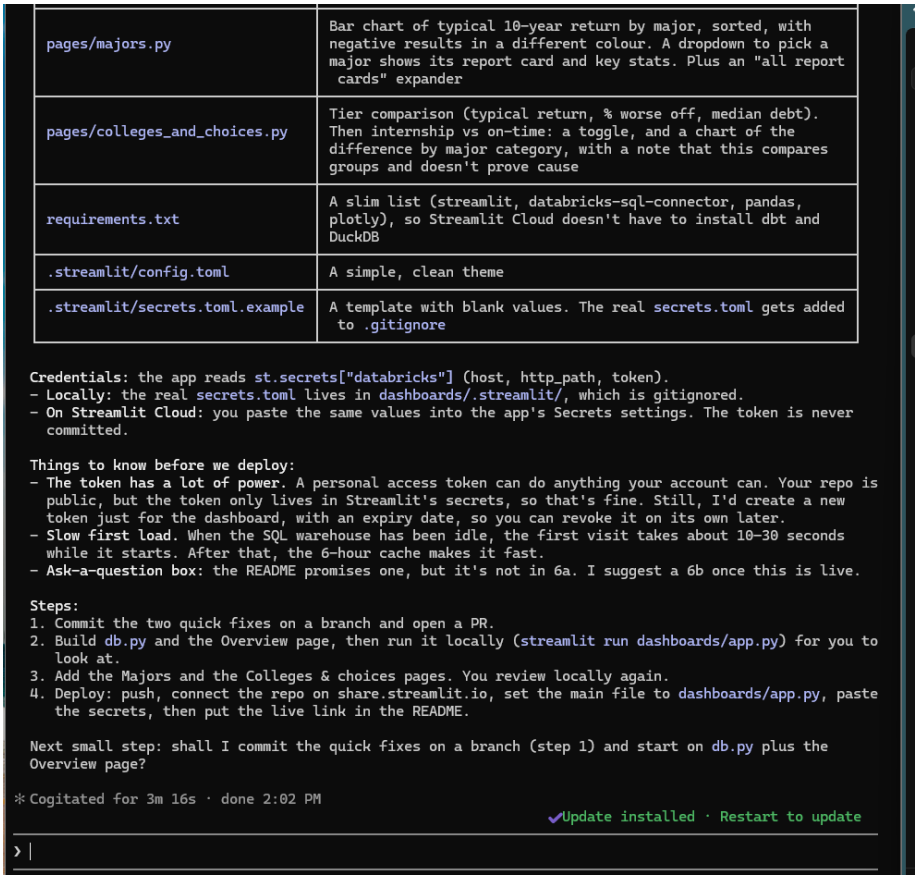


Figure 38. Plan details: secrets handling, token advice (make a dashboard-only token), slow first load, steps.

Dashboard token (least privilege): name streamlit-dashboard, 90 days, auto-scope OFF, scope **BI Tools / sql** only.

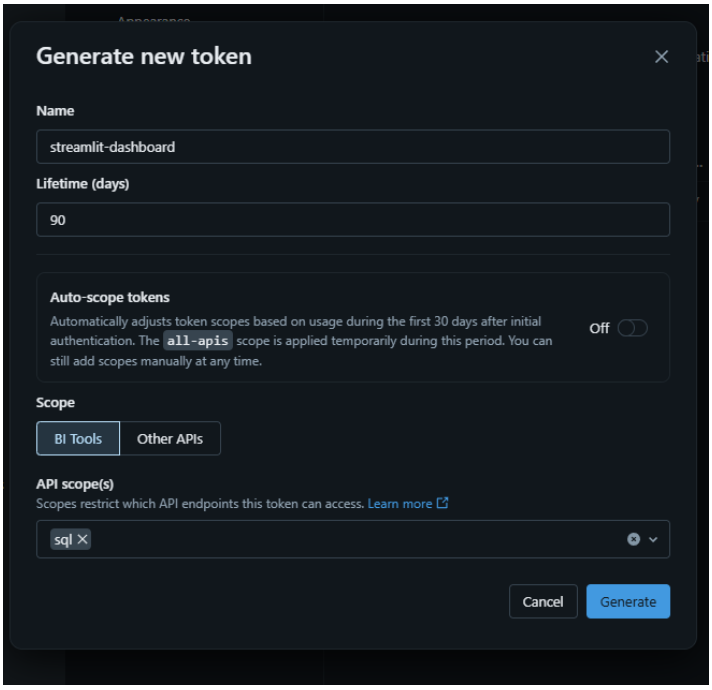


Figure 39. The read-only dashboard token: BI Tools scope with only sql.

Prompt given to Claude Code: `Yes, go ahead with steps 1 and 2. Create dashboards/.streamlit/secrets.toml with BLANK values and make sure it's gitignored before anything else. I'll paste the token in myself. Don't ask me for it in chat.`

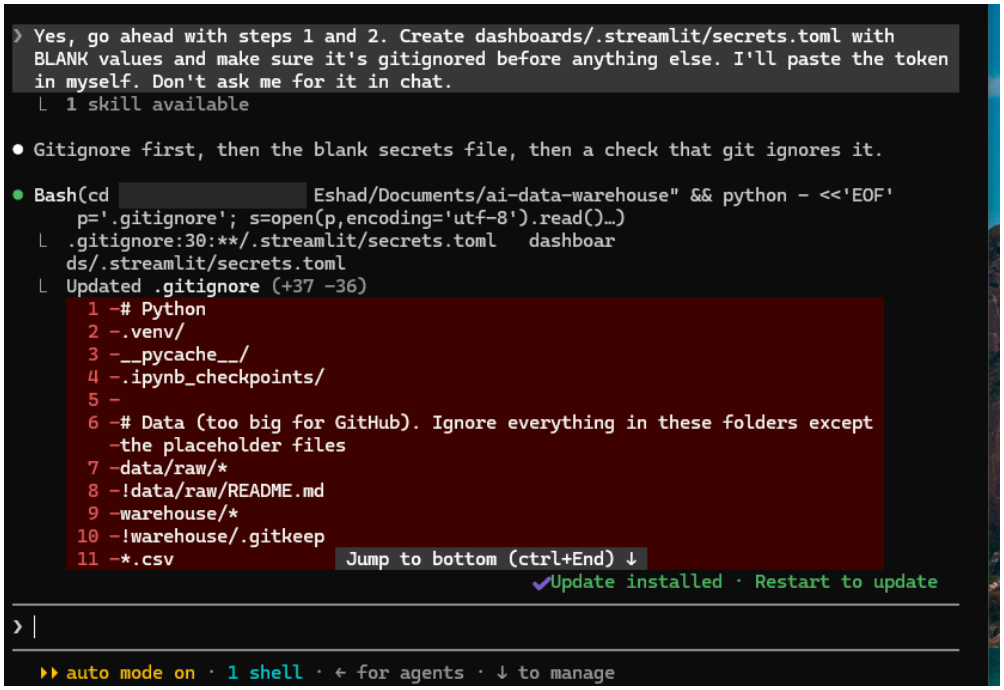


Figure 40. .gitignore updated first, before the secrets file existed.

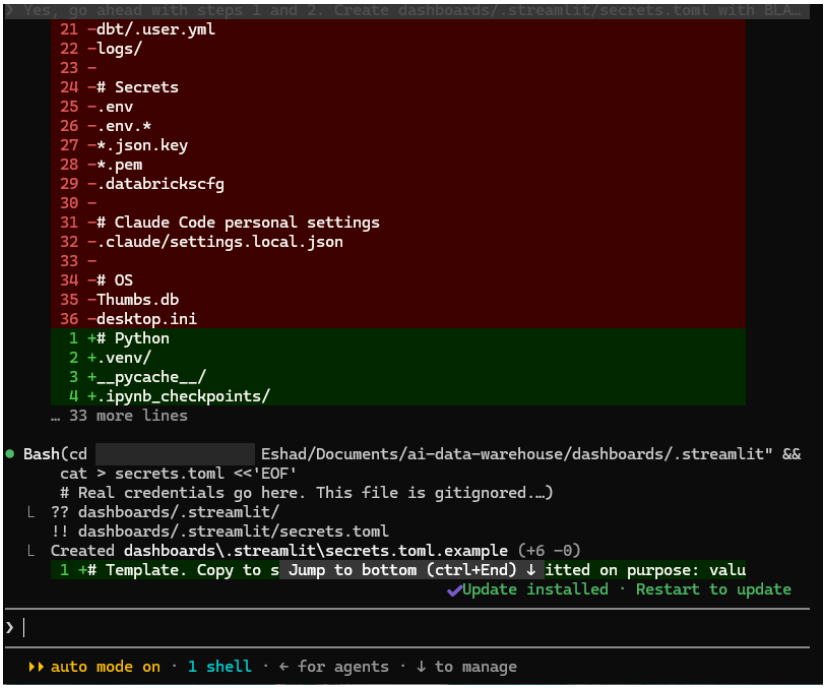


Figure 41. secrets.toml created blank; git status shows !! (ignored). Only secrets.toml.example is committed.

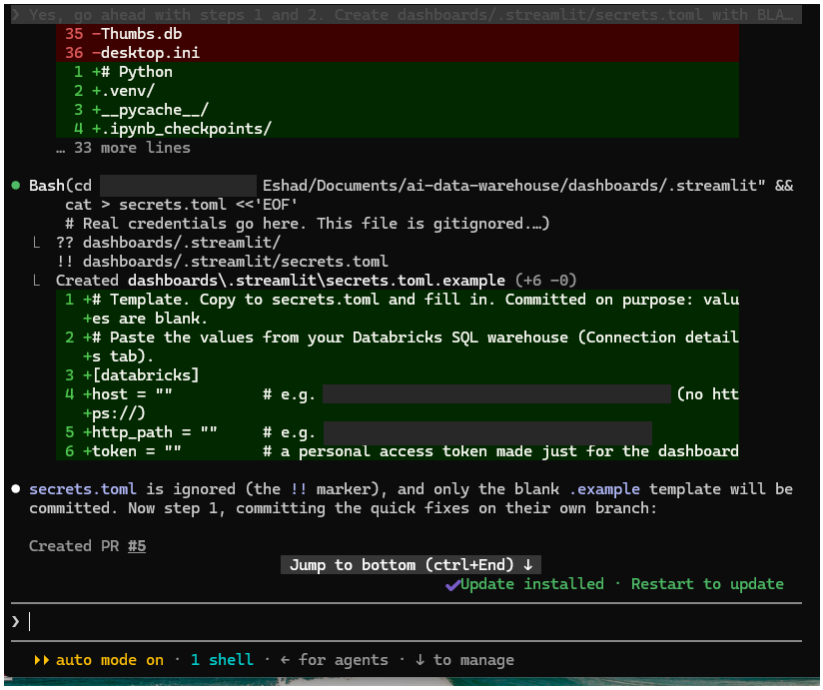


Figure 42. Quick-fix PR #5 created; dataviz guidance loaded for the dashboard.

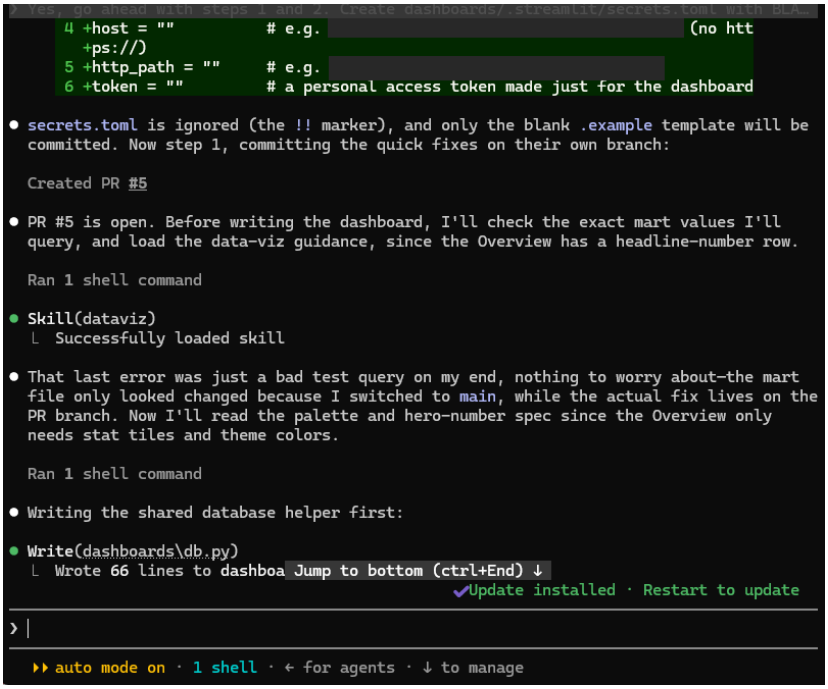


Figure 43. db.py and app.py written.

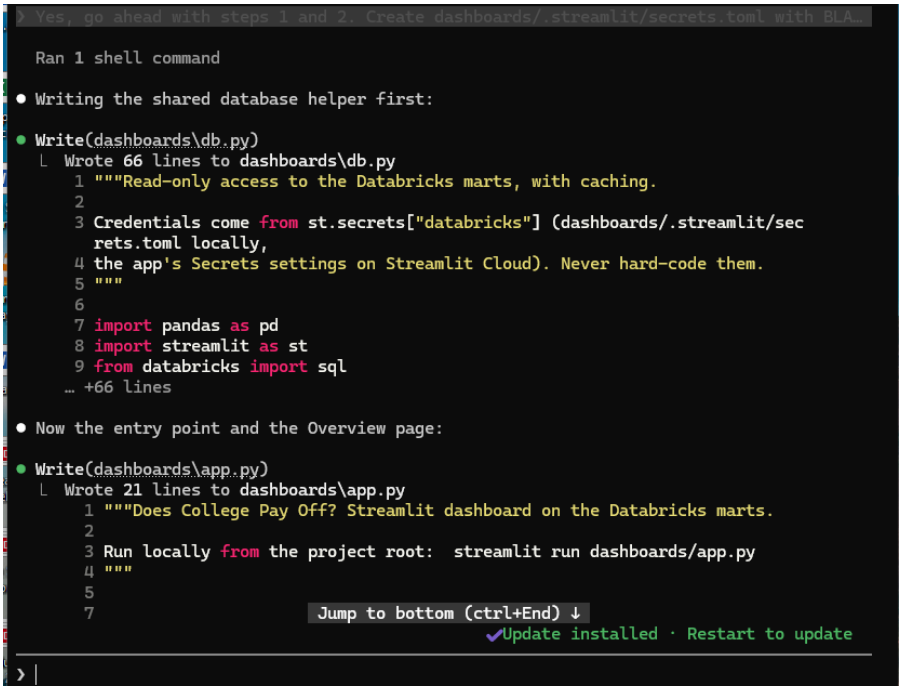


Figure 44. Overview page, slim requirements.txt and theme.

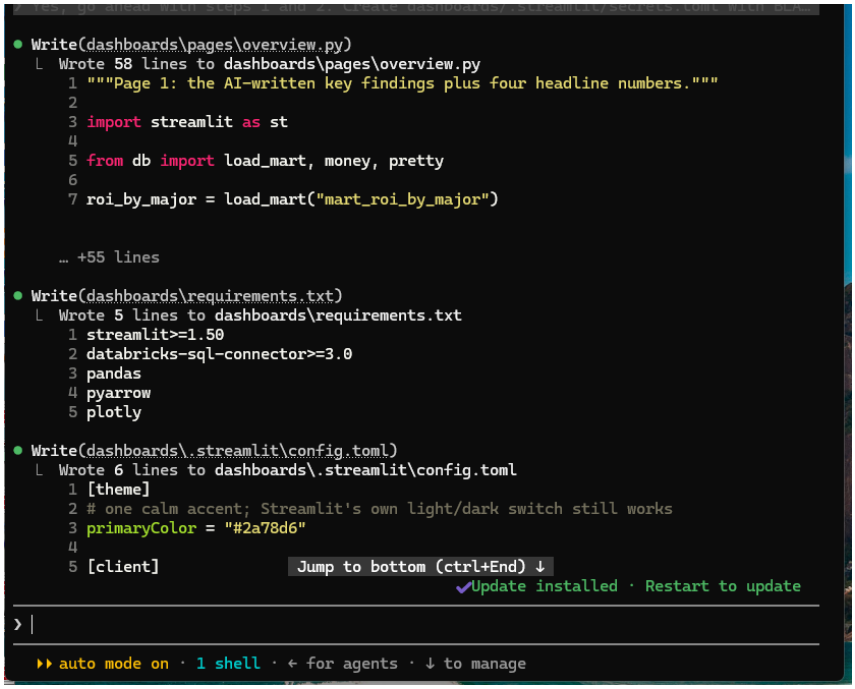


Figure 45. App running at localhost:8501, waiting for credentials.

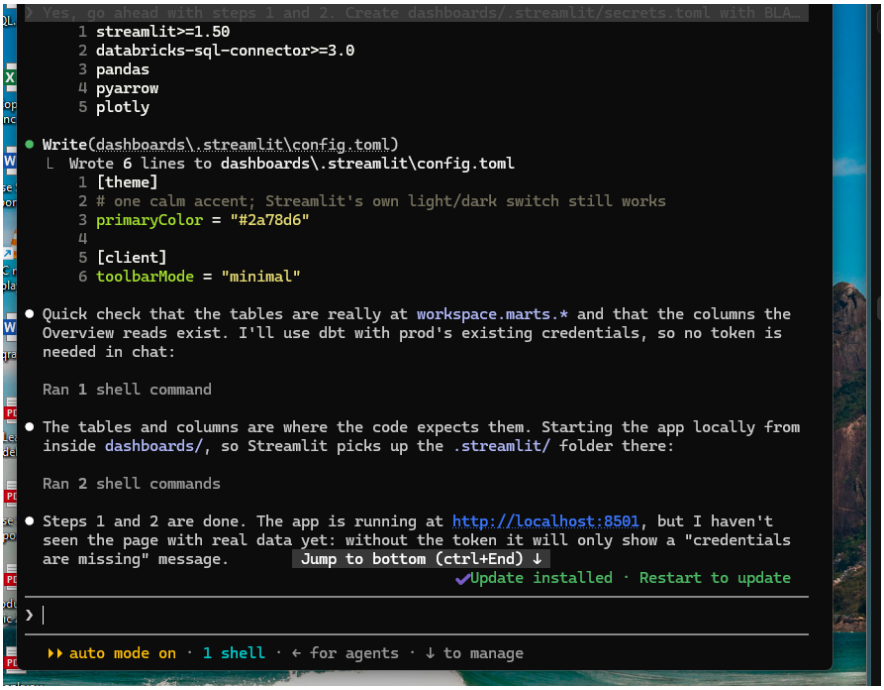


Figure 46. Summary of what was built and what is ignored by git.

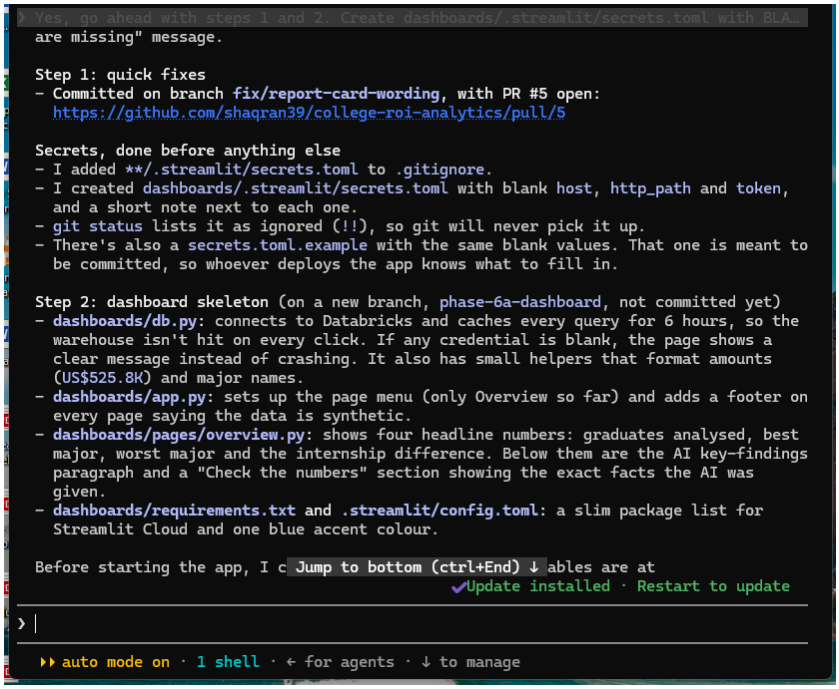


Figure 47. Your step: paste host, HTTP path and token into secrets.toml yourself.

secrets.toml (local file, never committed; on Streamlit Cloud the same 3 lines go in the app's Secrets box):

```
[databricks]
host = "dbc-xxxx.cloud.databricks.com"   # no https://
http_path = "/sql/1.0/warehouses/xxxx"
token = "the streamlit-dashboard token"
```

Find host and HTTP path again any time with `echo $env:DATABRICKS_HOST` and `echo $env:DATABRICKS_HTTP_PATH`.

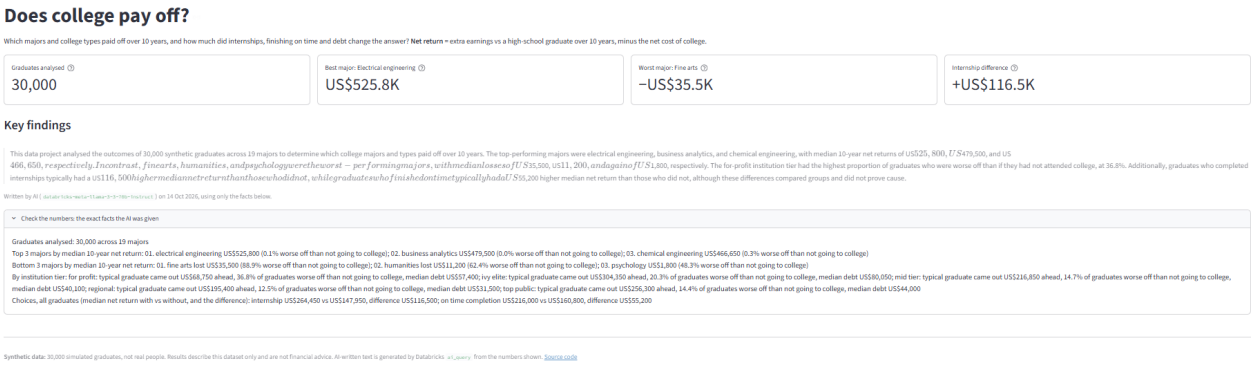


Figure 48. Bug: the key findings text is garbled because Streamlit reads text between two $ signs as LaTeX maths. Fixed by escaping $ (md_safe in db.py).

Prompt given to Claude Code: `Three things:`

```
1. Fix the key findings text: dollar signs are being read as LaTeX math. Escape them.
2. Merge PR #5 and rebuild mart_major_report_cards and mart_key_findings on prod so the new
wording shows.
3. Build the Majors and Colleges & choices pages as planned. Then add a 4th page "Deep dive"
with: GPA vs ROI, debt vs ROI, the spread of outcomes per major (best/typical/worst case), and a
"What if" tool where I pick major + tier + internship + on time and see the typical outcome. Add
any new marts needed with tests. Show me locally before committing.
```

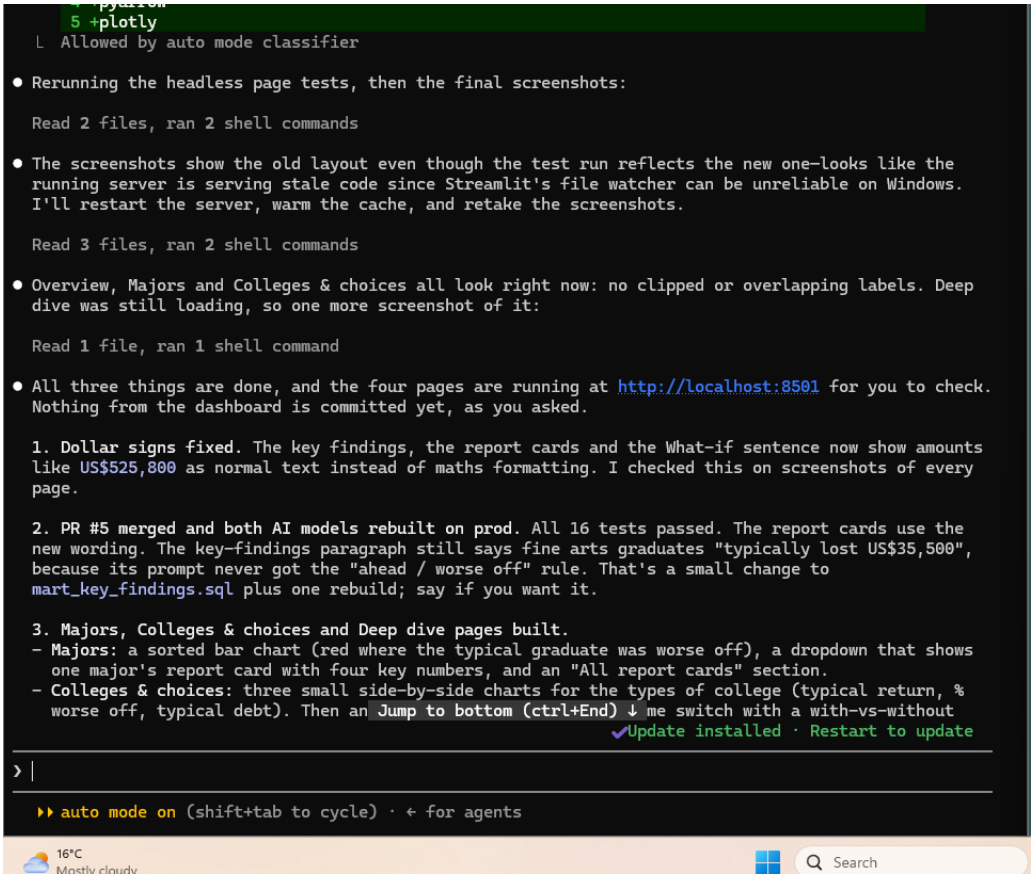


Figure 49. Three things done: $ fixed, PR #5 merged, pages built.

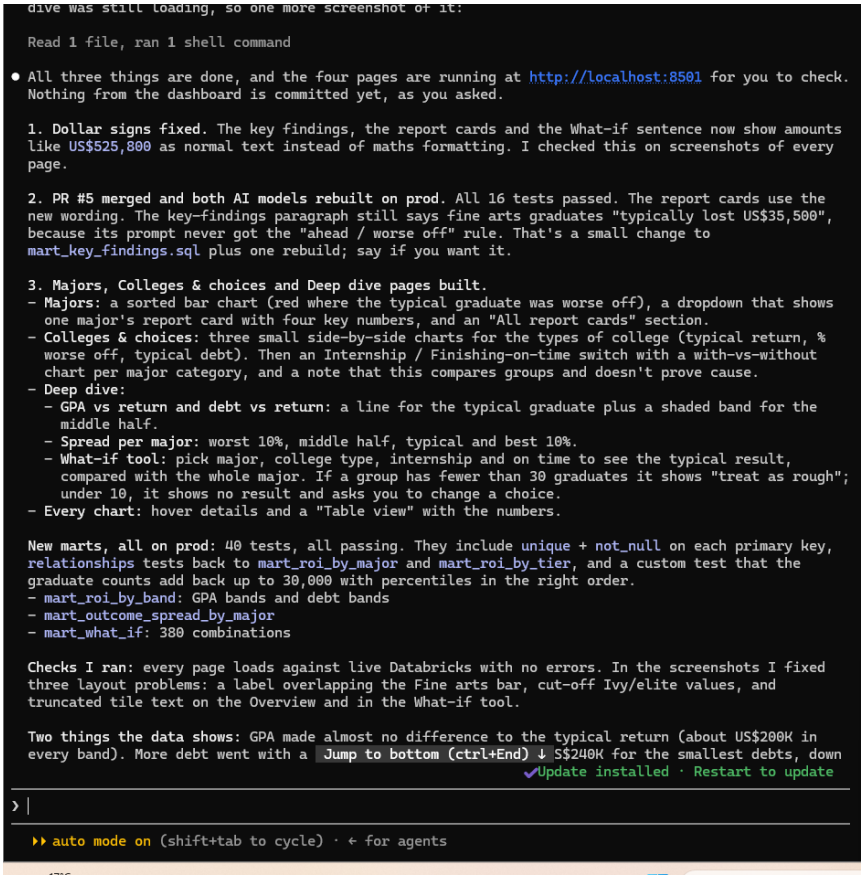


Figure 50. New marts (mart_roi_by_band, mart_outcome_spread_by_major, mart_what_if), 40 tests passing; GPA barely matters, more debt goes with lower return.

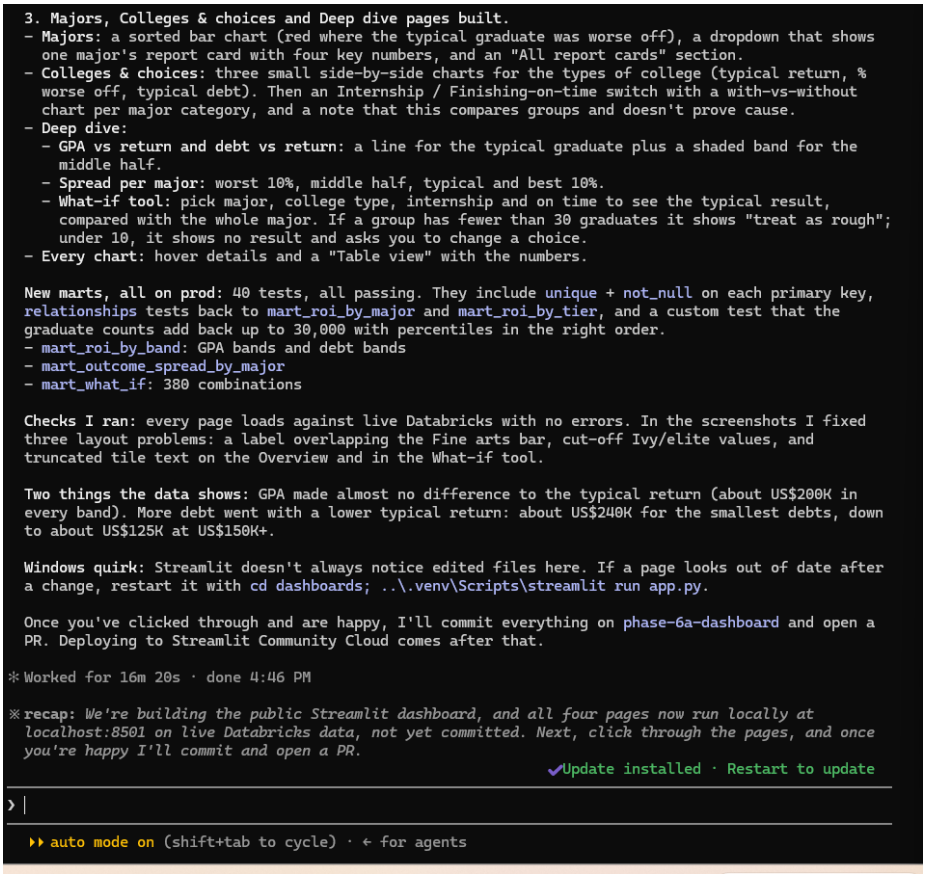


Figure 51. Windows quirk: restart Streamlit if a page looks out of date.

The finished dashboard pages:

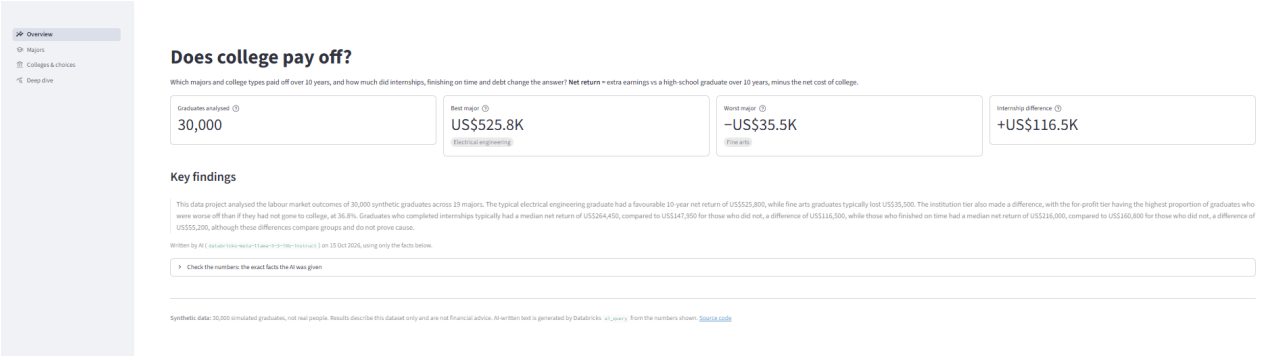


Figure 52. Overview: headline numbers and the AI key findings paragraph.

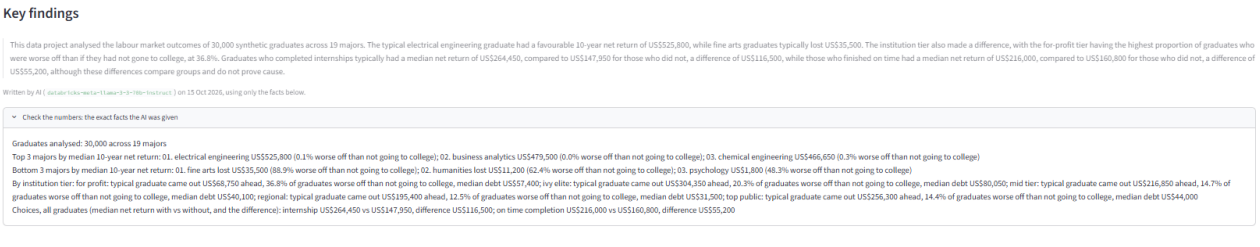


Figure 53. "Check the numbers": the exact fact sheet the AI was given (input_facts), for auditability.

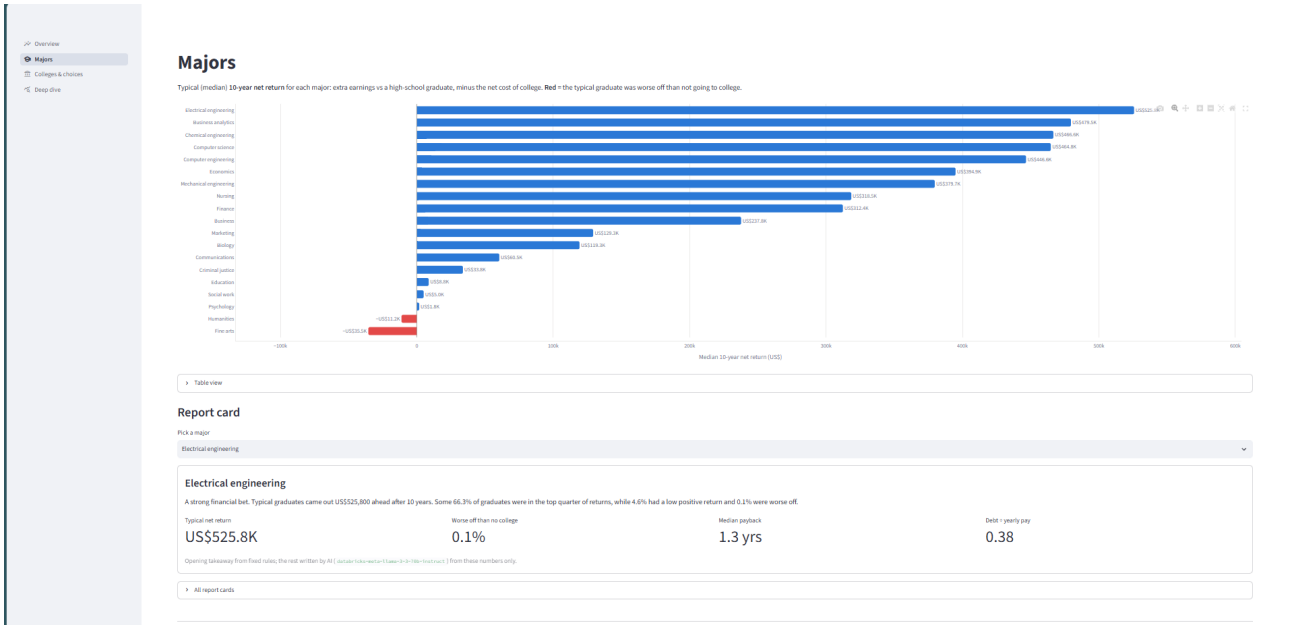


Figure 54. Majors: median 10-year net return per major (red = worse off) and the selected major's report card.



Figure 55. All 19 AI report cards; the opening takeaway comes from SQL rules.

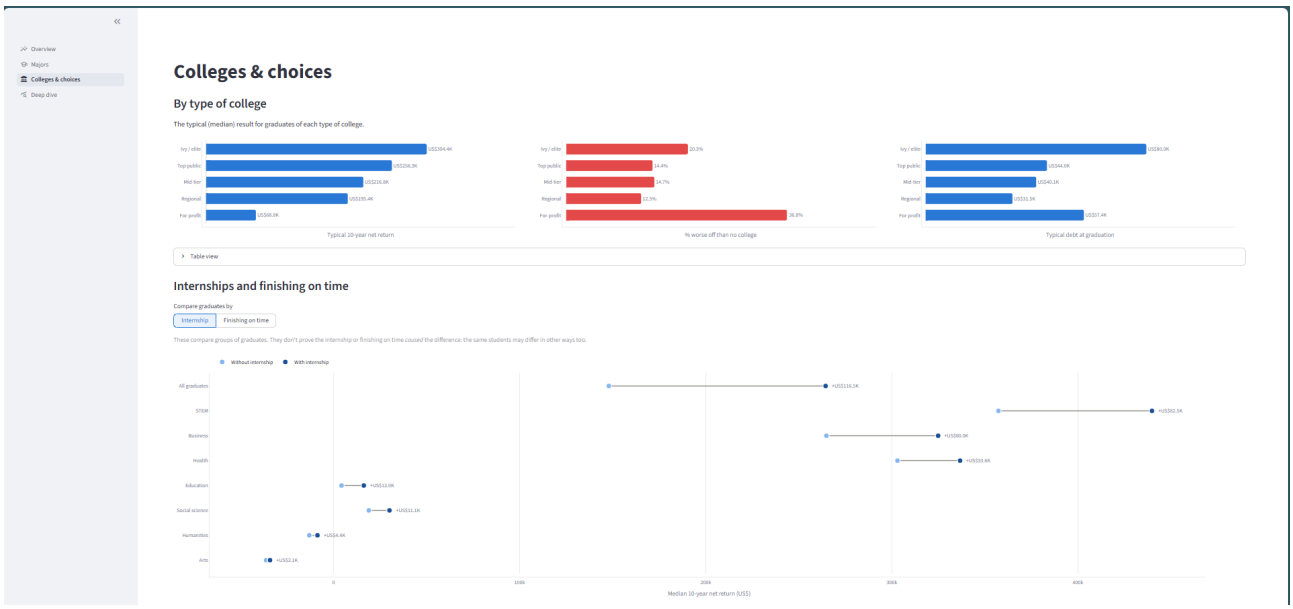


Figure 56. Colleges & choices: tier comparison and the internship effect by category.

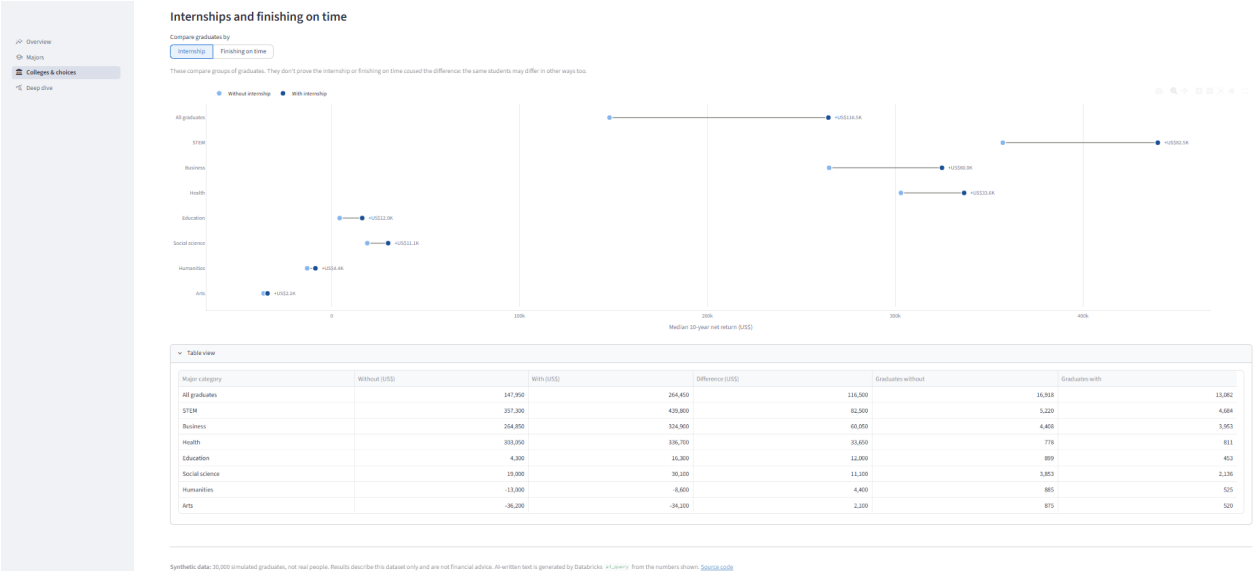


Figure 57. Internship table view: group sizes shown next to each difference.

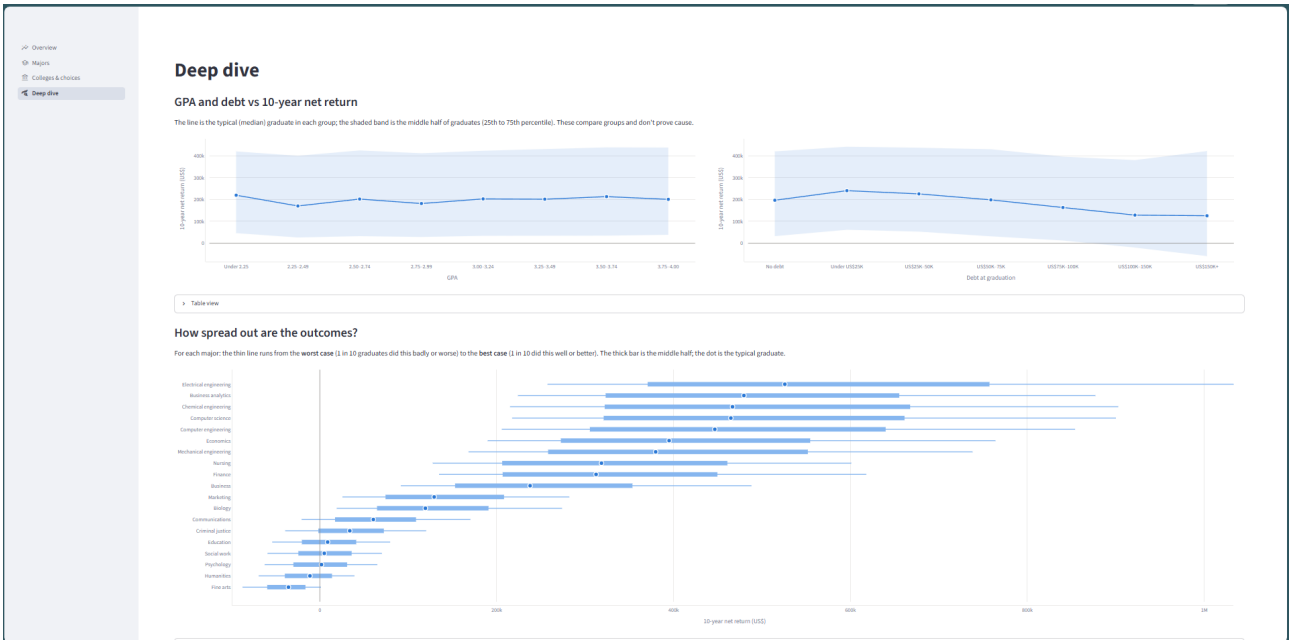


Figure 58. Deep dive: GPA and debt vs return (median line + middle-half band) and the spread of outcomes per major.

Prompt given to Claude Code: `Polish before deploying:`

```
1. Fix mart_key_findings to use "worse off" instead of "lost", and rebuild.
2. Make all chart text bigger (axis labels, tick labels, value labels, captions) so it reads well on a laptop and on mobile.
3. Stop the top value label on the Majors chart overlapping the toolbar.
4. On the debt chart, add a short note that "no debt" grads did slightly worse than small-debt grads, as a group difference, not cause.
5. Fix the fine arts report card's last phrase about debt.
Show me locally, then commit, PR, merge, and walk me through deploying to Streamlit Community Cloud.
```

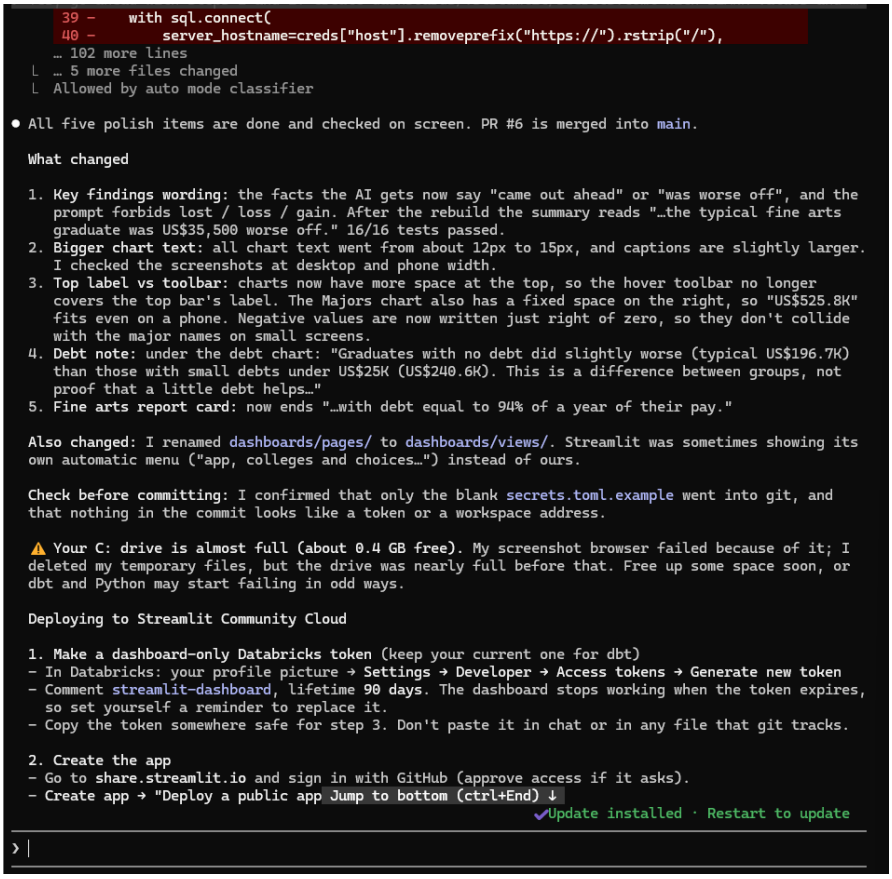


Figure 59. Polish done (PR #6). Pages folder renamed to views/ to stop Streamlit's automatic menu. Warning: C: drive nearly full.

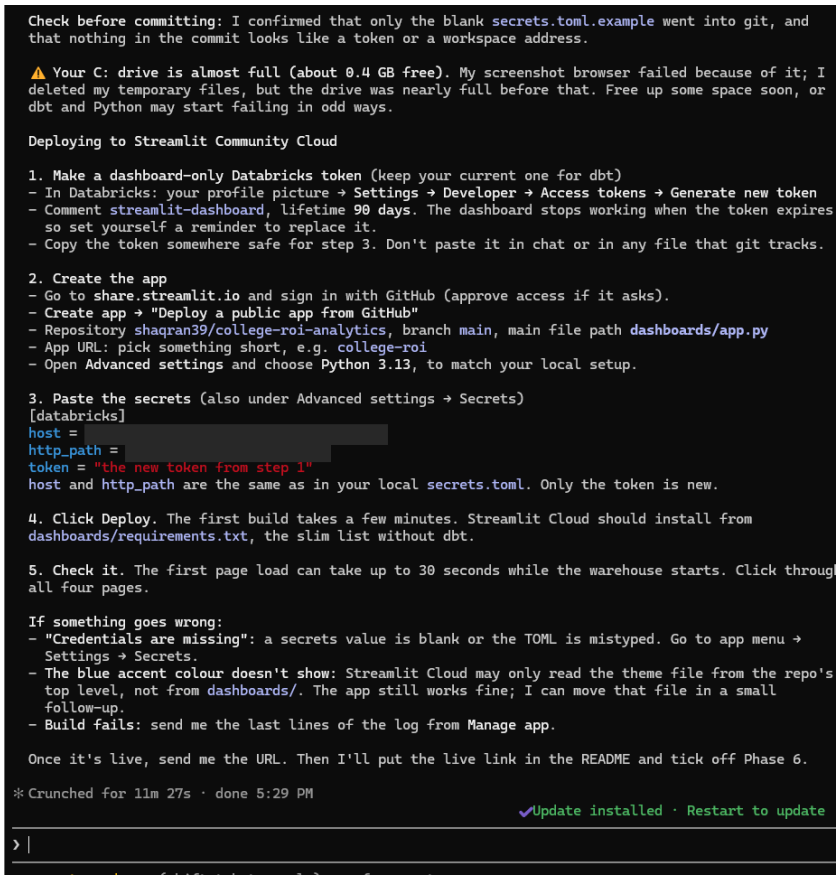


Figure 60. Deployment steps for Streamlit Community Cloud.

Deploy: share.streamlit.io, sign in with GitHub, Create app, repo shaqran39/college-roi-analytics, branch main, main file dashboards/app.py, Advanced settings: Python 3.13 and paste the secrets, Deploy. Then:

Prompt given to Claude Code: `The app is live at [link]. Add the live link to the top of the README, tick Phase 6, commit, PR and merge.`

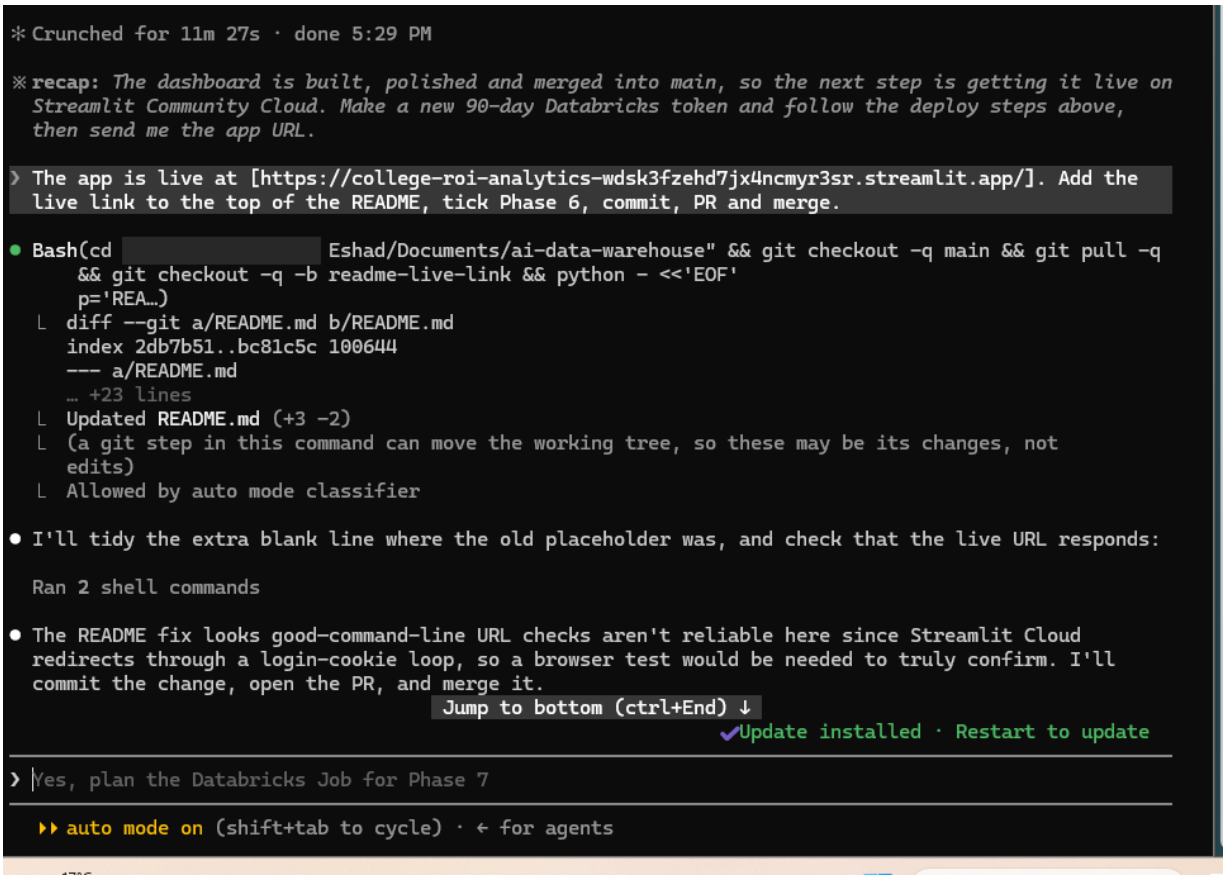


Figure 61. README gets the live link (PR #7).

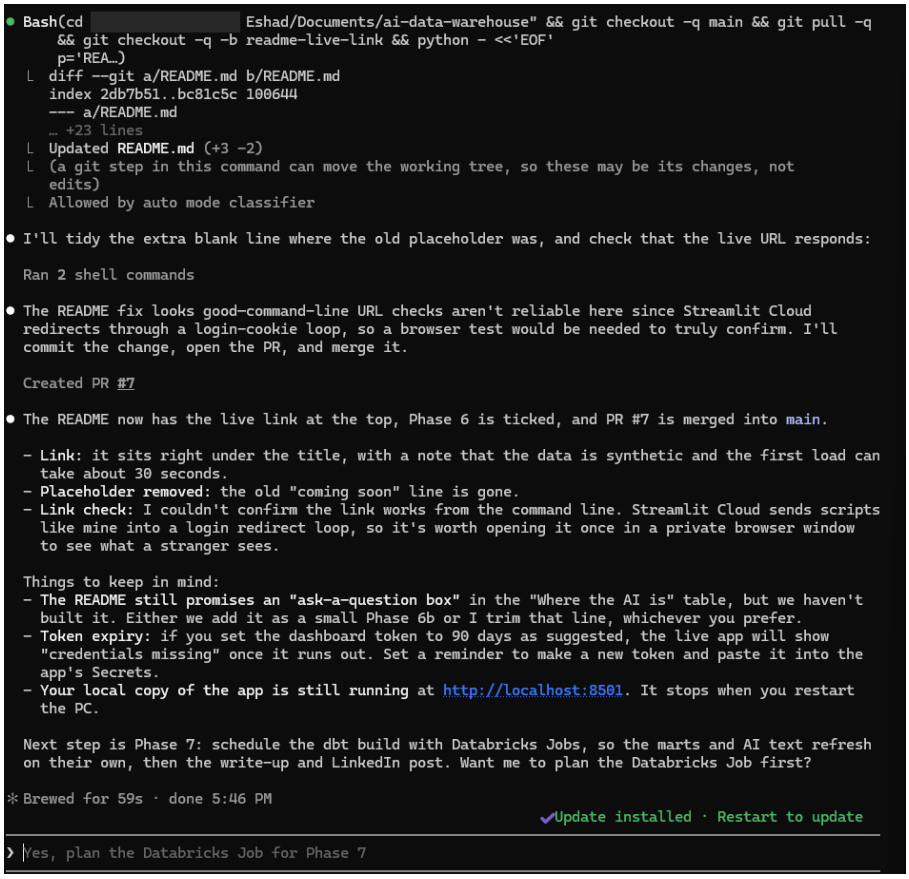


Figure 62. Reminders: check the link in a private window; the token expires after 90 days.

6.10 Phase 7: Automation with a Databricks Job

Prompt given to Claude Code: `Remove the "ask-a-question box" promise from the README. Then yes, plan the Databricks Job for Phase 7: when a new college_major_roi.csv is uploaded to the landing volume, it should automatically reload the raw table, run dbt build on prod (including the AI marts), and fail loudly if any test fails. Explain each step simply. Plan first.`

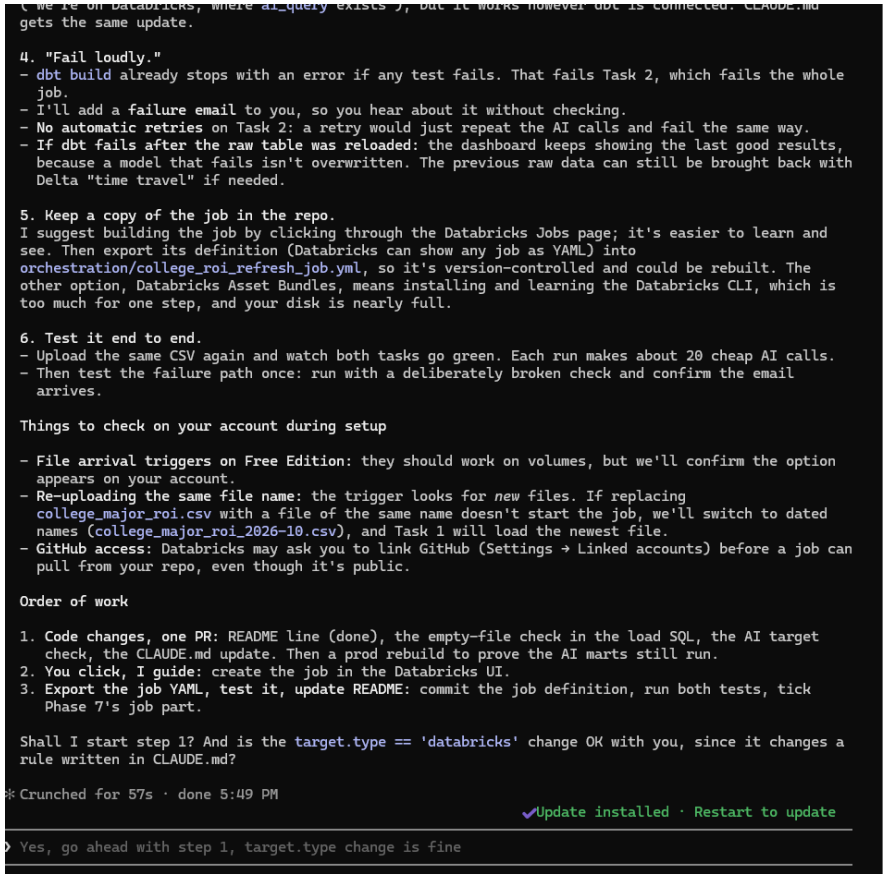


Figure 63. Job plan: fail loudly, failure email, no retries (retries would repeat AI calls), keep a YAML copy in the repo, test end to end.

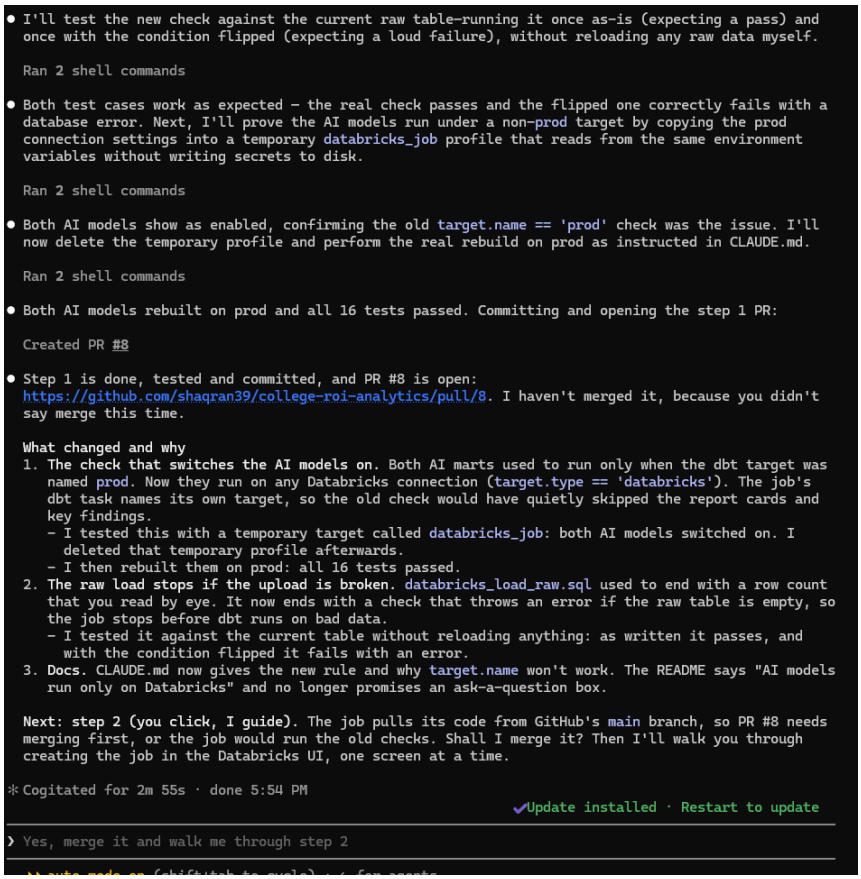


Figure 64. PR #8: AI models now enabled on any Databricks target (target.type), plus an empty-file check in the load SQL. Both tested.

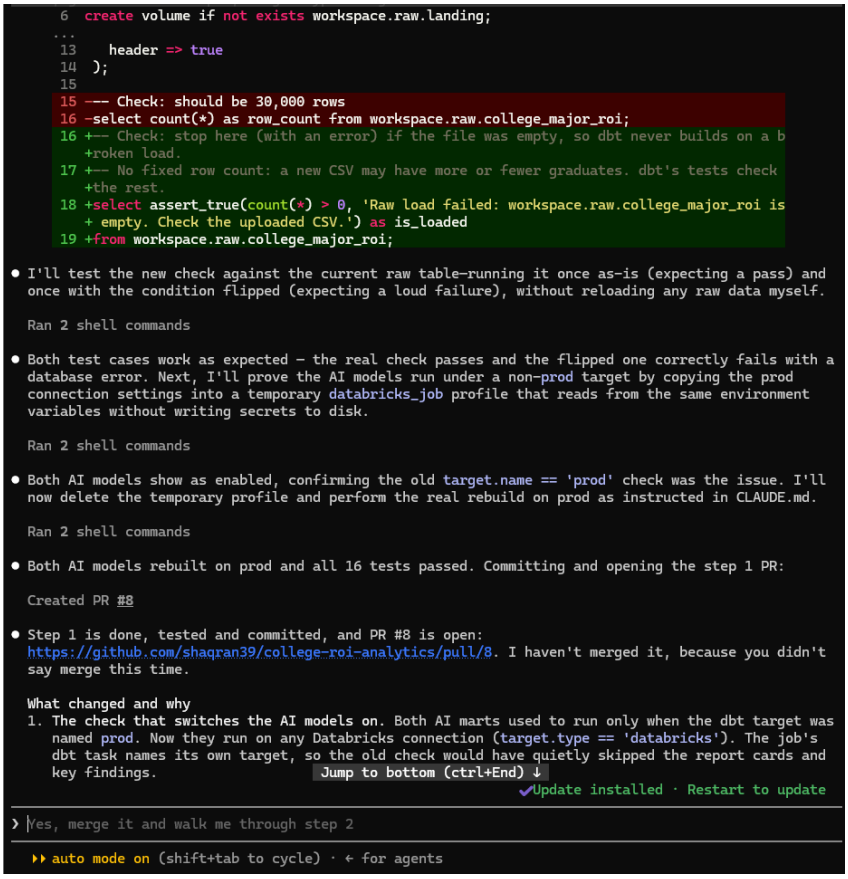


Figure 65. The empty-file check uses assert_true so the job stops before dbt runs on a broken load.

Why this matters: The Job's dbt task writes its own profile with its own target name, so `target.name == 'prod'` would have silently skipped the AI marts. The guard became `{{ config(enabled = target.type == 'databricks') }}`.

Creating the Job in the Databricks UI (Jobs & Pipelines, Create, Job, rename to college_roi_refresh):

⚠ Do not add --target prod. Databricks writes its own connection settings for this task, and they have no target called prod, so the run would fail at once. That's exactly why we made the AI check work on any Databricks target.

Click **Create task**. You should see two boxes joined by an arrow: load_raw → dbt_build. The arrow means dbt only runs if the load succeeded.

D. The trigger: run when a file arrives

In the right-hand panel, under **Schedules & Triggers**, click **Add trigger**:
- Trigger type: **File arrival**
- Storage location: /Volumes/workspace/raw/landing/ (keep the slash at the end)
- Under **Advanced**:
  - Minimum time between triggers: **60 seconds**
  - Wait after last change: **60 seconds**, so a slow upload finishes before the job starts

Leave the trigger **Paused** for now. Turn it on in step 3, once a manual run has worked.

If **File arrival** isn't in the list, tell me. Free Edition may not offer it, and then we'd use a schedule instead.

E. Safety settings

Still in the right-hand panel:
- **Maximum concurrent runs**: 1, so two uploads can't run two builds at once.
- **Job notifications → Add notification**: your email address, tick **Failure**. For the first test, also tick **Success** so you can see an email arrive. We'll untick it later.

F. First run, by hand

Click **Run now** at the top right. It reloads the raw table from the CSV that's already in the volume, then runs the full dbt build with about 20 AI calls. Expect about 3–6 minutes.

When it finishes, tell me:
1. Whether both tasks are green, or which one went red.
2. The last few lines of dbt_build's output (click the task in the run → **Output**). The line I'm looking for looks like Done. PASS=… WARN=0 ERROR=0 SKIP=0. **SKIP should be 0**; anything else means the AI models were skipped.

If anything goes red, paste the error and we'll fix it together. Step 3 comes after this run passes: turn on the trigger, re-upload the CSV to test it, test the failure email, then save the job's YAML into the repo.

✻ Worked for 38s · done 6:14 PM

✔ Update installed · Restart to update

Figure 66. Claude Code's guide to creating the job, part 1.

When you choose Git provider, a box pops up:
- Git URL: https://github.com/shaqran39/college-roi-analytics
- Provider: **GitHub**
- Branch: main

If it asks you to link GitHub first: **Settings → Linked accounts → Git integration → GitHub**, sign in, then come back. Both tasks share this Git setting, so you only do it once.

Leave **Retries** at none, then click **Create task**.

C. Task 2: dbt build

Click **+ Add task → dbt**.

Field	Value
Task name	dbt_build
Depends on	load_raw (usually filled in for you)
Source	Git provider (the same repo, already set)
Project directory	dbt
dbt commands	**dbt build** only. Delete the default dbt deps line; this project has no packages
SQL warehouse	same warehouse as Task 1
Catalog / Schema	workspace / analytics
Profiles directory	**leave empty**
Retries	none

⚠ Do not add --target prod. Databr Jump to bottom (ctrl+End) ↓ n settings for this task, and they

✔ Update installed · Restart to update

Figure 67. Part 2: the dbt task values.

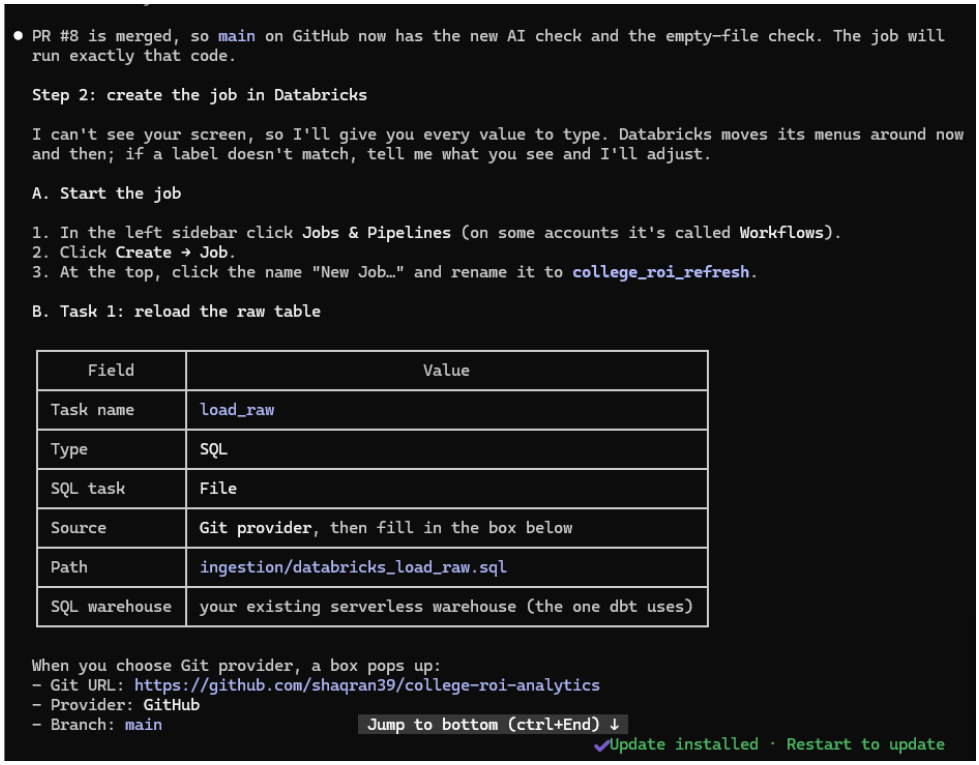


Figure 68. Part 3: trigger, safety settings, first run.

Task	Setting	Value
load_raw	Type / Source	SQL file / Git provider: https://github.com/shaqran39/college-roi-analytics, GitHub, branch main
load_raw	Path / Warehouse	ingestion/databricks_load_raw.sql / Serverless Starter Warehouse
dbt_build	Type / Depends on	dbt / load_raw (Run if: All succeeded)
dbt_build	Project directory	dbt (easy to miss: if empty, dbt runs in the wrong folder)
dbt_build	Commands	dbt build only (edit the first default line, delete dbt seed and dbt run)
dbt_build	Warehouse / catalog / schema	Serverless Starter Warehouse / workspace / default (models set their own schemas)
dbt_build	Compute / environment	Serverless / dbt-default
dbt_build	Retries	No retries AND untick "serverless auto-optimization" (it can add up to 3 retries)
Job	Trigger	File arrival on /Volumes/workspace/raw/landing/, 1 min between triggers, 1 min wait after last change
Job	Notifications	Email on Failure (Success only while testing)
Job	Max concurrent runs	1

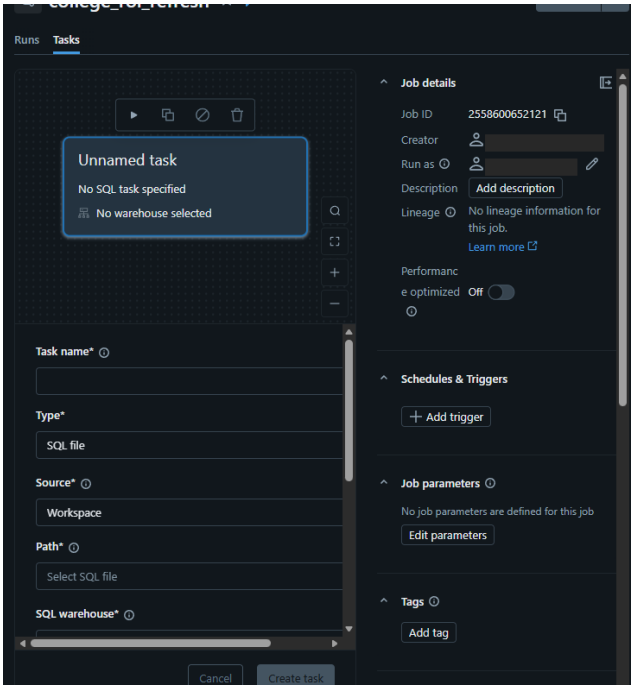


Figure 69. Empty task form: Type SQL file, Source Workspace (change to Git provider).

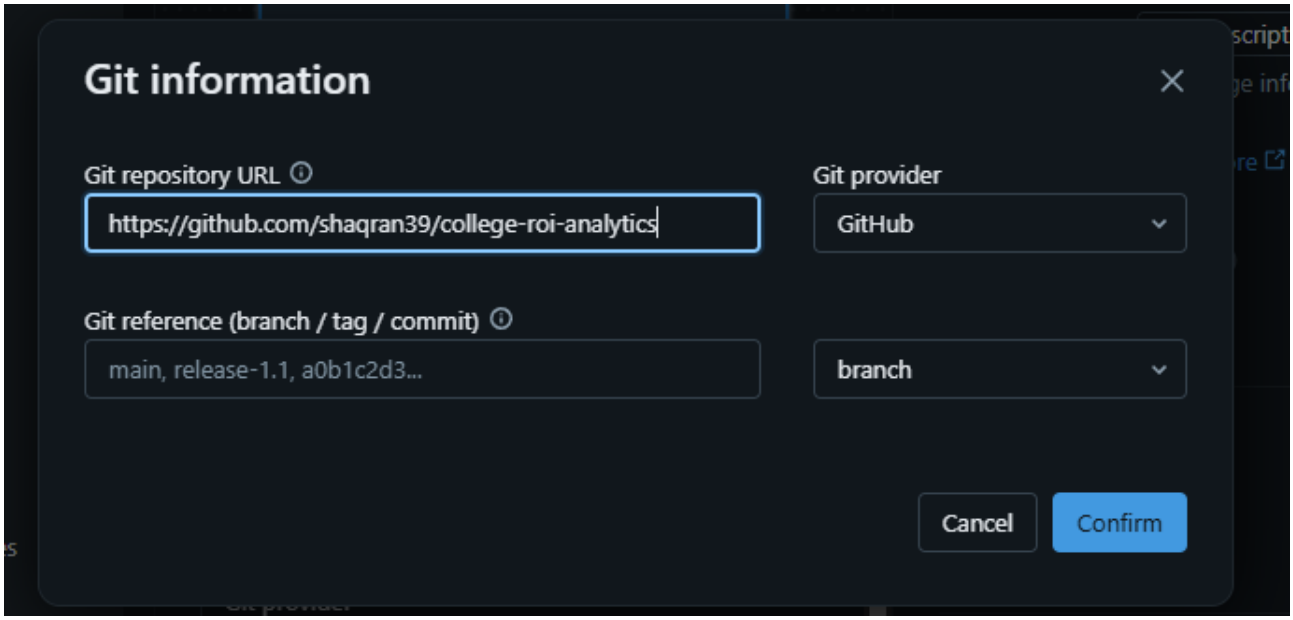


Figure 70. Git information: fill the branch box with main.

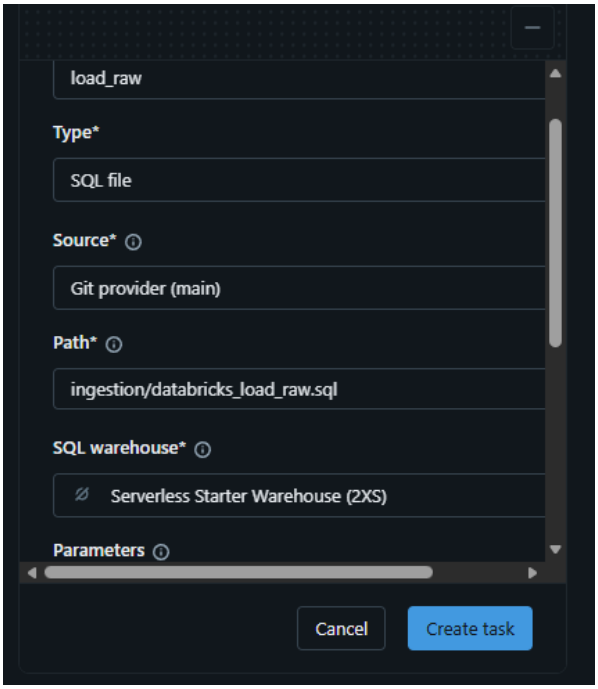


Figure 71. load_raw filled in correctly.

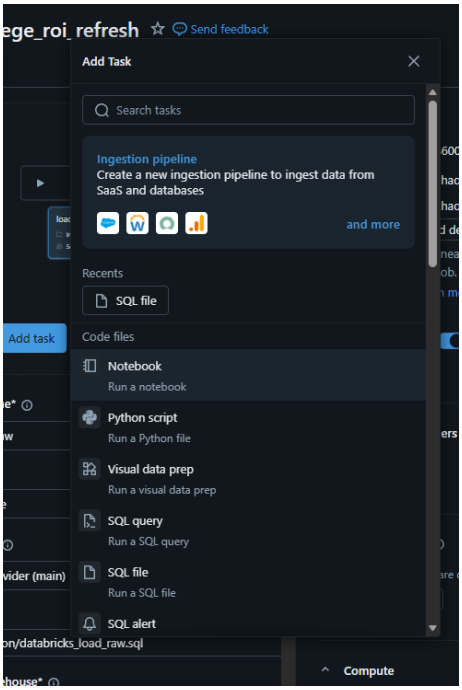


Figure 72. Add Task picker: search "dbt".

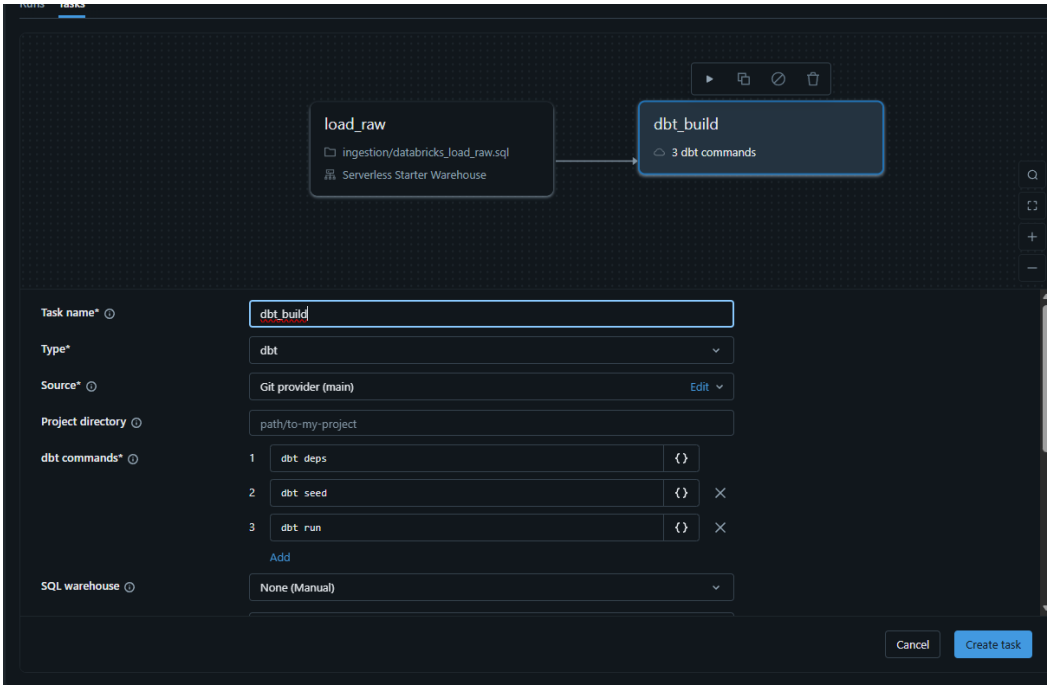


Figure 73. dbt task defaults: replace the three default commands with dbt build.

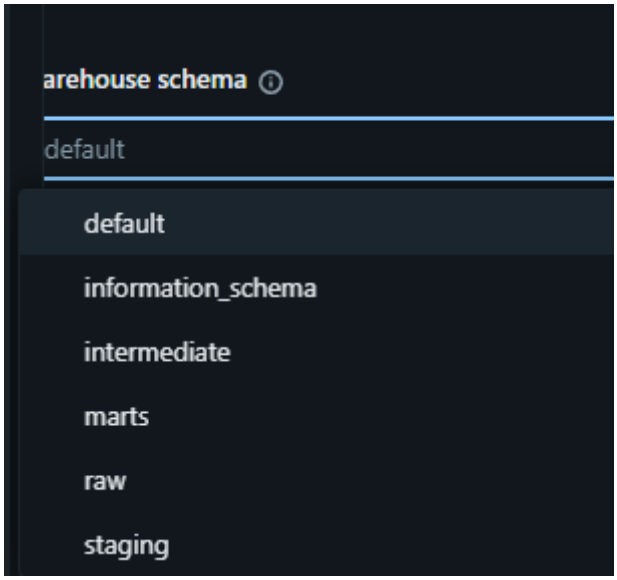


Figure 74. Schema dropdown: "analytics" does not exist, choose default.

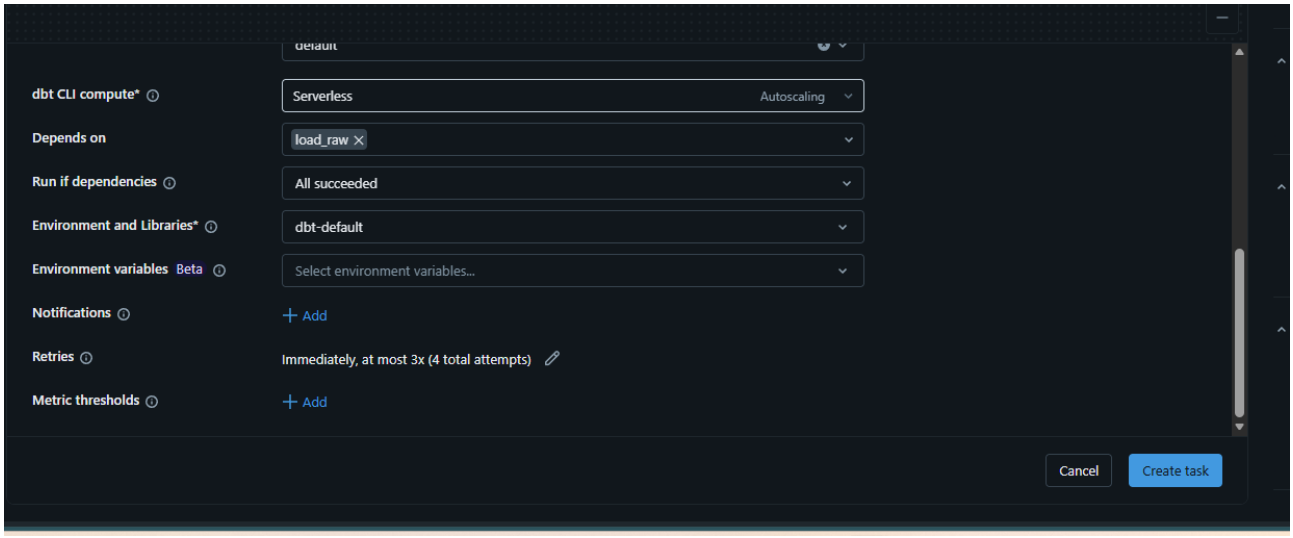


Figure 75. Bottom of the dbt task: retries default to 3, change it.

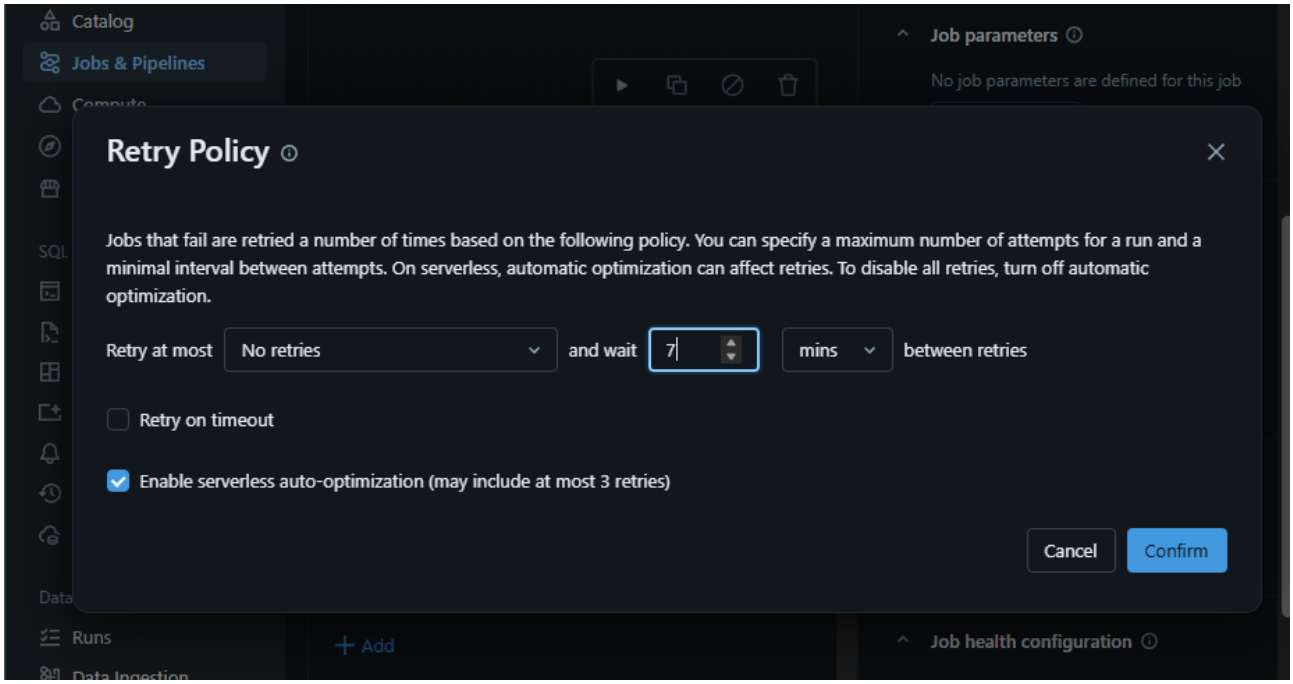


Figure 76. Retry policy: No retries and untick serverless auto-optimization.

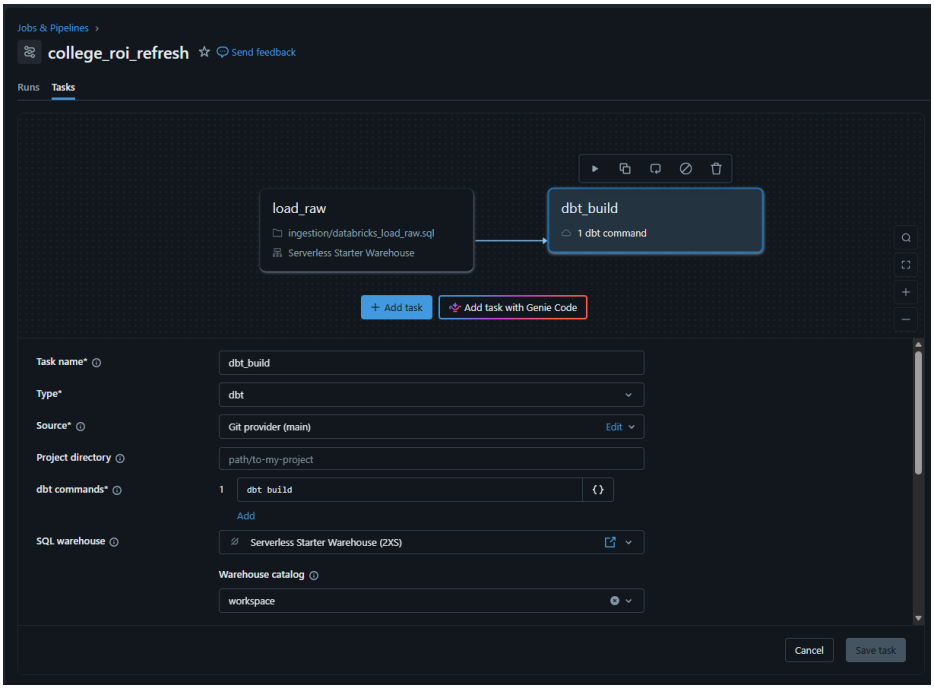


Figure 77. Both tasks linked, but Project directory still empty: type dbt.

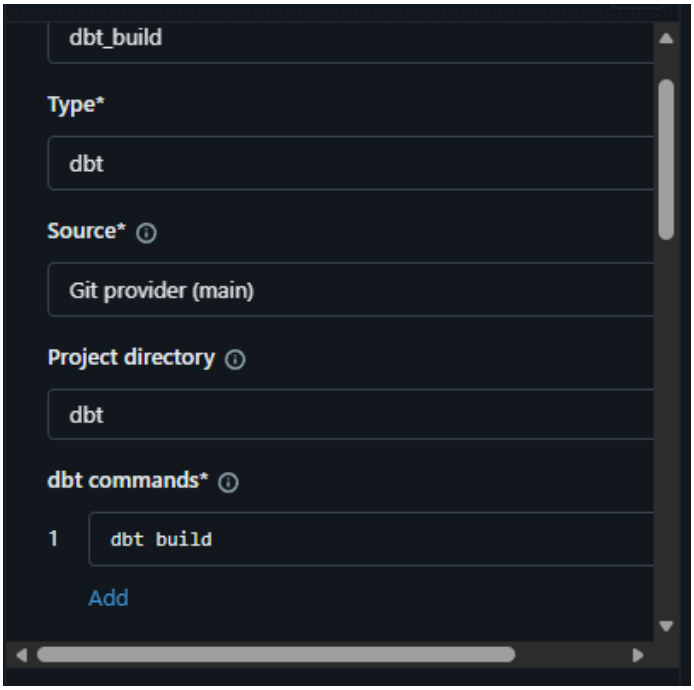


Figure 78. Project directory dbt and command dbt build.

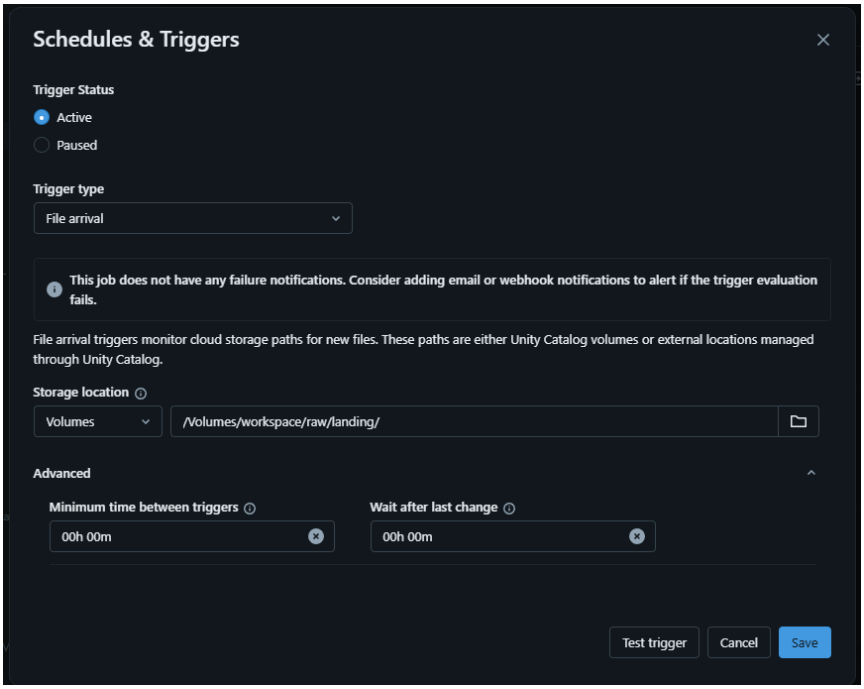


Figure 79. File arrival trigger: start Paused, set both waits to 1 minute.

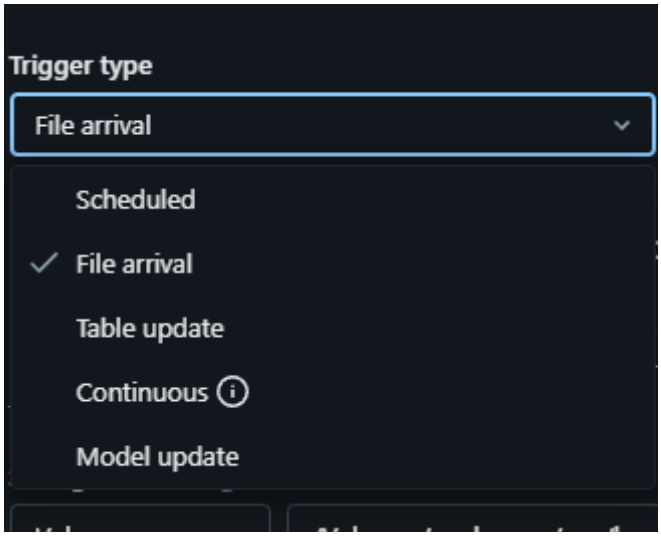


Figure 80. Trigger types: File arrival is the one that fits "upload a CSV and everything updates".

First test: Run now (manual).

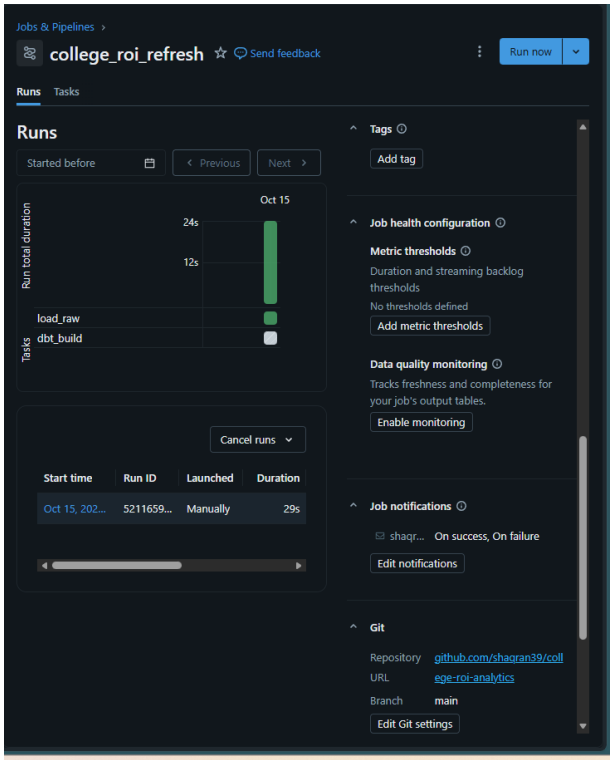


Figure 81. First manual run: load_raw green, dbt_build still running. The Gmail "subscribed" email is only a confirmation, not a result.

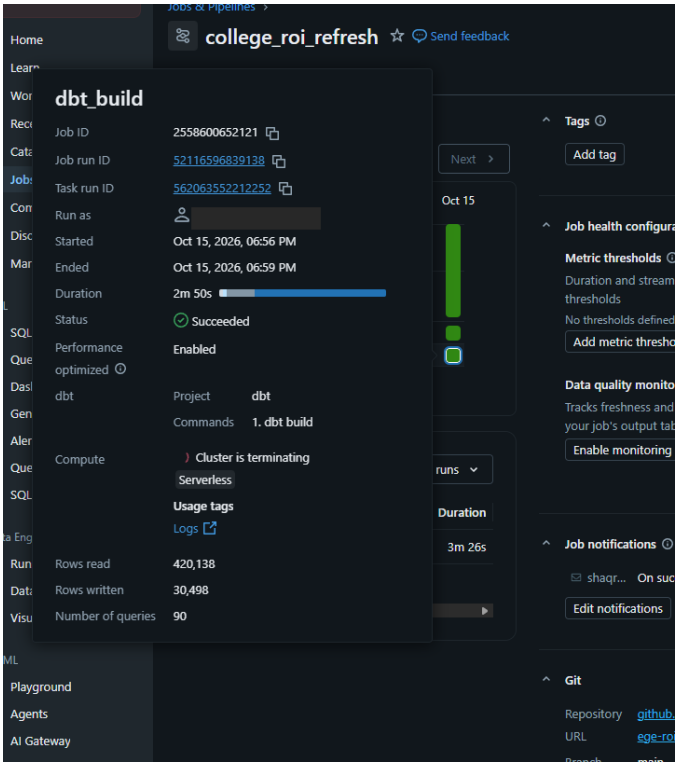


Figure 82. dbt_build succeeded: 2m 50s, 90 queries, 30,498 rows written.

Check the AI actually ran inside the job:

```
select major, generated_at from workspace.marts.mart_major_report_cards limit 3;
```

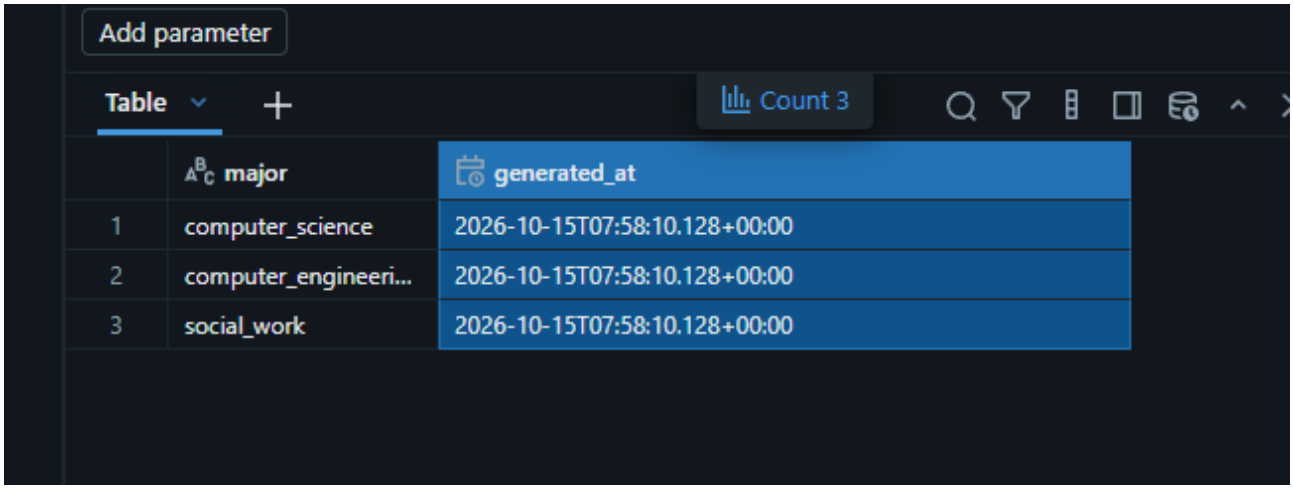


Figure 83. generated_at 07:58 UTC = 6:58 PM Sydney: the AI report cards were rewritten by the automated run.

Second test: upload the same file again with the trigger Active.

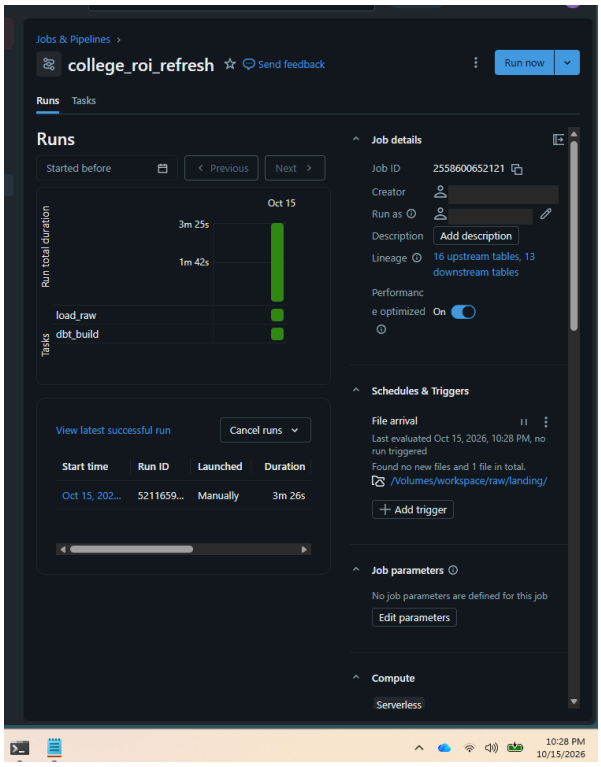


Figure 84. Trigger panel: "Found no new files". Re-uploading a file with the same name does not count as a new file.

Prompt given to Claude Code: `The file arrival trigger only fires for new file names, not re-uploads with the same name. Update ingestion/databricks_load_raw.sql so it loads only the NEWEST csv in /Volumes/workspace/raw/landing/ (by file modification time), whatever it's called. Test it, then commit, PR and merge. Also add a note to the README: "To refresh the data, upload a new CSV with a new name, e.g. college_major_roi_2026-11.csv".`

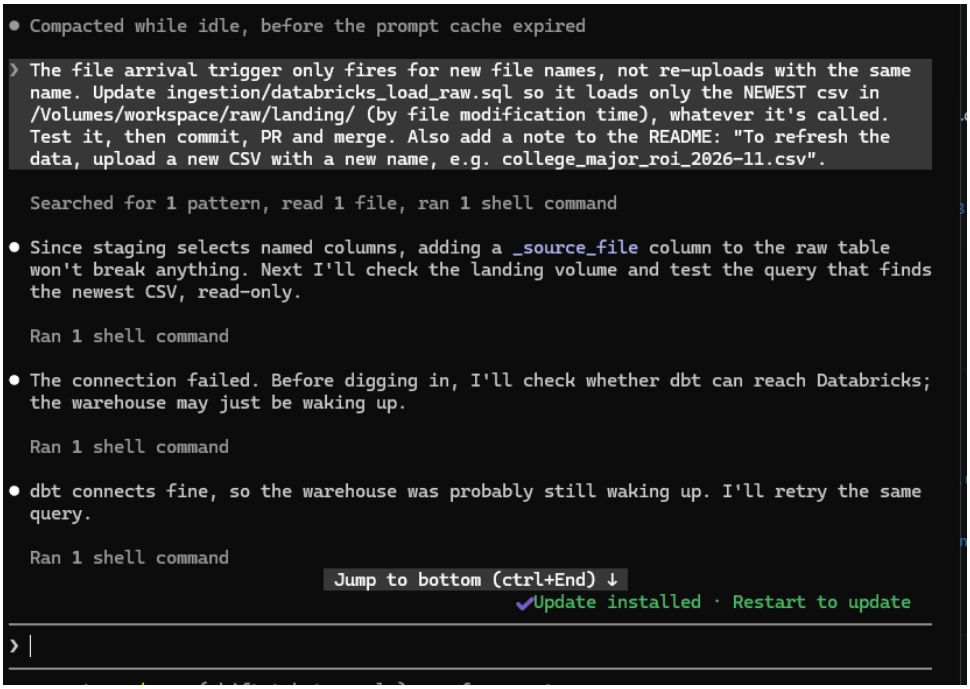


Figure 85. Claude tests the newest-file logic read-only and on a throwaway table.

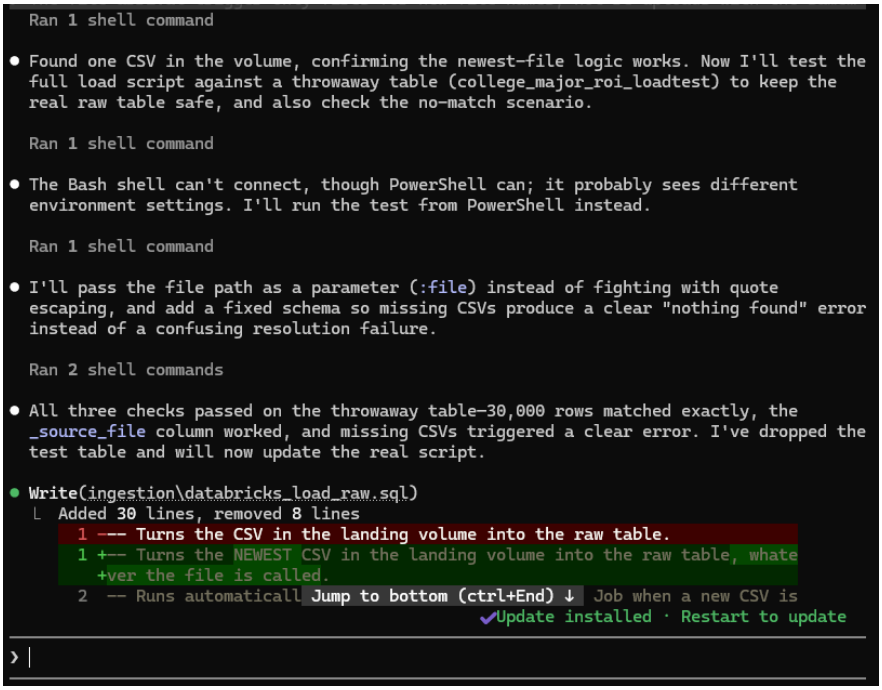


Figure 86. Tests against a throwaway table so the real raw table stays safe.

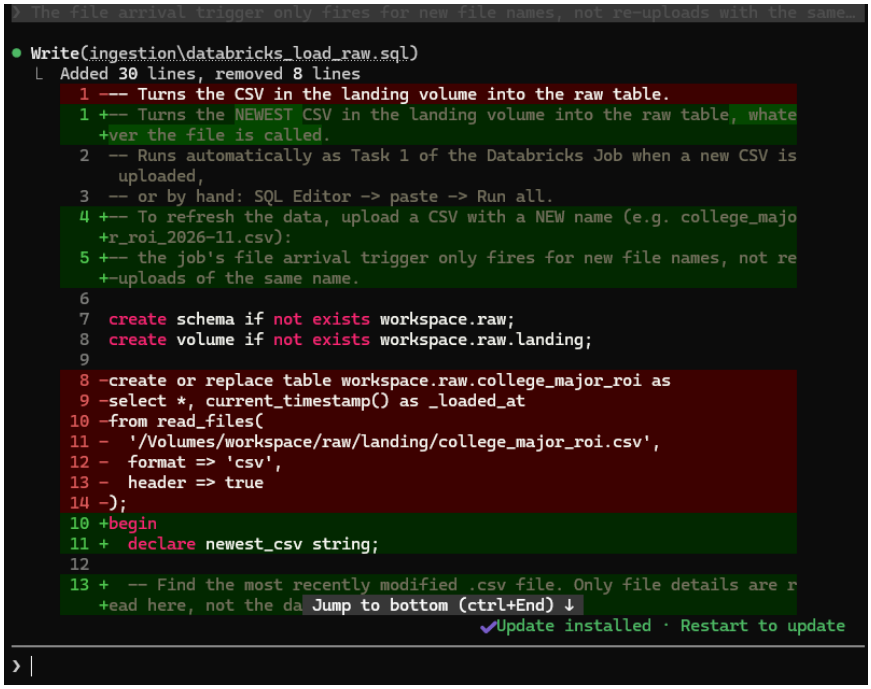


Figure 87. The new script (version 1) used a begin...end scripting block.

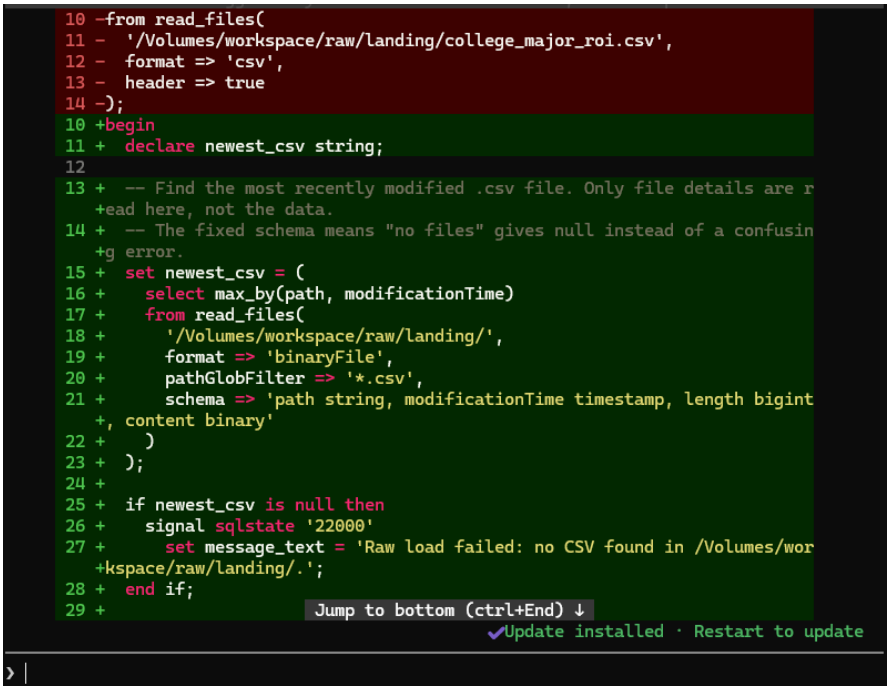


Figure 88. Finds newest file with max_by over file modification time.

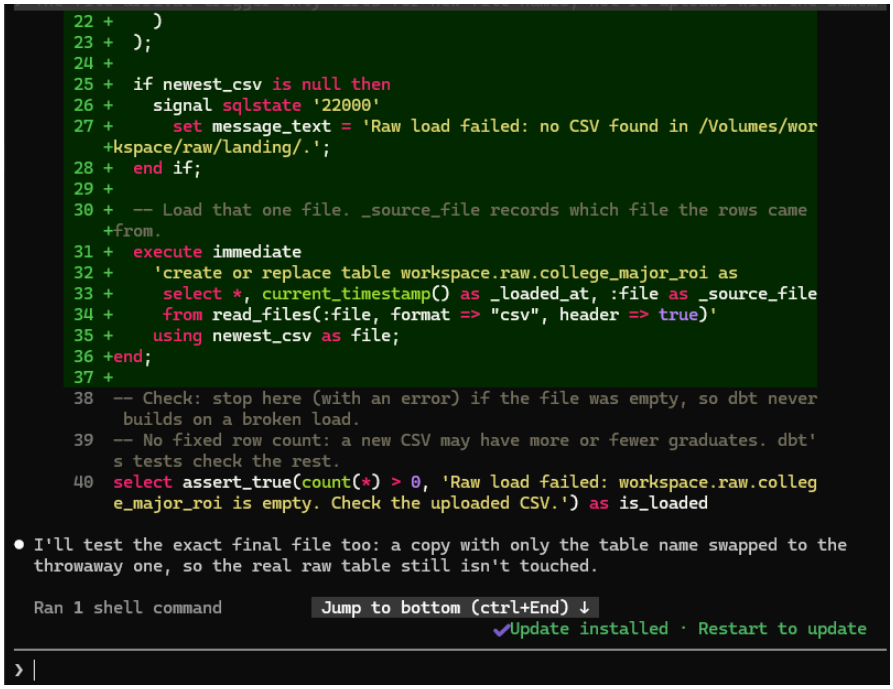


Figure 89. execute immediate with the file path as a parameter.

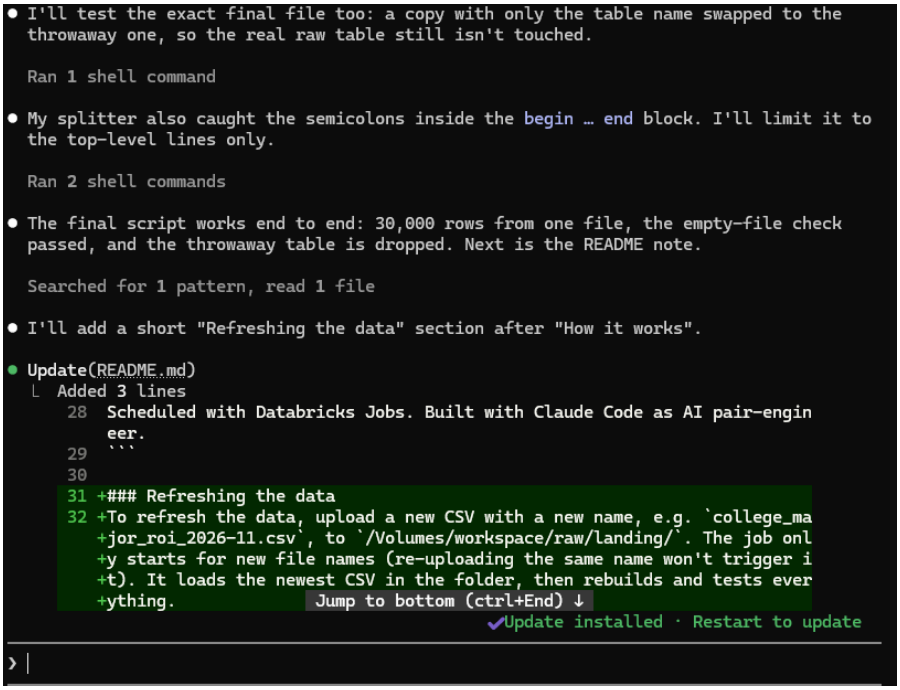


Figure 90. README gains a "Refreshing the data" section.

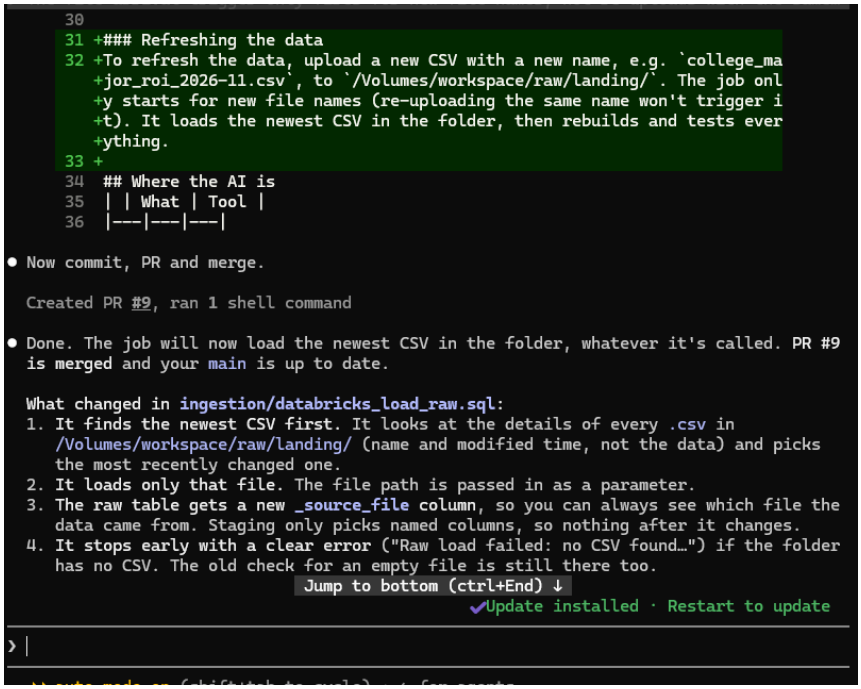


Figure 91. PR #9 merged.

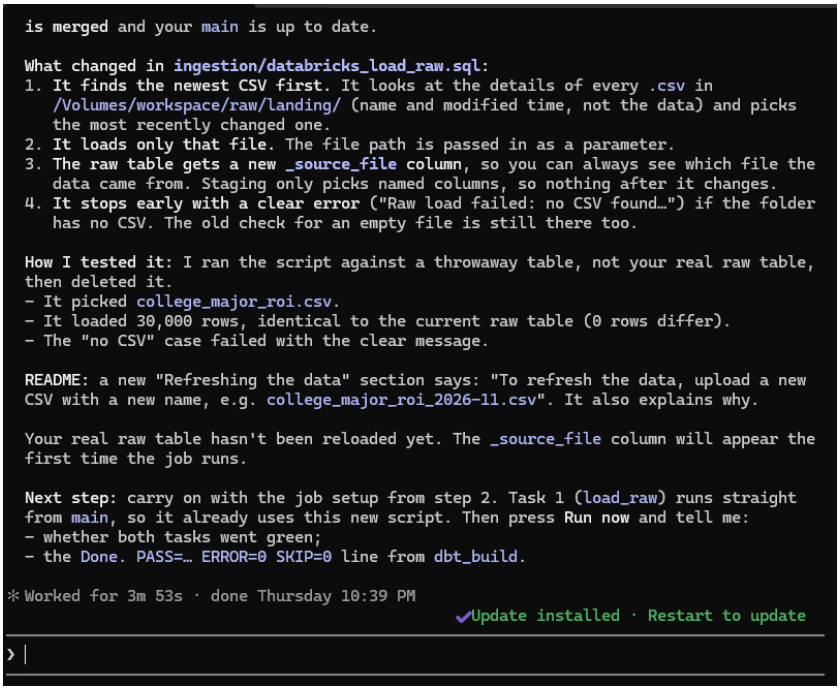


Figure 92. Tested via a connection that handles scripting blocks, not the way the Job runs it.

Third test: upload a renamed copy (college_major_roi_2026-10.csv). The trigger fired by itself, but load_raw failed:

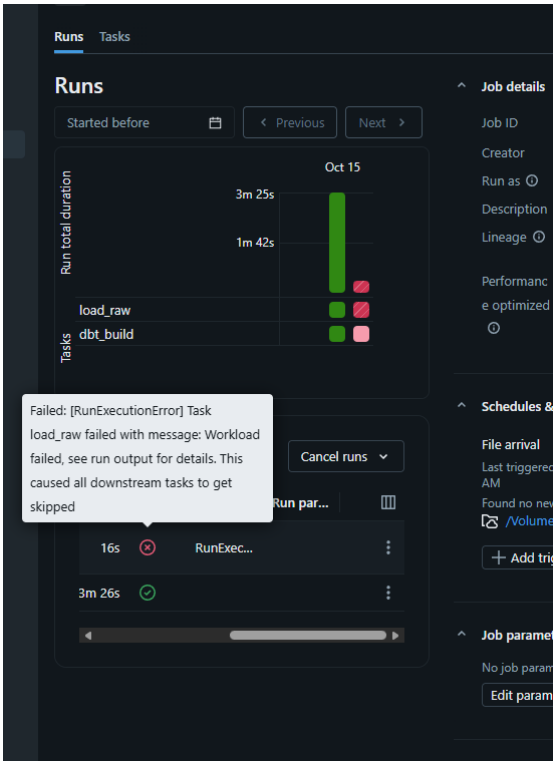


Figure 93. File-arrival run: load_raw failed, so dbt_build was skipped. The dashboard kept the last good data (the safety net worked).

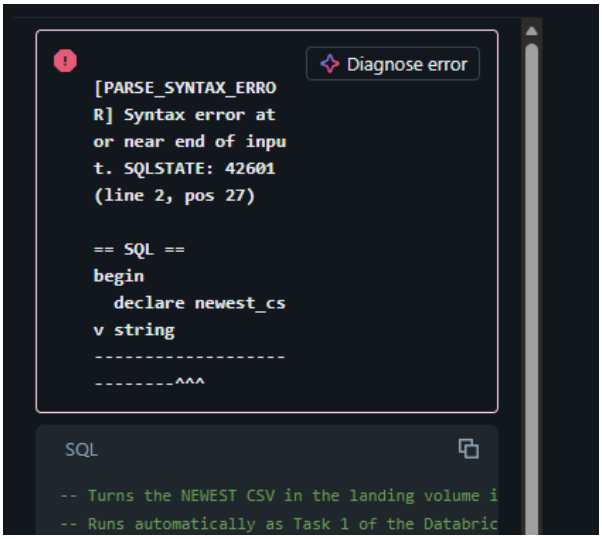


Figure 94. The error: PARSE_SYNTAX_ERROR at "begin declare newest_csv string". The SQL file task splits the file on semicolons.

Prompt given to Claude Code: `The Databricks job's load_raw task failed after the file arrival trigger fired. Error: [PARSE_SYNTAX_ERROR] Syntax error at or near end of input (line 2, pos 27), on "begin declare newest_csv string". The SQL file task splits on semicolons, so the begin...end block breaks.`

`Rewrite databricks_load_raw.sql with NO begin/end, declare or execute immediate. Use a single create-or-replace statement that reads all CSVs in /Volumes/workspace/raw/landing/ with read_files, uses _metadata.file_modification_time to keep only the rows from the newest file, and adds _source_file. Keep the empty-table check as a separate statement. Test it by running each statement separately (the way the job does), then commit, PR and merge.`

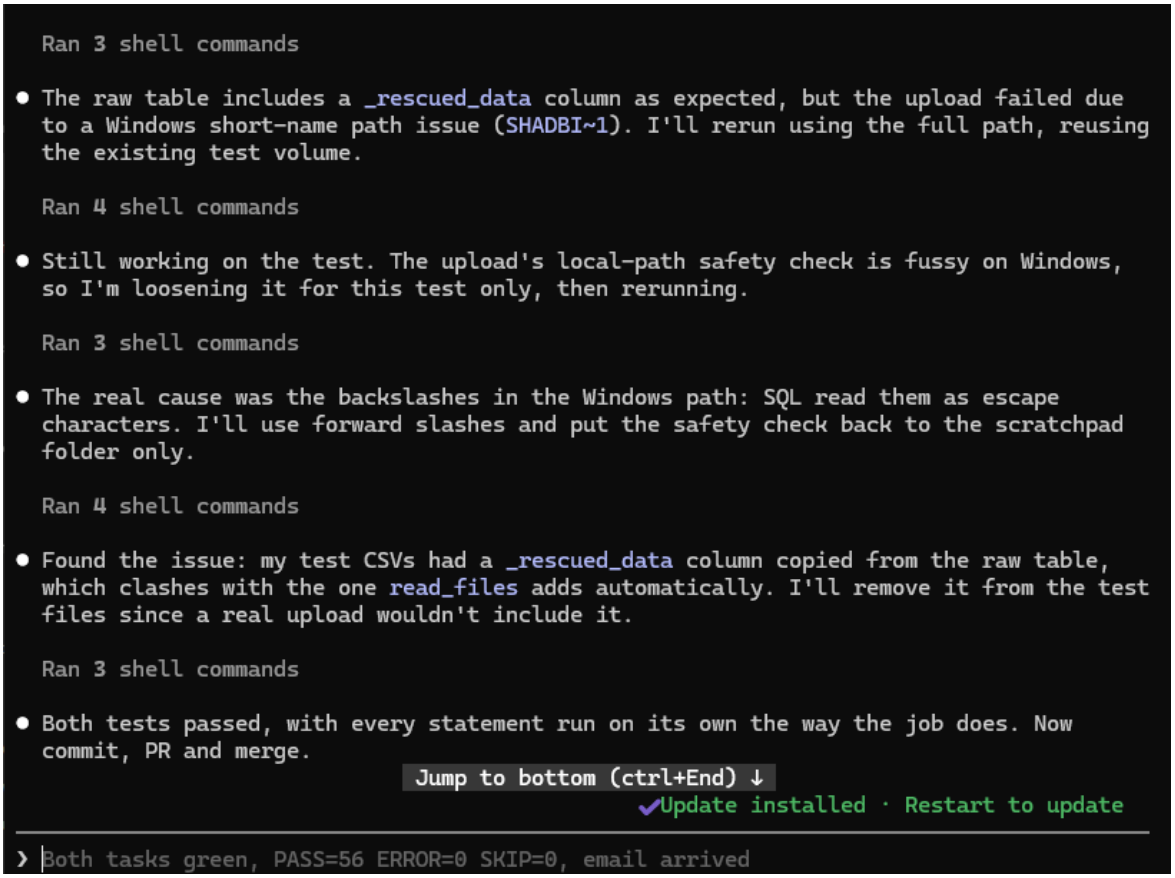


Figure 95. Testing each statement separately, the way the Job runs it.

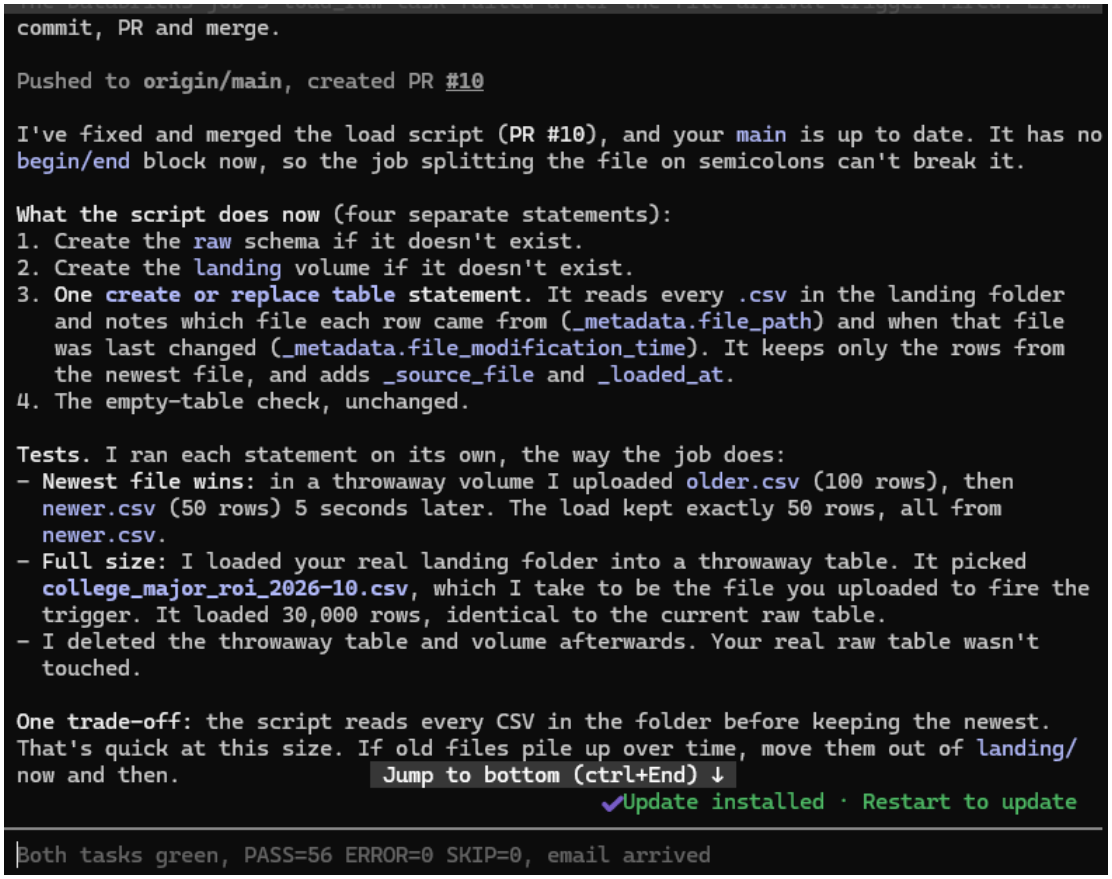


Figure 96. PR #10: four plain statements, newest-file-wins test (older.csv 100 rows vs newer.csv 50 rows, kept 50).

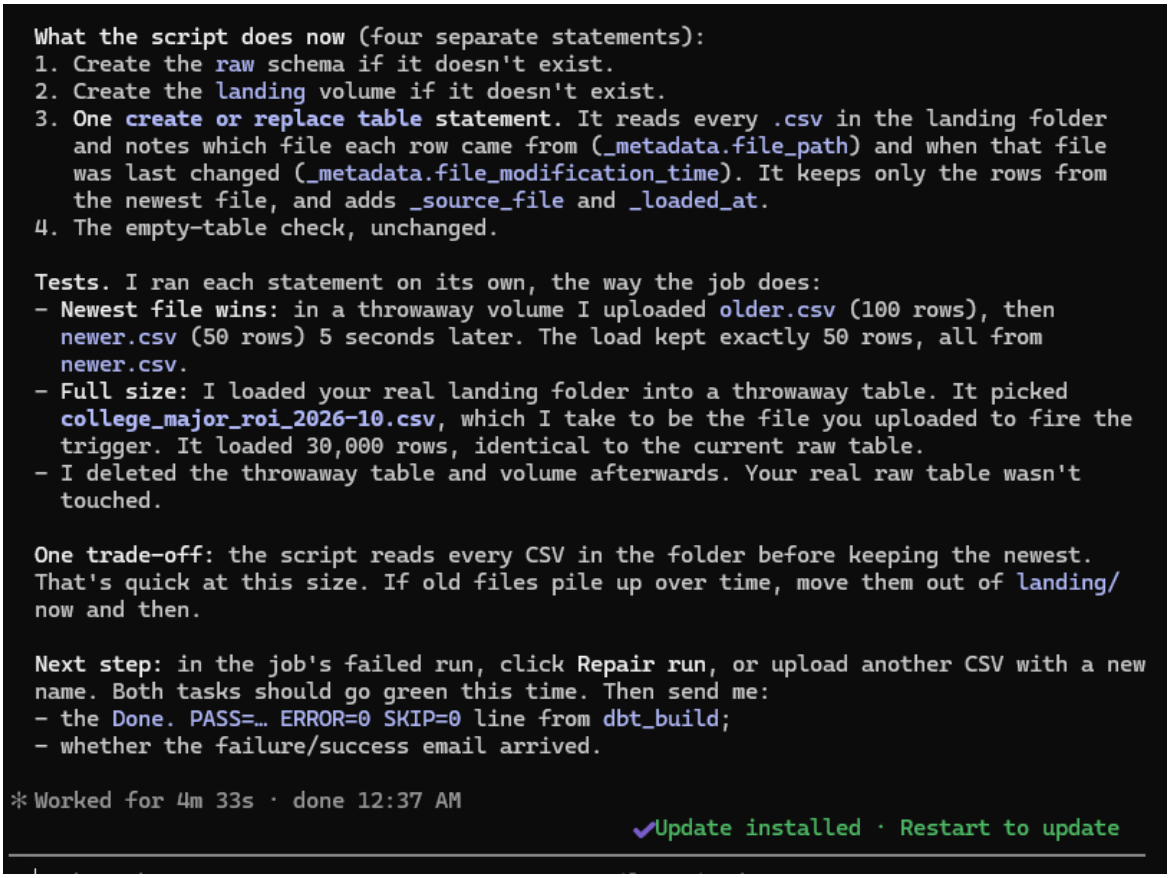


Figure 97. Trade-off: it reads every CSV before keeping the newest, so tidy the landing folder now and then.

Watch out: The grey text in the Claude Code prompt box ("Both tasks green, PASS=56...") was Claude Code's autocomplete suggestion, not a real result. Only report results you have actually seen.

Fourth test: upload college_major_roi_2026-10b.csv, touch nothing.

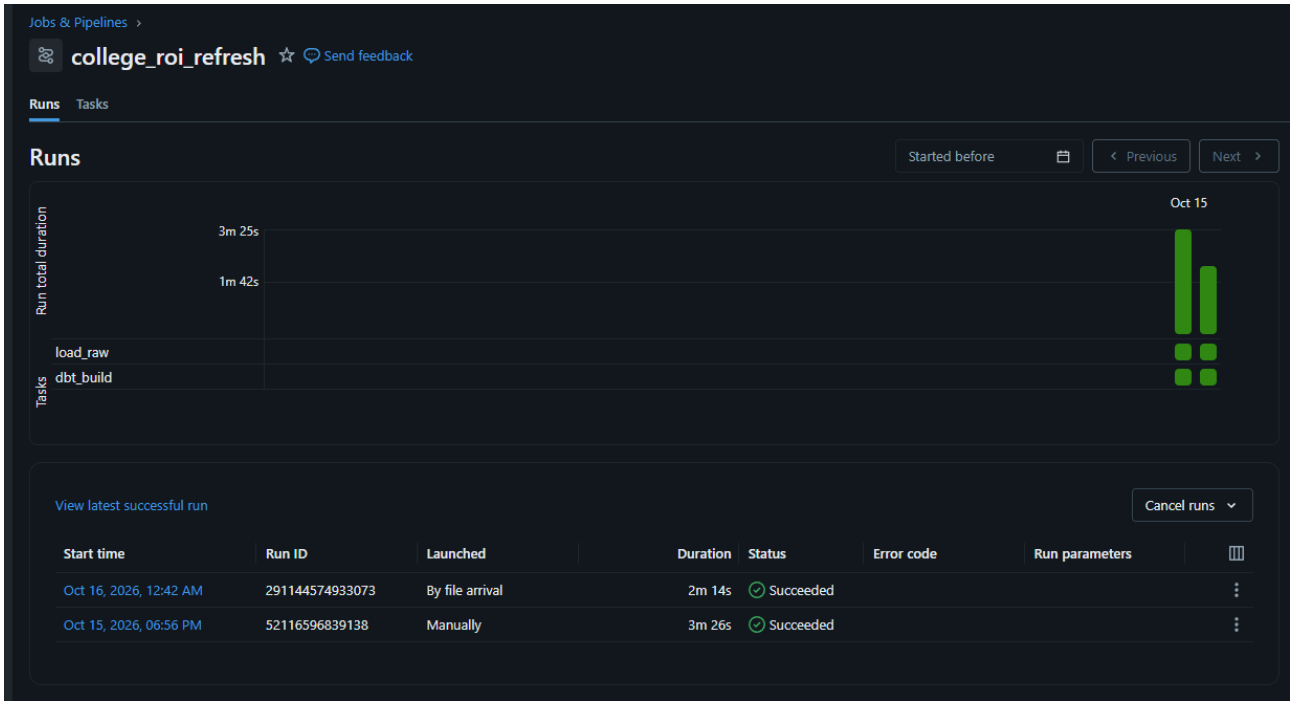


Figure 98. SUCCESS: run launched "By file arrival", both tasks green in 2m 14s. The pipeline is fully automated.

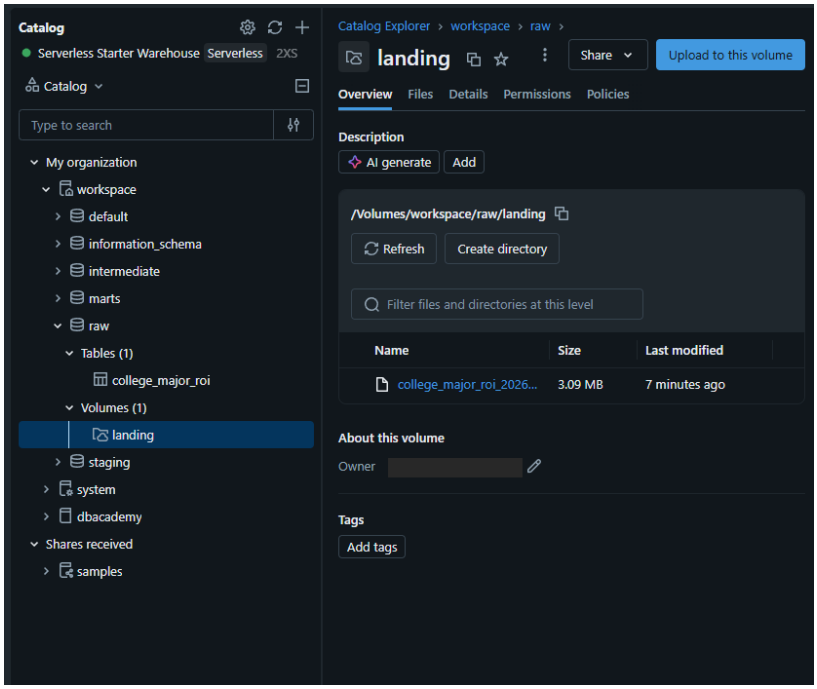


Figure 99. Landing folder tidied: only the newest CSV kept.

```
select distinct _source_file from workspace.raw.college_major_roi;   -- shows the newest file name
```

Prompt given to Claude Code: `The file-arrival run succeeded. Export the Databricks job definition as YAML into orchestration/college_roi_refresh_job.yml (no secrets in it), tick Phase 7 automation in the README, add a short "How the automation works" section, then commit, PR and merge.`

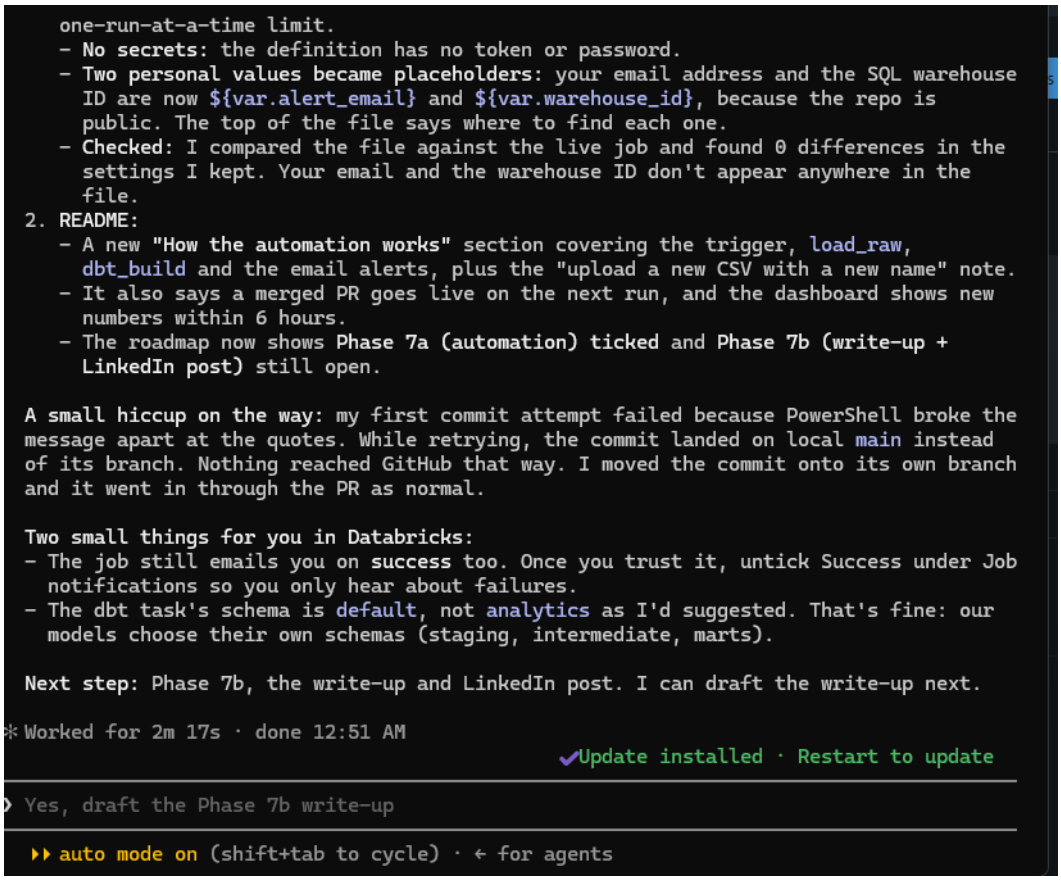


Figure 100. Job definition exported to YAML with email and warehouse ID as placeholders (PR #11).

6.11 Power BI (explored, left out on purpose)

A Power BI version was started and stopped once the data model was loaded, because Shaqran wants to learn Power BI properly first. What worked, so it can be picked up later:

1. Install Power BI Desktop (Microsoft Store). Free up disk space first.
2. Make a powerbi token: 90 days, BI Tools / sql.
3. Get data, Azure Databricks: server hostname + HTTP path, Data connectivity mode **Import**.
4. Sign in tab: **Personal Access Token** (not Azure Active Directory).
5. Navigator: workspace, marts: tick dim_institution_tier, dim_major, dim_region, fct_graduate_outcomes, mart_key_findings, mart_major_report_cards. Load.
6. Model view: keep fact-to-dimension links (1 to many). Delete the auto-detected mart_key_findings to mart_major_report_cards link (it is wrong).

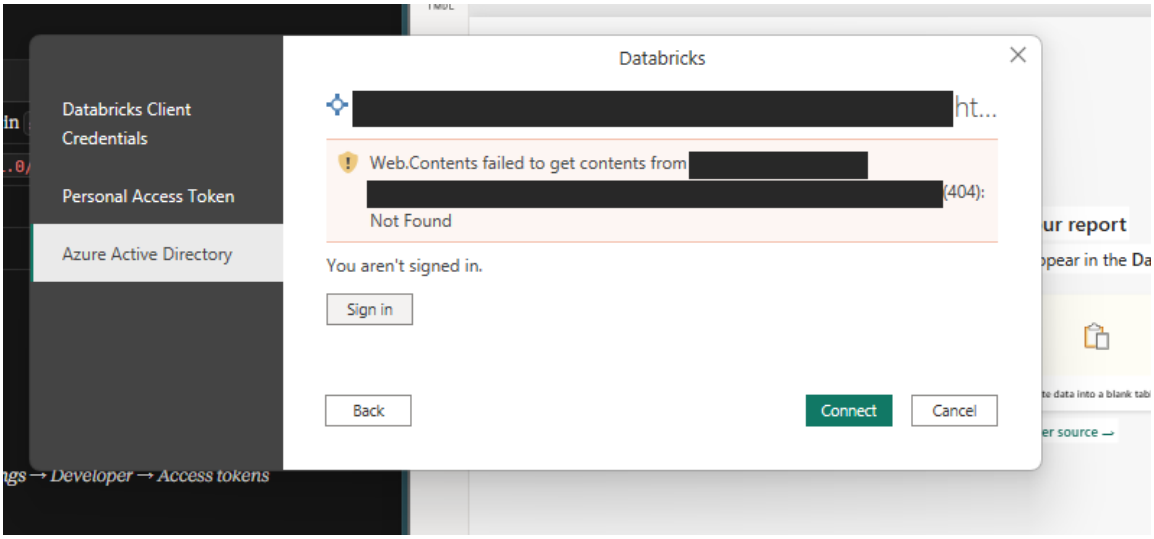


Figure 101. Wrong sign-in tab (Azure Active Directory) gives a 404 error. Use Personal Access Token.

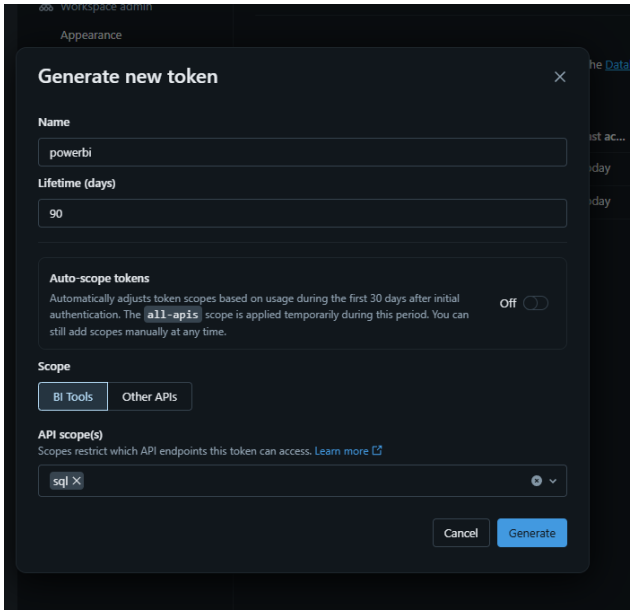


Figure 102. The Power BI token: BI Tools / sql, 90 days.

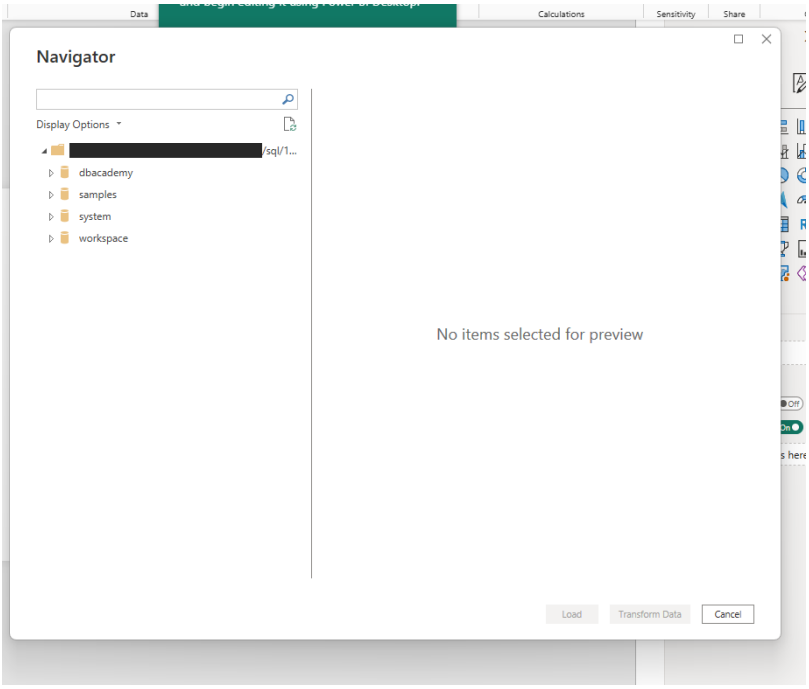


Figure 103. Navigator after connecting.

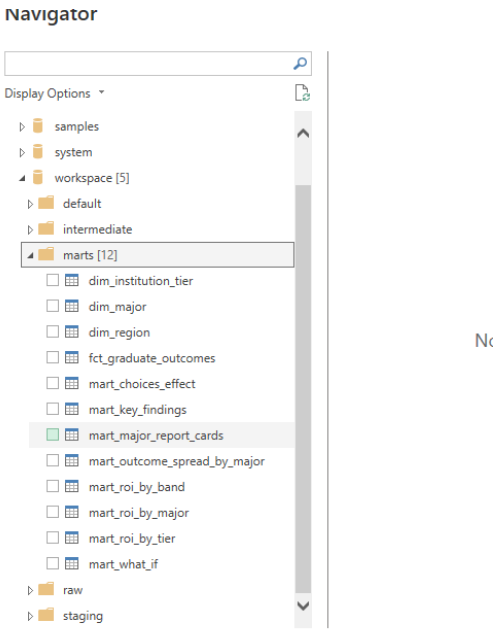


Figure 104. All marts visible under workspace, marts.

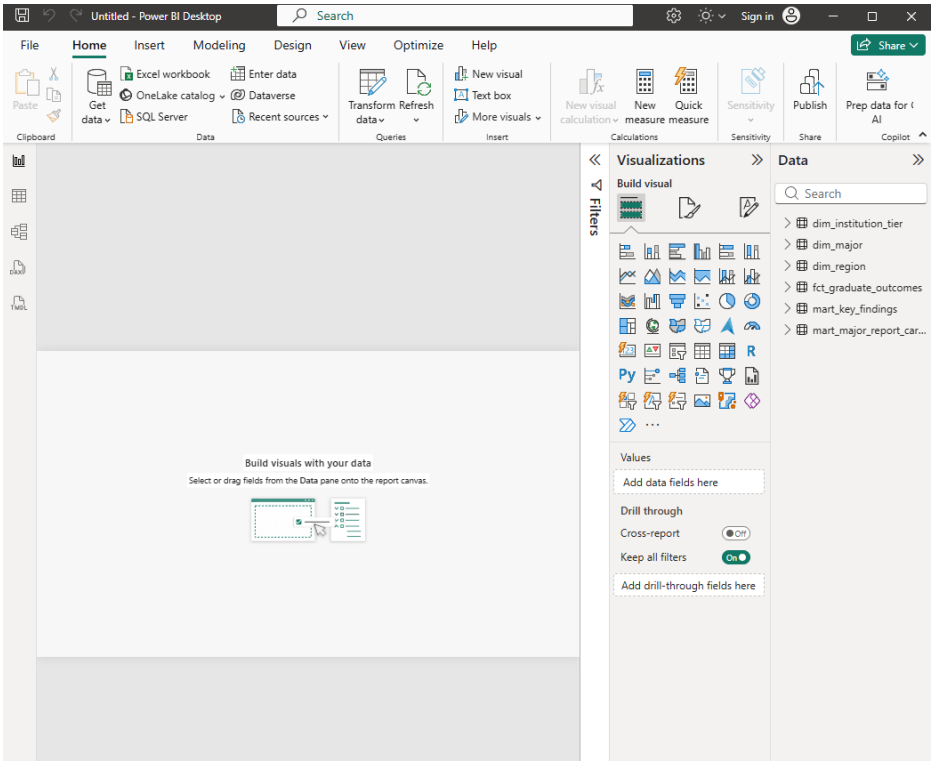


Figure 105. 6 tables loaded into Power BI.

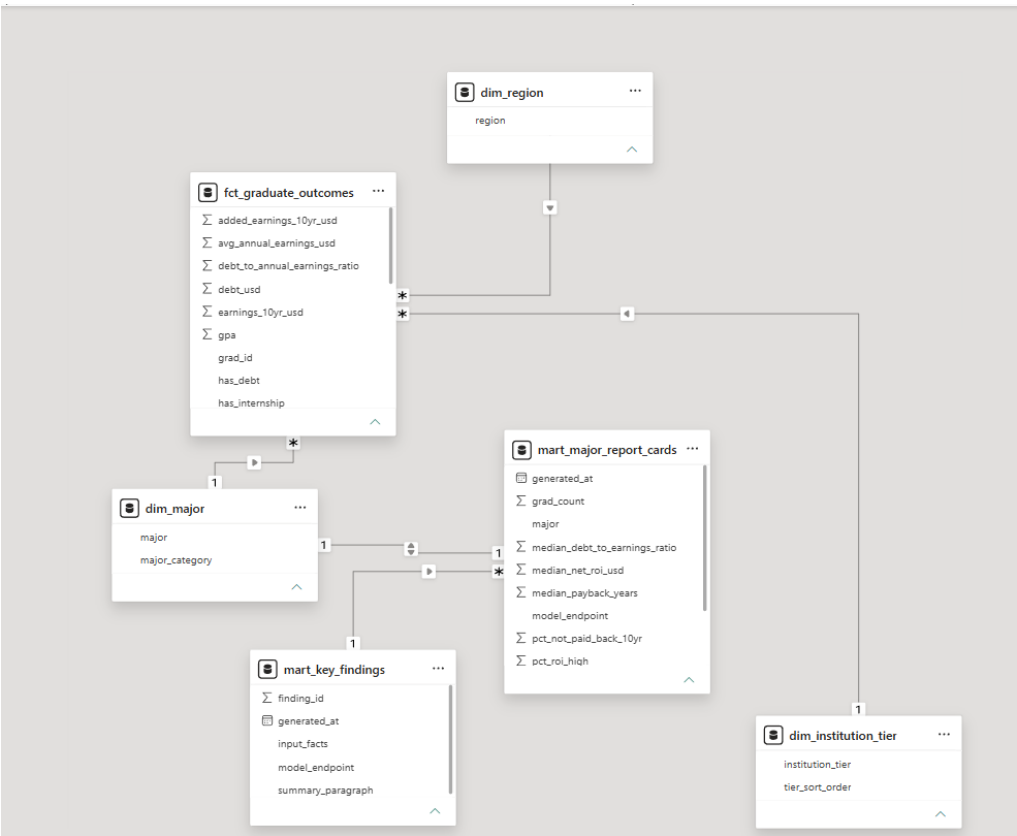


Figure 106. Model view: the star schema came through. One wrong auto link (key findings to report cards) to delete.

Starter DAX measures for when you return to it:

```
Graduates = COUNTROWS(fct_graduate_outcomes)
Median Net ROI = MEDIAN(fct_graduate_outcomes[net_roi_usd])
% Worse Off = DIVIDE(CALCULATE(COUNTROWS(fct_graduate_outcomes), fct_graduate_outcomes[is_positive
    _roi] = FALSE()), [Graduates])
Median Payback Years = MEDIAN(fct_graduate_outcomes[payback_years])
Median Debt = MEDIAN(fct_graduate_outcomes[debt_usd])
Internship Premium = CALCULATE([Median Net ROI], fct_graduate_outcomes[has_internship] = TRUE())
                   - CALCULATE([Median Net ROI], fct_graduate_outcomes[has_internship] = FALSE())
```

7. Impact map: if you change X, what happens to Y

If you change...	It affects...	What to do
The CSV (same columns, new rows)	Everything downstream on the next Job run: raw, staging, marts, AI text, dashboard (within 6 h cache)	Upload with a NEW file name; check the email
The CSV columns (renamed/removed)	stg_graduates breaks; tests fail; Job emails failure; dashboard keeps old data	Ask Claude Code to update staging and downstream models
stg_graduates	All later models (they all read it)	dbt build --target prod --select stg_graduates+
roi_band rules (int model)	mart_roi_by_major, report card takeaways, dashboard charts	Rebuild with + selector; review AI text
mart_roi_by_major	mart_major_report_cards, mart_key_findings, Majors page, Overview	Rebuild AI marts too
AI prompt or model endpoint	Wording of report cards/summary (~20 AI calls per build)	Trial on 3 majors with --vars report_card_majors
ai_model_endpoint var	Which model writes the text	One line in dbt_project.yml
Code merged to main	Next Job run uses it (Job pulls main from GitHub)	Always merge via PR
databricks_load_raw.sql	Task 1 of the Job	No begin/end blocks; test each statement separately
Dashboard code	Streamlit redeploys from main automatically	Run locally first
streamlit-dashboard token expires	Dashboard shows "credentials missing"	New token, paste into Streamlit app Secrets
dbt env-var token expires	Local dbt commands fail (the Job does not use it)	New token, setx DATABRICKS_TOKEN, reopen terminal
Databricks account inactive for long	Free Edition accounts can be deleted	Log in / run the job occasionally

8. Repository map and key files

```
college-roi-analytics/
  CLAUDE.md                       rules for Claude Code
  README.md                       public page: live link, architecture, automation, AI, data qualit
    y, roadmap
  SETUP_GUIDE.md                  original beginner setup guide
  data_dictionary.csv             column definitions
  requirements.txt                local Python packages
  .claude/commands/               /explain and /next-step slash commands
  ingestion/
    databricks_load_raw.sql       Job task 1: newest CSV -> workspace.raw.college_major_roi
    load_raw.py                   old local DuckDB loader (practice only)
  dbt/
    dbt_project.yml               layer folders -> schemas, vars (ai_model_endpoint, report_card_ma
    jors)
    profiles.yml                  dev (DuckDB, unused) + prod (Databricks via env vars)
    macros/generate_schema_name.sql   clean schema names (staging, intermediate, marts)
    models/staging/               _sources.yml, stg_graduates.sql
    models/intermediate/          int_graduates_enriched.sql
    models/marts/                 fct/dim star schema, 6 summary marts, 2 AI marts, YAML docs + tes
    ts
    tests/                        custom tests (ranges, reconciliation, one row per major...)
  dashboards/
```

```
    app.py                        navigation + synthetic-data footer
    db.py                         Databricks connection, 6-hour cache, formatting helpers
    charts.py                     shared chart styling
    views/                        overview, majors, colleges_and_choices, deep_dive
    .streamlit/config.toml        theme;  secrets.toml (gitignored);  secrets.toml.example
  orchestration/
    college_roi_refresh_job.yml   Databricks Job definition (no secrets)
```

Models at a glance

Model	Grain (one row per)	Feeds
stg_graduates (view)	graduate	int_graduates_enriched
int_graduates_enriched (view)	graduate	all marts
fct_graduate_outcomes	graduate	Power BI / star joins
dim_major / dim_institution_tier / dim_region	major (19) / tier (5) / region (4)	star schema
mart_roi_by_major	major	Majors page, AI report cards, key findings
mart_roi_by_tier	tier	Colleges page, key findings
mart_choices_effect	category x choice x with/without (32 rows)	Colleges page, key findings
mart_roi_by_band	GPA band / debt band	Deep dive
mart_outcome_spread_by_major	major	Deep dive (p10, p25, median, p75, p90)
mart_what_if	major x tier x internship x on-time (380)	What-if tool
mart_major_report_cards (AI)	major	Majors page
mart_key_findings (AI)	1 row	Overview

Final load SQL (ingestion/databricks_load_raw.sql)

```
-- Turns the NEWEST CSV in the landing volume into the raw table, whatever the file is called.
-- Runs automatically as Task 1 of the Databricks Job when a new CSV is uploaded,
-- or by hand: SQL Editor -> paste -> Run all.
-- To refresh the data, upload a CSV with a NEW name (e.g. college_major_roi_2026-11.csv):
-- the job's file arrival trigger only fires for new file names, not re-uploads of the same name.
-- Keep every statement free of begin/end blocks: the job's SQL file task splits this file on semi
    colons.

create schema if not exists workspace.raw;

create volume if not exists workspace.raw.landing;

-- Read every CSV in the folder, then keep only the rows from the most recently modified file.
-- _source_file records which file the rows came from.
create or replace table workspace.raw.college_major_roi as
with all_files as (
  select
    *,
    _metadata.file_path              as _source_file,
    _metadata.file_modification_time as _file_modified_at
  from read_files(
    '/Volumes/workspace/raw/landing/',
```

```
    format => 'csv',
    header => true,
    pathGlobFilter => '*.csv'
  )
),

newest_file as (
  select max_by(_source_file, _file_modified_at) as file_path
  from all_files
)

select * except (_file_modified_at), current_timestamp() as _loaded_at
from all_files
where _source_file = (select file_path from newest_file);

-- Check: stop here (with an error) if the load came out empty, so dbt never builds on a broken lo
    ad.
-- No fixed row count: a new CSV may have more or fewer graduates. dbt's tests check the rest.
select assert_true(count(*) > 0, 'Raw load failed: workspace.raw.college_major_roi is empty. Check
     the uploaded CSV.') as is_loaded
from workspace.raw.college_major_roi;
```

Job definition (orchestration/college_roi_refresh_job.yml)

```
# Databricks Job: college_roi_refresh
# Exported from the Databricks UI job definition (Declarative Automation Bundle format).
# When a new CSV lands in /Volumes/workspace/raw/landing/, it reloads the raw table
# from the newest CSV, then runs dbt build (models + tests, including the AI marts).
# Any failure stops the job and sends an email.
#
# No secrets in here. Fill in the two variables for your own workspace:
#   warehouse_id: SQL Warehouses -> your warehouse -> Overview -> ID
#   alert_email:  where failure (and success) emails go

variables:
  warehouse_id:
    description: ID of the serverless SQL warehouse both tasks run on
  alert_email:
    description: Email address for job notifications

resources:
  jobs:
    college_roi_refresh:
      name: college_roi_refresh
      max_concurrent_runs: 1        # two uploads can't run two builds at once
      queue:
```

```
        enabled: true

      # Run when a new file arrives (new file names only; re-uploading the same name won't trigger
    )
      trigger:
        pause_status: UNPAUSED
        file_arrival:
          url: /Volumes/workspace/raw/landing/
          min_time_between_triggers_seconds: 60
          wait_after_last_change_seconds: 60    # let a slow upload finish first

      email_notifications:
        on_failure:
          - ${var.alert_email}
        on_success:
          - ${var.alert_email}

      # Both tasks run the code on main in GitHub
      git_source:
        git_url: https://github.com/shaqran39/college-roi-analytics
        git_provider: gitHub
        git_branch: main
```

```
      tasks:
        # Task 1: newest CSV in the landing volume -> workspace.raw.college_major_roi
        - task_key: load_raw
          sql_task:
            file:
              path: ingestion/databricks_load_raw.sql
              source: GIT
            warehouse_id: ${var.warehouse_id}

        # Task 2: dbt build (only runs if load_raw succeeded; any failed test fails the job)
        - task_key: dbt_build
          depends_on:
            - task_key: load_raw
          dbt_task:
            project_directory: dbt
            commands:
              - dbt build             # no --target: Databricks writes its own profile for this ta
    sk
            catalog: workspace
            schema: default
            source: GIT
```

```
            warehouse_id: ${var.warehouse_id}
          environment_key: dbt-default
          max_retries: 0

      environments:
        - environment_key: dbt-default
          spec:
            environment_version: "6"
            dependencies:
              - dbt-databricks>=1.0.0,<2.0.0
```

Dashboard data access (dashboards/db.py)

```
"""Read-only access to the Databricks marts, with caching.

Credentials come from st.secrets["databricks"] (dashboards/.streamlit/secrets.toml locally,
the app's Secrets settings on Streamlit Cloud). Never hard-code them.
"""

import pandas as pd
import streamlit as st
from databricks import sql

MARTS_SCHEMA = "workspace.marts"

# results are kept for 6 hours, so clicking around the dashboard doesn't wake the warehouse
CACHE_SECONDS = 6 * 60 * 60


def _credentials() -> dict:
    """Return the Databricks credentials, or stop the page with a friendly message if they're miss
    ing."""
    creds = st.secrets.get("databricks", {})
    missing = [key for key in ("host", "http_path", "token") if not creds.get(key)]
    if missing:
```

```
        st.error(
            "Databricks credentials are missing: " + ", ".join(missing) + ". "
            "Fill them in `dashboards/.streamlit/secrets.toml` (locally) or the app's Secrets sett
    ings "
            "(Streamlit Cloud), then refresh."
        )
        st.stop()
    return creds


@st.cache_data(
    ttl=CACHE_SECONDS,
    show_spinner="Loading from Databricks. The first load can take up to 30 seconds while the ware
    house wakes up...",
)
def run_query(query: str) -> pd.DataFrame:
    """Run a SQL query and return the result as a DataFrame. Same query = cached result."""
    creds = _credentials()
    # a fresh connection per (rare, cached) query, so an idle warehouse can't leave us a dead conn
    ection
    with sql.connect(
        server_hostname=creds["host"].removeprefix("https://").rstrip("/"),
```

```
        http_path=creds["http_path"],
        access_token=creds["token"],
    ) as connection, connection.cursor() as cursor:
        cursor.execute(query)
        return cursor.fetchall_arrow().to_pandas()


def load_mart(table_name: str) -> pd.DataFrame:
    """Load a whole mart table, e.g. load_mart("mart_roi_by_major")."""
    return run_query(f"select * from {MARTS_SCHEMA}.{table_name}")


def money(value: float) -> str:
    """Compact US$ amount for headline numbers: 525800 -> 'US$525.8K', -35500 -> '−US$35.5K'."""
    sign = "−" if value < 0 else ""
    amount = abs(value)
    if amount >= 1_000_000:
        return f"{sign}US${amount / 1_000_000:.1f}M"
    if amount >= 1_000:
        return f"{sign}US${amount / 1_000:.1f}K"
    return f"{sign}US${amount:,.0f}"
```

```
def md_safe(text: str) -> str:
    """Escape $ so Streamlit markdown shows 'US$1,800' instead of reading '$...$' as LaTeX maths."
    ""
    return text.replace("$", r"\$")


def pretty(name: str) -> str:
    """'electrical_engineering' -> 'Electrical engineering'."""
    return name.replace("_", " ").capitalize()
```

CLAUDE.md (final)

```
# CLAUDE.md: project rules for Claude Code

Claude Code reads this file at the start of every session. Keep it up to date.

## What this project is
An end-to-end analytics project on `college_major_roi.csv`: 30,000 graduates (synthetic data), one
     row each, with major, institution tier, region, GPA, internship, cost, debt and 10-year earni
    ngs vs a high-school baseline.
Business question: **Which majors and college types actually pay off over 10 years, and how much d
    o internships, finishing on time and debt change the answer?**

Known data issues: 288 rows have institution_selectivity_pctile > 100 (capped in stg_graduates). L
    og any new issue in the README "Data quality" table.

## About the owner
Shaqran is learning AI-assisted data engineering. When you make changes:
- Explain what you did and why in plain, simple language
- Work in small steps and say what the next step is
- Never make big changes across many files without asking first

## Stack
- **Production: Databricks Free Edition.** Catalog `workspace`, raw table `workspace.raw.college_m
```

```
    ajor_roi`, dbt builds into `workspace.staging`, `workspace.intermediate` and `workspace.marts`
- **We only build on prod now.** Always use `--target prod`. Skip local DuckDB builds (`warehouse/
    college.duckdb`, target `dev`); they're kept only as an old practice setup
- Ingestion (cloud): CSVs uploaded to the volume `/Volumes/workspace/raw/landing`, tables created
    by `ingestion/databricks_load_raw.sql`
- Transformation: dbt Core in `dbt/` (run all dbt commands from inside the `dbt/` folder)
- AI in pipeline: Databricks `ai_query` (per-major report cards, findings summary). Only run these
     on Databricks, and guard them with `{{ config(enabled = target.type == 'databricks') }}` (not
     `target.name`: the scheduled Databricks Job's dbt task uses its own target name)
- Dashboard: Streamlit app in `dashboards/`, deployed to Streamlit Community Cloud (public)
- OS: Windows. Use PowerShell-friendly commands. The venv lives in `.venv`.
- Databricks credentials come from env vars DATABRICKS_HOST, DATABRICKS_HTTP_PATH, DATABRICKS_TOKE
    N

## Common commands
```
cd dbt
dbt debug --target prod   # check Databricks connection
dbt build --target prod   # run models + tests in Databricks
dbt build --target prod --select stg_graduates+   # one model and everything after it
dbt docs generate; dbt docs serve
```
```

```
## Free Edition limits to respect
- One small SQL warehouse, serverless only, max 5 concurrent job tasks
- Keep AI function calls cheap: test on `limit 100` before running on the full table
- Non-commercial use only

## dbt layer rules
| Layer | Folder | Prefix | Materialisation | Purpose |
|---|---|---|---|---|
| Staging | models/staging | `stg_` | view | 1:1 with a raw table. Rename, cast types, light clean
    ing. No joins. |
| Intermediate | models/intermediate | `int_` | view | Joins and business logic |
| Marts | models/marts | `fct_` / `dim_` / `mart_` | table | Star schema + summary tables for the
    dashboard |

## Conventions
- snake_case for everything. Timestamps end in `_at`, dates in `_date`, booleans start with `is_`
    / `has_`
- Every model gets an entry in a `.yml` file with a description
- Every primary key gets `unique` + `not_null` tests. Foreign keys get `relationships` tests
- Always reference other models with `ref()` and raw tables with `source()`, never hard-coded tabl
    e names
```

```
- Use CTEs: import CTEs at the top, then logic, then a final `select`
- After changing a model, run `dbt build --target prod --select <model>+` and fix failures before
    saying you're done

## Do not
- Commit anything in `data/raw/` or `warehouse/` (they're gitignored, the data is too big)
- Delete or overwrite raw data
- Commit secrets or credentials (e.g. Databricks tokens). Use environment variables.
```

9. Security and credentials

Credential	Scope	Used by	Stored in	Expires
dbt token	Other APIs / all-apis	Local dbt, Claude Code	Windows env vars (setx)	90 days
streamlit-dashboard	BI Tools / sql only	Public dashboard	Local secrets.toml (gitignored) + Streamlit Secrets	90 days (about mid-Jan 2027)
powerbi	BI Tools / sql only	Power BI (dropped)	Power BI credential store	Revoke it
GitHub login	gh CLI	Claude Code PRs	gh credential store	Until logged out

- **Never paste a token into a chat, screenshot or file that goes to Git.** During the build two tokens were pasted into chat; both must be treated as burnt and revoked.
- Revoke: Databricks, profile, Settings, Developer, Access tokens, bin icon. Then generate a new one with the same settings.
- Least privilege: anything public (dashboard) or third-party (Power BI) gets a BI Tools / sql token.
- Secrets file pattern: add the path to .gitignore first, then create the file, then check git status shows it as ignored (!!).
- The job YAML in the repo uses placeholders for the warehouse ID and email because the repo is public.
- No free-text AI box on the public site (prompt injection, cost).

10. Troubleshooting and lessons learned

Problem	Cause	Fix	Lesson
claude not recognised	Not installed / not on PATH	Install, add .local\bin to PATH, reopen terminal	Read installer warnings
Claude proposed rewriting a correct README	Stale context from earlier in its session	"Re-read it from disk"	AI only knows what it read
Claude did extra work unasked	Auto mode on	Shift+Tab to manual approve / plan mode	Stay the reviewer
Claude forgot the job already existed	Context compacted	Remind it; keep the big picture yourself	Use /clear + CLAUDE.md between phases
Old Olist model broke dbt	Leftover stg_orders.sql	Deleted it	Remove dead code when switching
Garbled key findings text	Streamlit treats $...$ as LaTeX	Escape $ (md_safe)	Test real text in the UI
AI said "the majority" for a median	Meaning error	Prompt rule: "half", never "most"	Check meaning, not only numbers
AI called risky majors "a solid choice"	AI judgement	Takeaway decided by SQL rules	Rules decide, AI words
AI marts would skip in the Job	target.name check	target.type == databricks	Prod may name things differently
Trigger did not fire on re-upload	Same file name = not new	Upload new names; load newest file	Know your trigger semantics
load_raw PARSE_SYNTAX_ERROR	SQL file task splits on ;	No begin/end; one statement per step	Test the way production runs it
Tokens pasted in chat	Habit	Revoke and regenerate	Keys go only into the tool or a password manager
Gray "PASS=56" text in prompt box	Claude Code autocomplete	Do not send it	Report only real results
C: drive almost full	Low disk	Disk Cleanup, uninstall unused apps	dbt/Python/Power BI fail oddly when disk is full
Retries default to 3	Job default + serverless auto-optimization	No retries, untick auto-optimization	Retries would repeat AI calls

11. Operations runbook

Refresh the data

1. Name the new CSV with a new name, e.g. college_major_roi_2026-11.csv (same columns).
2. Databricks, Catalog, workspace, raw, Volumes, landing, Upload to this volume.
3. Wait 3 to 5 minutes. Check Jobs & Pipelines, college_roi_refresh, Runs: "By file arrival", both green.
4. Optional: `select distinct _source_file from workspace.raw.college_major_roi;`
5. The dashboard shows new numbers within 6 hours (cache).

If you get a failure email

1. Open the run, click the red task, read the error (Diagnose error helps).
2. The dashboard is still showing the last good data, so there is no rush.
3. Paste the error to Claude Code with what you uploaded; ask it to fix, test the way the Job runs, PR and merge.
4. Re-run: Repair run on the failed run, or upload the file again under another new name.

Renew the dashboard token (before mid-January 2027)

1. Databricks: generate a new BI Tools / sql token, 90 days.
2. share.streamlit.io, your app, Settings, Secrets: replace the token line, Save.
3. Update your local dashboards/.streamlit/secrets.toml too. Revoke the old token.

Make a change to the code

1. Open PowerShell in the project folder, .venv\Scripts\activate, claude.
2. /clear, plan mode, describe the change, review the plan, approve.
3. Claude builds on a branch, runs dbt build --target prod, opens a PR; you review and merge.

Run things by hand

```
cd dbt
dbt debug --target prod
dbt build --target prod                                  # everything
dbt build --target prod --select int_graduates_enriched+  # one model and everything after it
cd ..\dashboards ; ..\.venv\Scripts\streamlit run app.py   # dashboard locally
```

12. How to work with Claude Code (the workflow that worked)

- **One phase per chat.** /clear before each phase. CLAUDE.md and README.md carry the memory.
- **Plan mode first.** Ask it to plan and "explain each column simply". Read it, push back, then approve.
- **Manual approval, not auto mode,** while learning. Auto mode made unrequested changes twice.
- **Trial before scale.** 3 majors before 19; throwaway tables before the real one.
- **Always end with "run dbt build --target prod, PR and merge".**
- **Paste the exact error** when something fails, plus your guess of the cause.
- **Verify claims.** Check results yourself (SQL Editor, dashboard, run history).
- **Secrets never go through Claude.** Ask it to create blank secret files; you fill them in.
- **Use /explain <file>** to learn what each model does and practise interview answers.

13. Findings (for the write-up and interviews)

Question	Answer from the data (synthetic, 30,000 graduates)
Best majors	Electrical engineering US$525.8K typical 10-yr net return, 1.3-yr payback; business analytics US$479.5K; chemical engineering US$466.7K; computer science US$464.8K
Worst majors	Fine arts -US$35.5K (88.9% worse off, 18.9-yr payback); humanities -US$11.2K (62.4% worse off); psychology about break-even (US$1.8K, 48.3% worse off)
College type	Ivy/elite US$304.4K but 20.3% worse off and US$80K debt; top public US$256.3K; mid tier US$216.8K; regional US$195.4K and fewest worse off (12.5%); for-profit US$68.8K, 36.8% worse off
Internships	+US$116.5K overall, but within category only +US$2.1K (arts) to +US$82.5K (STEM): confounded by major choice
Finishing on time	+US$55.2K overall; finishing late hurts most where gains are small (humanities 53.1% to 83.1% worse off)
GPA	About US$200K typical return in every GPA band: barely matters
Debt	Typical return falls from about US$240K (small debt) to about US$125K (US$150K+); no-debt slightly below small-debt
Region	US$193K to US$202K, about 16% worse off everywhere: barely matters

Watch out: All comparisons are between groups of graduates. They do not prove that a choice caused the difference.

14. Wrap-up checklist and future work

Done?	Item
[ ]	Revoke the powerbi token (and any other token that was ever pasted into a chat)
[ ]	Keep any .pbix file out of the repo
[ ]	Job notifications: keep Failure only
[ ]	Final README write-up, Power BI mentions removed, Phase 7b ticked (prompt in the chat reply)
[ ]	Add this PDF to the repo under docs/
[ ]	Calendar reminder: renew streamlit-dashboard token early January 2027
[ ]	Resume entry + LinkedIn post with the live link

Future work ideas: Power BI report on the star schema (measures above); a /new-dataset Claude Code command to adapt the pipeline to a new CSV; surrogate keys; CI that runs dbt tests on every PR; a Databricks Genie space for private natural-language questions; dbt docs site.

Appendix: list of figures

1. First attempt: typing claude failed with "not recognized" because Claude Code had not been installed yet (and pip install had been skipped).
2. Claude Code installed, but the installer warned that C:\Users\...\.local\bin was not on PATH.
3. Claude Code explains the project back and points to /next-step. Note "auto mode on" at the bottom: switch it off with Shift+Tab while learning.
4. Claude Code, working from an old copy, wrongly says the README still describes the old project. It had already been updated on disk.
5. Token dialog for the dbt token. Rename it (e.g. dbt-laptop) and set lifetime to 90 days so it does not expire mid-project.
6. Claude Code installed the GitHub CLI and handed the login step back to the human.
7. Phase 3 result: ROI bands summarised, early finding that about 62% of graduates have debt in every band. Claude also added data_dictionary.csv.
8. mart_roi_by_major, top 5: engineering and tech pay back in about 1.3 to 1.5 years.
9. Bottom 5: fine arts and humanities have negative typical returns and debt close to a full year of pay.
10. mart_choices_effect: the internship gap shrinks once you compare within the same field (a confounding variable). Claude notes group differences are not proof of cause.
11. Star schema plan, part 1: fct_graduate_outcomes columns and the decision to use readable names as keys.
12. Star schema plan, part 2: dimension tables and their keys.
13. Star schema plan, part 3: relationships tests and a reconciliation test (same 30,000 rows and same total ROI as the source).
14. Star schema plan, part 4: run, sample star join, PR and merge.
15. Star schema built: 27 tests passed, PR #3 merged.
16. The four tables of the star schema and what each holds.
17. First "star join": median ROI by region. Region makes almost no difference.
18. Phase 4 complete; Claude proposes Phase 5.
19. Claude Code suggests /clear to free 106K tokens of context before a new phase.
20. Phase 5 plan: 12 AI endpoints available; Llama 3.3 70B chosen.
21. Why Llama 3.3 70B: follows instructions, plain text output, cheap; model name kept in a dbt variable.
22. Files to add: two AI marts, their YAML, two row-count tests.
23. AI models only on prod; prompt built from one row of mart_roi_by_major.
24. Prompt rules: only given numbers, Australian English, temperature 0.2.
25. mart_key_findings: one paragraph, must say data is synthetic; input_facts stores the exact fact sheet.
26. Tests and the 3-major trial with a check-in.
27. Verification plan and cost (~23 AI calls planned).
28. Trial on 3 majors: every number matched the table.
29. Claude found three wording problems, including "the majority... within 1.3 years" (1.3 is the median, so it means half). A meaning error, not a number error.
30. Prompt tuning: length cap, double full stops, a lost regex backslash found and fixed.
31. The regex fix in mart_major_report_cards.sql.
32. Prompt rule changes (past tense, never "most", about 45 words).
33. Phase 5 merged (PR #4): 19 report cards + key findings paragraph, 16 tests pass.
34. Changes after review: takeaway labels moved to SQL rules because the AI called 42-48%-worse-off majors "A solid choice".
35. Quick fixes applied: "came out ahead / worse off" wording only.
36. CLAUDE.md updated to prod-only builds.
37. Dashboard plan: app.py, db.py with 6-hour cache, pages.
38. Plan details: secrets handling, token advice (make a dashboard-only token), slow first load, steps.
39. The read-only dashboard token: BI Tools scope with only sql.
40. .gitignore updated first, before the secrets file existed.
41. secrets.toml created blank; git status shows !! (ignored). Only secrets.toml.example is committed.
42. Quick-fix PR #5 created; dataviz guidance loaded for the dashboard.
43. db.py and app.py written.
44. Overview page, slim requirements.txt and theme.
45. App running at localhost:8501, waiting for credentials.
46. Summary of what was built and what is ignored by git.
47. Your step: paste host, HTTP path and token into secrets.toml yourself.
48. Bug: the key findings text is garbled because Streamlit reads text between two $ signs as LaTeX maths. Fixed by escaping $ (md_safe in db.py).
49. Three things done: $ fixed, PR #5 merged, pages built.
50. New marts (mart_roi_by_band, mart_outcome_spread_by_major, mart_what_if), 40 tests passing; GPA barely matters, more debt goes with lower return.
51. Windows quirk: restart Streamlit if a page looks out of date.
52. Overview: headline numbers and the AI key findings paragraph.

53. "Check the numbers": the exact fact sheet the AI was given (input_facts), for auditability.
54. Majors: median 10-year net return per major (red = worse off) and the selected major's report card.
55. All 19 AI report cards; the opening takeaway comes from SQL rules.
56. Colleges & choices: tier comparison and the internship effect by category.
57. Internship table view: group sizes shown next to each difference.
58. Deep dive: GPA and debt vs return (median line + middle-half band) and the spread of outcomes per major.
59. Polish done (PR #6). Pages folder renamed to views/ to stop Streamlit's automatic menu. Warning: C: drive nearly full.
60. Deployment steps for Streamlit Community Cloud.
61. README gets the live link (PR #7).
62. Reminders: check the link in a private window; the token expires after 90 days.
63. Job plan: fail loudly, failure email, no retries (retries would repeat AI calls), keep a YAML copy in the repo, test end to end.
64. PR #8: AI models now enabled on any Databricks target (target.type), plus an empty-file check in the load SQL. Both tested.
65. The empty-file check uses assert_true so the job stops before dbt runs on a broken load.
66. Claude Code's guide to creating the job, part 1.
67. Part 2: the dbt task values.
68. Part 3: trigger, safety settings, first run.
69. Empty task form: Type SQL file, Source Workspace (change to Git provider).
70. Git information: fill the branch box with main.
71. load_raw filled in correctly.
72. Add Task picker: search "dbt".
73. dbt task defaults: replace the three default commands with dbt build.
74. Schema dropdown: "analytics" does not exist, choose default.
75. Bottom of the dbt task: retries default to 3, change it.
76. Retry policy: No retries and untick serverless auto-optimization.
77. Both tasks linked, but Project directory still empty: type dbt.
78. Project directory dbt and command dbt build.
79. File arrival trigger: start Paused, set both waits to 1 minute.
80. Trigger types: File arrival is the one that fits "upload a CSV and everything updates".
81. First manual run: load_raw green, dbt_build still running. The Gmail "subscribed" email is only a confirmation, not a result.
82. dbt_build succeeded: 2m 50s, 90 queries, 30,498 rows written.
83. generated_at 07:58 UTC = 6:58 PM Sydney: the AI report cards were rewritten by the automated run.
84. Trigger panel: "Found no new files". Re-uploading a file with the same name does not count as a new file.
85. Claude tests the newest-file logic read-only and on a throwaway table.
86. Tests against a throwaway table so the real raw table stays safe.
87. The new script (version 1) used a begin...end scripting block.
88. Finds newest file with max_by over file modification time.
89. execute immediate with the file path as a parameter.
90. README gains a "Refreshing the data" section.
91. PR #9 merged.
92. Tested via a connection that handles scripting blocks, not the way the Job runs it.
93. File-arrival run: load_raw failed, so dbt_build was skipped. The dashboard kept the last good data (the safety net worked).
94. The error: PARSE_SYNTAX_ERROR at "begin declare newest_csv string". The SQL file task splits the file on semicolons.
95. Testing each statement separately, the way the Job runs it.
96. PR #10: four plain statements, newest-file-wins test (older.csv 100 rows vs newer.csv 50 rows, kept 50).
97. Trade-off: it reads every CSV before keeping the newest, so tidy the landing folder now and then.
98. SUCCESS: run launched "By file arrival", both tasks green in 2m 14s. The pipeline is fully automated.
99. Landing folder tidied: only the newest CSV kept.
100. Job definition exported to YAML with email and warehouse ID as placeholders (PR #11).
101. Wrong sign-in tab (Azure Active Directory) gives a 404 error. Use Personal Access Token.
102. The Power BI token: BI Tools / sql, 90 days.
103. Navigator after connecting.
104. All marts visible under workspace, marts.
105. 6 tables loaded into Power BI.
106. Model view: the star schema came through. One wrong auto link (key findings to report cards) to delete.